Number Theory for Competitive Programming

A Self-Contained Reference from Foundations to Advanced Techniques

Compiled for the Competitive Programming Community

October 29, 2026

Preface

This book is written for competitive programmers who want a single, self-contained reference for the number-theoretic techniques that appear over and over again in contests. The presentation assumes only that the reader can write code in C++ and is comfortable with high-school algebra. From there, we walk through every standard idea—divisibility and primes, the Euclidean algorithm and its extensions, modular arithmetic, the Chinese Remainder Theorem, Euler's totient, the Möbius function and Dirichlet convolution, primality testing, integer factorisation, quadratic residues, primitive roots, the discrete logarithm, the Number-Theoretic Transform, linear recurrences, Fibonacci numbers, and combinatorial number theory—and end with a dedicated chapter on *pattern recognition*: given the wording of a problem, how do you spot which of these techniques is the right one?

A few stylistic choices are worth flagging. First, every theorem of any importance is accompanied by a proof. Number theory is unforgiving: a tool whose proof you do not understand is a tool you will misuse. The competitive programmer who tries to apply Möbius inversion without understanding why $\sum_{d\mid n} \mu(d) = [n = 1]$ will eventually invert the wrong sum and waste a contest debugging a formula. Second, every algorithm is accompanied by C++ code that has been tested against the standard problem-setter expectations: `int` overflow is avoided with `int64_t`, multiplication of two 64-bit values is guarded by `__int128`, and every function listed in the appendix can be pasted directly into a contest solution. Third, each topic ends with one or two worked problems, with a brief statement and a sketch of the solution. Where a problem is well known we cite its canonical source (Codeforces, SPOJ, AtCoder, Project Euler, etc.).

The book is structured in four parts. **Part I** (Chapters 1–5) covers the foundations that every contestant must know cold: divisibility, primes, sieves, the Euclidean algorithm, and modular arithmetic. **Part II** (Chapters 6–11) builds on this with the workhorses of intermediate number theory: the Chinese Remainder Theorem, Euler's totient, primality testing, factorisation, the Möbius function, and Dirichlet convolution. **Part III** (Chapters 12–15) handles the

advanced machinery: quadratic residues and Tonelli–Shanks, primitive roots, the discrete logarithm via baby-step giant-step, and the Number-Theoretic Transform. **Part IV** (Chapters 16–24) closes with linear recurrences, Fibonacci identities, combinatorial number theory, miscellaneous tricks, generating functions, continued fractions, advanced topics (Lucas sequences, Beatty sequences, prime counting), a dedicated pattern-recognition chapter, and an appendix collecting every reusable template.

Read the book in order if you are learning, or jump to the relevant chapter via the table of contents if you are debugging a contest problem. Either way, may your solutions always be within time limit.

Contents

Preface **ii**

1 Introduction and Notation **1**

1.1 Why Number Theory? . . . 1

1.2 Notation . . . 2

1.3 Asymptotic Notation . . . 3

1.4 The Complexity Targets . . . 3

1.5 The Standard Moduli . . . 4

1.6 Common Identities . . . 5

1.7 How to Read This Book . . . 6

2 Divisibility and Primes **7**

2.1 The Division Algorithm . . . 7

2.2 Divisibility and GCD . . . 8

2.3 Primes and the Fundamental Theorem . . . 9

2.4 The Infinitude of Primes . . . 9

2.5 The Prime Number Theorem and Density of Primes . . . 10

2.6 Trial Division . . . 10

2.7 Pattern Recognition . . . 11

3 Sieve Methods **12**

3.1 The Sieve of Eratosthenes . . . 12

3.2 The Linear Sieve . . . 13

3.3 Computing Multiplicative Functions with the Linear Sieve . . . 15

3.4 The Segmented Sieve . . . 16

3.5 Pattern Recognition . . . 17

4 GCD and the Euclidean Algorithm 18

4.1 The Euclidean Algorithm . . . 18
4.2 Lamé's Theorem and the Complexity . . . 19
4.3 The Extended Euclidean Algorithm . . . 19
4.4 Linear Diophantine Equations . . . 20
4.5 Pattern Recognition . . . 21

5 Modular Arithmetic 22

5.1 The Ring $\mathbb{Z}/n\mathbb{Z}$ . . . 22
5.2 Fast Modular Exponentiation . . . 23
5.3 The Modular Inverse . . . 23
5.4 Linear Congruences . . . 24
5.5 Wilson's Theorem . . . 24
5.6 Pattern Recognition . . . 25

6 The Chinese Remainder Theorem and Garner's Algorithm 26

6.1 The Classical CRT . . . 26
6.2 Generalisation: Non-Coprime Moduli . . . 27
6.3 Garner's Algorithm . . . 28
6.4 Applications . . . 29
6.5 Pattern Recognition . . . 30

7 Euler's Totient Function 31

7.1 Definition and Multiplicativity . . . 31
7.2 The Closed Form . . . 31
7.3 Euler's Theorem . . . 32
7.4 Summation Identity . . . 33
7.5 Reducing Exponents Modulo φ . . . 33
7.6 Pattern Recognition . . . 34

8 Primality Testing 35

8.1 The Fermat Test and Carmichael Numbers . . . 35
8.2 Miller–Rabin: Setup . . . 35
8.3 Deterministic Miller–Rabin . . . 36

8.4 Performance . . . 37
8.5 Worked Problem: SPOJ PON — Prime or Not . . . 38
8.6 Pattern Recognition . . . 38

9 Integer Factorisation 39

9.1 Pollard's Rho: The Idea . . . 39
9.2 Floyd's Cycle Finding and the GCD Step . . . 40
9.3 Brent's Improvement . . . 41
9.4 Combining Miller–Rabin with Pollard's Rho . . . 42
9.5 Pollard's $p-1$ Algorithm . . . 43
9.6 Pattern Recognition . . . 43

10 The Möbius Function and Möbius Inversion 45

10.1 Definition and Properties . . . 45
10.2 The Fundamental Identity . . . 46
10.3 Möbius Inversion . . . 46
10.4 Application: Counting Coprime Pairs . . . 47
10.5 Application: Sum of GCDs . . . 48
10.6 Computing μ via the Linear Sieve . . . 48
10.7 Pattern Recognition . . . 49

11 Dirichlet Convolution 50

11.1 Definition and Basic Properties . . . 50
11.2 The Standard Functions . . . 50
11.3 Möbius Inversion as $\mu * 1 = \varepsilon$ . . . 51
11.4 Identities via Convolution . . . 51
11.5 Multiplicativity is Preserved . . . 52
11.6 Application: Sum of GCDs via Dirichlet . . . 53
11.7 Application: Counting $\sum_{\gcd=k}$ via Dirichlet . . . 53
11.8 Pattern Recognition . . . 53

12 Quadratic Residues and Tonelli–Shanks 54

12.1 Quadratic Residues . . . 54
12.2 The Legendre Symbol and Euler's Criterion . . . 55

12.3 Tonelli–Shanks Algorithm . . . 55
12.4 Cipolla's Algorithm . . . 57
12.5 Pattern Recognition . . . 58

13 Primitive Roots **59**
13.1 Orders of Elements . . . 59
13.2 Existence of Primitive Roots . . . 59
13.3 Finding a Primitive Root . . . 60
13.4 Applications . . . 61
13.5 Pattern Recognition . . . 61

14 Discrete Logarithm and BSGS **62**
14.1 The Discrete Logarithm Problem . . . 62
14.2 Baby-Step Giant-Step . . . 62
14.3 Pohlig–Hellman . . . 64
14.4 Discrete Roots . . . 65
14.5 Pattern Recognition . . . 65

15 Number Theoretic Transform (NTT) **66**
15.1 Roots of Unity in Finite Fields . . . 66
15.2 The Transform . . . 67
15.3 Iterative Cooley–Tukey . . . 67
15.4 Arbitrary Modulus via Triple-Modulus NTT . . . 69
15.5 Polynomial Operations via NTT . . . 69
15.6 Pattern Recognition . . . 70

16 Linear Recurrences and Berlekamp–Massey **71**
16.1 Matrix Exponentiation . . . 71
16.2 Berlekamp–Massey . . . 72
16.3 Kitamasa's Algorithm . . . 73
16.4 Pattern Recognition . . . 75

17 Fibonacci Numbers and Pisano Period **76**
17.1 Identities . . . 76

17.2 Fast Doubling . . . 77
17.3 Pisano Period . . . 77
17.4 Applications . . . 78
17.5 Pattern Recognition . . . 78

18 Combinatorial Number Theory **80**

18.1 Binomial Coefficients Modulo a Prime . . . 80
18.2 Kummer's Theorem: p-adic Valuation of Binomials . . . 81
18.3 Precomputation for Fast Binomials . . . 81
18.4 Stirling Numbers . . . 82
18.5 Catalan Numbers . . . 82
18.6 Applications . . . 83
18.7 Pattern Recognition . . . 83

19 Number Theory Tricks **85**

19.1 Josephus Problem . . . 85
19.2 Gray Code . . . 86
19.3 Fast Subset Enumeration (SOS DP) . . . 86
19.4 Popcount and Bit Manipulation . . . 86
19.5 Hensel Lifting (Newton's Method Modulo p^k) . . . 87
19.6 Pigeonhole in Modular Arithmetic . . . 88
19.7 Other Useful Identities . . . 88
19.8 Pattern Recognition . . . 89

20 Generating Functions and Formal Power Series **90**

20.1 Ordinary Generating Functions . . . 90
20.2 Exponential Generating Functions . . . 91
20.3 Closed Forms and the Rational Ansatz . . . 92
20.4 Newton Iteration for Polynomial Operations . . . 92
20.5 Worked Example: Counting Labeled Trees (Cayley) . . . 93
20.6 Worked Example: Subset Sum Generating Function . . . 93
20.7 Pattern Recognition . . . 93

21 Continued Fractions and Rational Approximation **94**

21.1 Definitions . . . 94

21.2 Convergents . . . 95

21.3 Pell's Equation . . . 96

21.4 Applications . . . 97

21.5 Pattern Recognition . . . 97

22 Advanced Topics: Lucas Sequences, Beatty Sequences, and Prime Counting **98**

22.1 Lucas Sequences . . . 98

22.2 Beatty Sequences . . . 99

22.3 Counting Primes: Lucy Hedgehog's Algorithm . . . 100

22.4 Bibliography Note . . . 101

22.5 Pattern Recognition . . . 102

23 Pattern Recognition Guide **103**

23.1 Signal Phrases and Likely Techniques . . . 103

23.2 Constraint-Driven Decision . . . 105

23.3 The Constraint Hierarchy . . . 106

23.4 The Consolidated Signature Table . . . 107

23.5 Final Heuristics . . . 108

24 Exercises and Practice Problems **109**

24.1 Divisibility, Primes, and Sieves (Chapters 2–3) . . . 109

24.2 GCD, Euclidean, and Modular Arithmetic (Chapters 4–5) . . . 110

24.3 Chinese Remainder Theorem (Chapter 6) . . . 110

24.4 Euler's Totient (Chapter 7) . . . 111

24.5 Primality Testing and Factorisation (Chapters 8–9) . . . 111

24.6 Möbius Function and Dirichlet Convolution (Chapters 10–11) . . . 112

24.7 Quadratic Residues and Primitive Roots (Chapters 12–13) . . . 112

24.8 Discrete Logarithm (Chapter 14) . . . 113

24.9 Number Theoretic Transform (Chapter 15) . . . 113

24.10 Linear Recurrences (Chapter 16) . . . 114

24.11 Fibonacci and Pisano Period (Chapter 17) . . . 114

24.12Combinatorial Number Theory (Chapter 18) . . . 114
24.13Number Theory Tricks (Chapter 19) . . . 115
24.14Generating Functions (Chapter 20) . . . 115
24.15Continued Fractions (Chapter 21) . . . 116
24.16Advanced Topics: Lucas Sequences, Beatty, and Prime Counting (Chapter 22) . . . 116
24.17General Pattern-Recognition Exercises . . . 117
24.18Closing Note on Practice . . . 117

25 Common Pitfalls, Edge Cases, and Implementation Tips 119

25.1 Integer Overflow . . . 119
25.2 Modular Arithmetic . . . 120
25.3 Sieve Memory and Cache . . . 120
25.4 Factorisation . . . 121
25.5 CRT and Garner . . . 121
25.6 NTT . . . 121
25.7 Berlekamp–Massey and Kitamasa . . . 122
25.8 Discrete Logarithm . . . 122
25.9 Recursion and Stack Overflow . . . 123
25.10Floating-Point . . . 123
25.11Time Limits . . . 123
25.12Common Heuristics . . . 124
25.13Final Tip: Build a Personal Template Library . . . 124

26 Selected Solutions 125

26.1 Solution to Exercise 24.M1 (Segmented Sieve) . . . 125
26.2 Solution to Exercise 24.M2 (Codeforces 231B) . . . 126
26.3 Solution to Exercise 24.M3 (Codeforces 803F) . . . 126
26.4 Solution to Exercise 24.H1 (Codeforces 906D) . . . 126
26.5 Solution to Exercise 24.H2 (Codeforces 506E) . . . 127
26.6 Solution to Exercise 24.H3 (Project Euler 78) . . . 127
26.7 Solution to Exercise 24.H1 (Continued Fractions: Project Euler 64) . . . 128
26.8 Solution to Exercise 24.H2 (Pell, Project Euler 66) . . . 129
26.9 Solution to Exercise 24.H1 (Lucy–Hedgehog) . . . 129

26.10Solution to Exercise 24.H4 (Matrix Exponentiation on Graphs) 129
26.11Closing Note on Solutions 130

27 Historical Notes and Further Reading **131**

27.1 Antiquity: Euclid, Diophantus, and the Foundations 131
27.2 The Renaissance and Early Modern Period 131
27.3 The 19th Century: Rigor and Abstraction 132
27.4 Late 19th and Early 20th Century 133
27.5 The Computer Era 133
27.6 Modern Era and Competitive Programming 134
27.7 Mathematical References 134
27.8 Online Resources 135
27.9 Closing Note 135

28 Appendix: Templates and Reference Tables **137**

28.1 Core Number Theory Templates 137
28.2 Primality Testing and Factorisation Templates 138
28.3 NTT Template 139
28.4 Berlekamp–Massey and Kitamasa Templates 140
28.5 BSGS Template 141
28.6 Complexity Cheat Sheet 142
28.7 NTT-friendly Primes 143
28.8 Common Competitive Programming Moduli 143
28.9 Closing Note 144

Chapter 1

Introduction and Notation

1.1 Why Number Theory?

Number theory is, in the words of Gauss, the *queen of mathematics*. It is also the branch of mathematics that appears most often in competitive programming, because its problems are easy to state, hard to solve, and admit algorithmic answers of striking elegance. A typical competitive programming problem in number theory looks deceptively short: "Given n, compute $f(n) \bmod 10^9 + 7$" or "Count the number of pairs (a, b) with $1 \le a, b \le N$ and $\gcd(a, b) = 1$." The constraint $n \le 10^{18}$ in the input description is the giveaway: a direct $O(n)$ loop will time out, and the contest expects you to apply some identity or algorithm that you have seen before. This book is the catalogue of those identities and algorithms.

There are three reasons number theory is so common in contests. First, the problems are short: a problem setter can describe a number-theoretic challenge in two or three lines, leaving the rest of the page free for the inevitable "Input" and "Output" sections. Second, the answers are precise: there is no need for floating-point comparison or random testing, which makes the problems easy to grade. Third, the techniques are *composable*: a single problem can require the contestant to combine modular arithmetic, the Chinese Remainder Theorem, fast exponentiation, and a sieve, all in one solution. The contestant who knows the techniques can recognise the patterns and assemble the solution; the contestant who does not, cannot.

This chapter sets the notation we will use throughout the book and gives a brief refresher on the asymptotic analysis we will need. The reader who is comfortable with both can skip directly to Chapter 2.

1.2 Notation

We collect the notational conventions used throughout the book.

Definition 1.1 (Integers, naturals)**.** $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ is the set of integers. $\mathbb{N} = \{1, 2, 3, \dots\}$ is the set of positive integers. We write $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$ when zero is included.

Definition 1.2 (Divisibility)**.** For $a, b \in \mathbb{Z}$ with $a \neq 0$, we say a *divides* b, written $a \mid b$, if there exists $k \in \mathbb{Z}$ with $b = ak$. If no such k exists we write $a \nmid b$.

Note that $a \mid b$ implies $|a| \leq |b|$ when $b \neq 0$, and that every nonzero integer divides 0. We will repeatedly use the trivial fact that if $a \mid b$ and $b \mid c$ then $a \mid c$ (transitivity), and that if $a \mid b$ and $a \mid c$ then $a \mid (bx + cy)$ for any $x, y \in \mathbb{Z}$ (linearity).

Definition 1.3 (Congruence)**.** For $a, b, n \in \mathbb{Z}$ with $n \geq 2$, we write $a \equiv b \pmod{n}$ if $n \mid (a - b)$. Equivalently, a and b have the same remainder when divided by n.

The congruence relation $\equiv \pmod{n}$ is an equivalence relation, and it respects $+$, $-$, $\times$, and (when the modulus is prime) division. We will study its properties carefully in Chapter 5.

Definition 1.4 (GCD and LCM)**.** For $a, b \in \mathbb{Z}$ not both zero, the greatest common divisor $\gcd(a, b)$ is the largest positive integer dividing both a and b. For $a, b > 0$, the least common multiple $\operatorname{lcm}(a, b)$ is the smallest positive integer divisible by both. We write (a, b) for $\gcd(a, b)$ when there is no risk of confusion.

Definition 1.5 (Primes)**.** A positive integer $p \geq 2$ is *prime* if its only positive divisors are 1 and p. Otherwise it is *composite*. By convention, 1 is neither prime nor composite.

We will use $\pi(x)$ for the prime-counting function: $\pi(x) = |\{p \leq x : p \text{ prime}\}|$. The Prime Number Theorem (proven by Hadamard and de la Vallée-Poussin in 1896) states that $\pi(x) \sim x/\ln x$, that is, $\pi(x) \ln x / x \to 1$ as $x \to \infty$. We will only need this asymptotic to estimate the density of primes, e.g. to choose between trial division and a sieve.

Definition 1.6 (Euler's totient)**.** $\varphi(n) = |\{1 \leq k \leq n : \gcd(k, n) = 1\}|$. The function φ is multiplicative and we will study it carefully in Chapter 7.

Definition 1.7 (Möbius function)**.** For $n \in \mathbb{N}$ with prime factorisation $n = p_1^{a_1} \cdots p_k^{a_k}$,

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ (-1)^k & \text{if } a_1 = a_2 = \cdots = a_k = 1 \text{ (i.e., } n \text{ is square-free)}, \\ 0 & \text{if some } a_i \geq 2 \text{ (i.e., } n \text{ is divisible by a square)}. \end{cases}$$

The Möbius function will be the subject of Chapter 10; its inversion formula is the single most important identity in the intermediate-number-theory toolkit of competitive programmers.

1.3 Asymptotic Notation

We use the standard Landau symbols. For functions $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$ we write $f(n) = \mathcal{O}(g(n))$ if there exist constants C, N with $f(n) \leq Cg(n)$ for all $n \geq N$. We write $f(n) = \Omega(g(n))$ if $g(n) = \mathcal{O}(f(n))$, and $f(n) = \Theta(g(n))$ if both. We will occasionally also write $f(n) = \widetilde{\mathcal{O}}(g(n))$ to mean $f(n) = \mathcal{O}(g(n) \log^k n)$ for some constant k, but the reader should be warned that contest problem setters use this notation inconsistently.

Note. When we say an algorithm runs in $\mathcal{O}(\sqrt{n})$ we mean it runs in time *at most* $C\sqrt{n}$ on inputs of size n, for some absolute constant C. The implicit constant matters in practice: a $\mathcal{O}(\sqrt{n})$ algorithm with $C = 10^3$ is slower than a $\mathcal{O}(n)$ algorithm with $C = 1$ for $n \leq 10^6$. We will give both asymptotic bounds and rough absolute estimates where possible.

1.4 The Complexity Targets

The following table collects the input sizes for which the standard time complexities are practical in C++ under typical contest time limits (1–2 seconds, single core):

Complexity	Comfortable input size
$\mathcal{O}(n!)$	$n \le 11$
$\mathcal{O}(2^n \cdot n)$	$n \le 22$
$\mathcal{O}(n^3)$	$n \le 700$
$\mathcal{O}(n^2)$	$n \le 5000$
$\mathcal{O}(n\sqrt{n})$	$n \le 10^5$
$\mathcal{O}(n \log n)$	$n \le 10^6$
$\mathcal{O}(n)$	$n \le 10^7$
$\mathcal{O}(\sqrt{n})$	$n \le 10^{14}$
$\mathcal{O}(\log n)$	n up to 10^{18} or beyond

Number-theoretic problems with $n \le 10^{18}$ are common; they cannot be solved by iterating up to n, but they can almost always be solved in $\mathcal{O}(\sqrt{n})$ (using some variant of the baby-step–giant-step idea) or in $\mathcal{O}(\log n)$ (using fast exponentiation, the Euclidean algorithm, or matrix exponentiation). The first skill a competitive programmer develops in number theory is the reflex that converts the constraint $n \le 10^{18}$ in the input into "find a log or $\sqrt{n}$ algorithm" in the brain.

1.5 The Standard Moduli

The following primes are the de facto standard moduli in competitive programming:

Modulus	Type	Notes
$10^9 + 7$	prime	The default. Just below 2^{30}, so the sum of two residues fits in a signed 32-bit integer.
$10^9 + 9$	prime	A common "second modulus" for hashing, used to reduce collisions.
998244353	prime	$119 \cdot 2^{23} + 1$. NTT-friendly: large power-of-two order.
167772161	prime	$5 \cdot 2^{25} + 1$. NTT-friendly, used in triple-modulus NTT.

469762049	prime	$7 \cdot 2^{26} + 1$.	NTT-friendly, used in triple-modulus NTT.

The modulus $10^9 + 7$ is the canonical choice for any problem that says "modulo a prime." For Number-Theoretic Transform problems, 998244353 is the standard choice because it admits a 2^{23}-th primitive root of unity, namely 3. The triple $(998244353, 167772161, 469762049)$ is used together with the Chinese Remainder Theorem to multiply polynomials modulo an arbitrary prime (the so-called MTT technique, see Chapter 15).

A practical warning: many problem setters use $10^9 + 7$ specifically to break solutions that take advantage of special structure. If a problem says "print modulo 998244353" instead, this is a strong hint that an NTT-based solution is expected.

1.6 Common Identities

We collect here identities that are used silently throughout the book. Each is easy to prove and the reader should verify them as an exercise.

Proposition 1.8 (Basic divisibility identities)**.** *For $a, b, c, m \in \mathbb{Z}$ with a, b not both zero and $m > 0$:*

1. $a \mid b \iff a \mid -b \iff |a| \mid |b|$ *(when $b \neq 0$).*
2. *If $a \mid b$ and $b \mid a$ then $a = \pm b$.*
3. *If $a \mid b$ and $b \mid c$ then $a \mid c$.*
4. *If $a \mid b$ and $a \mid c$ then $a \mid (xb + yc)$ for any $x, y \in \mathbb{Z}$.*
5. $a \equiv b \pmod{m} \iff a \equiv b \pmod{d}$ *for every $d \mid m$.*

Proposition 1.9 (Basic congruence arithmetic)**.** *If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then:*

1. $a + c \equiv b + d \pmod{m}$*;*
2. $a - c \equiv b - d \pmod{m}$*;*
3. $ac \equiv bd \pmod{m}$*;*
4. $a^k \equiv b^k \pmod{m}$ *for any $k \in \mathbb{N}$.*

We will use these identities without explicit reference.

1.7 How to Read This Book

Each chapter follows a uniform structure:

- **Definitions and statements**: the formal setup.
- **Theorems with proofs**: the mathematical core.
- **Algorithms**: pseudocode and C++ implementation.
- **Worked problems**: brief statement and solution sketch, drawn from Codeforces, SPOJ, AtCoder, and similar sources.
- **Pattern recognition**: a short subsection flagging the phrases in problem statements that should make you think of this chapter's technique.

The book is best read with a code editor open in another window: every algorithm listed in the appendix has been tested, and the reader who re-implements each one as they encounter it will finish the book with a personal library of templates ready for any contest.

Chapter 2

Divisibility and Primes

This chapter develops the elementary theory of divisibility and primes from the ground up. The Fundamental Theorem of Arithmetic—that every positive integer factors uniquely into primes—is the bedrock on which the rest of number theory rests, and its proof illustrates the role of the Euclidean algorithm even before we study that algorithm in its own right in Chapter 4.

2.1 The Division Algorithm

The first result is so basic that it is sometimes called the *division algorithm*, though it is a theorem rather than an algorithm: it asserts that dividing one integer by another always leaves a unique quotient and remainder.

Theorem 2.1 (Division algorithm)**.** *For any integers a and b with $b > 0$, there exist unique integers q and r such that $a = bq + r$ and $0 \le r < b$.*

Proof. We prove existence by the well-ordering principle. Consider the set $S = \{a - bk : k \in \mathbb{Z}, a - bk \ge 0\}$. The set S is nonempty (take $k = a$ if $a \ge 0$, or $k = a - b + 1$ if $a < 0$) and bounded below by 0. By the well-ordering principle, S has a smallest element r. Let q be the integer with $r = a - bq$. Then $a = bq + r$ and $r \ge 0$. We need to show $r < b$. Suppose for contradiction that $r \ge b$. Then $r - b = a - b(q + 1) \ge 0$ and $r - b < r$, contradicting the minimality of r.

For uniqueness, suppose $a = bq_1 + r_1 = bq_2 + r_2$ with $0 \le r_1, r_2 < b$. Then $b(q_1 - q_2) = r_2 - r_1$. The left side is divisible by b; the right side has $|r_2 - r_1| < b$. The only multiple of b strictly less than b in absolute value is 0, so $r_2 = r_1$ and consequently $q_1 = q_2$. □

In C++ the quotient and remainder are produced by the operators `/` and `%`. Be careful: for negative dividends, C++ rounds the quotient towards zero, so `(-7) % 3 == -1`, which is not in the range $[0, b)$. The standard competitive-programming workaround is

```
int rem(long long a, long long b) { return (a % b + b) % b; }
```

which always returns a value in $[0, b)$ for $b > 0$.

2.2 Divisibility and GCD

Definition 2.2 (GCD)**.** For $a, b \in \mathbb{Z}$ not both zero, the greatest common divisor $\gcd(a, b)$ is the unique positive integer d such that (i) $d \mid a$ and $d \mid b$, and (ii) every e with $e \mid a$ and $e \mid b$ satisfies $e \mid d$.

The two characterisations of gcd ("largest" and "divides every common divisor") are equivalent, but the second is more useful in proofs because it is phrased purely in terms of divisibility. We will compute gcd in Chapter 4 using the Euclidean algorithm; for now we record some of its basic properties.

Lemma 2.3. *For $a, b \in \mathbb{Z}$ not both zero and $x, y \in \mathbb{Z}$, $\gcd(a, b) \mid (ax + by)$. In particular, $\gcd(a, b)$ is the smallest positive integer that can be written as $ax + by$ for some $x, y \in \mathbb{Z}$.*

Proof. Let $d = \gcd(a, b)$. Then $d \mid a$ and $d \mid b$, so by linearity of divisibility $d \mid (ax + by)$ for any x, y. For the second part, let $S = \{ax + by : x, y \in \mathbb{Z}, ax + by > 0\}$. The set S is nonempty (e.g. $a \cdot 1 + b \cdot 0 = a$ or $a \cdot 0 + b \cdot 1 = b$, taking absolute values as needed). By well-ordering, S has a smallest element $m = ax_0 + by_0$. We claim $m = \gcd(a, b)$. By the first part $\gcd(a, b) \mid m$, so $\gcd(a, b) \le m$. For the reverse inequality, divide a by m using the division algorithm: $a = mq + r$ with $0 \le r < m$. Then $r = a - mq = a - (ax_0 + by_0)q = a(1 - x_0 q) - b(y_0 q)$, which is a linear combination of a and b. If $r > 0$ then $r \in S$ with $r < m$, contradicting minimality of m. So $r = 0$, meaning $m \mid a$. The same argument gives $m \mid b$. Hence m is a common divisor of a and b, so $m \le \gcd(a, b)$. Combined with the earlier inequality, $m = \gcd(a, b)$. □

Lemma 2.3 is the heart of Bézout's identity, which we will use in Chapter 4 to compute the modular inverse and in Chapter 6 to construct the solution of the Chinese Remainder Theorem.

2.3 Primes and the Fundamental Theorem

Definition 2.4 (Prime)**.** A positive integer $p \geq 2$ is *prime* if its only positive divisors are 1 and p. An integer $n \geq 2$ that is not prime is *composite*.

The number 1 is excluded from the primes by convention. The reason is the Fundamental Theorem of Arithmetic, which we state and prove below: if 1 were allowed to be prime, the factorisation $6 = 2 \cdot 3 = 1 \cdot 2 \cdot 3 = 1^2 \cdot 2 \cdot 3 = \dots$ would not be unique.

Lemma 2.5 (Euclid's lemma)**.** *If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.*

Proof. Suppose $p \nmid a$. Then $\gcd(p, a) = 1$ (since the only divisors of p are 1 and p, and $p \nmid a$). By Lemma 2.3 there exist x, y with $px + ay = 1$. Multiplying by b gives $pbx + aby = b$. Since $p \mid pbx$ (obviously) and $p \mid aby$ (since $p \mid ab$), we have $p \mid (pbx + aby) = b$. □

Euclid's lemma extends by induction: if $p \mid a_1 a_2 \cdots a_k$ then $p \mid a_i$ for some i.

Theorem 2.6 (Fundamental Theorem of Arithmetic)**.** *Every positive integer $n \geq 2$ can be written as a product of primes. Furthermore, this factorisation is unique up to the order of the factors.*

Proof. We prove existence by strong induction on n. The base case $n = 2$ is prime. For the inductive step, assume every integer $2 \leq m < n$ has a prime factorisation. If n is prime we are done; otherwise $n = ab$ with $2 \leq a, b < n$. By the inductive hypothesis both a and b factor into primes, and concatenating the two factorisations gives a factorisation of n.

For uniqueness, suppose $n = p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_\ell$ are two prime factorisations. By Euclid's lemma applied to the prime p_1 and the product $q_1 q_2 \cdots q_\ell$, we have $p_1 \mid q_i$ for some i. Reordering, $p_1 \mid q_1$, but q_1 is prime, so $p_1 = q_1$. Cancelling gives $p_2 \cdots p_k = q_2 \cdots q_\ell$. Repeating, we conclude $k = \ell$ and the two factorisations agree up to reordering. □

We will write the prime factorisation of n as $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$, and we will repeatedly use the shorthand $n = \prod_{p \mid n} p^{v_p(n)}$ where $v_p(n)$ is the largest exponent e with $p^e \mid n$. The function v_p is called the *p-adic valuation*; we will use it in Chapter 18 (Kummer's theorem).

2.4 The Infinitude of Primes

Theorem 2.7 (Euclid)**.** *There are infinitely many primes.*

Proof. Suppose for contradiction that $p_1, p_2, \dots, p_k$ is the complete list of all primes. Consider $N = p_1 p_2 \cdots p_k + 1$. By the Fundamental Theorem, N has some prime factor q. Then $q = p_i$ for some i, so $q \mid p_1 \cdots p_k$. But $q \mid N$, so $q \mid (N - p_1 \cdots p_k) = 1$, which is impossible since $q \geq 2$. Hence there is no finite list of all primes. $\square$

This argument is constructive in the sense that it shows how to produce a new prime from any finite list, but it is not useful in practice because N can be astronomically large compared to the primes it is built from.

2.5 The Prime Number Theorem and Density of Primes

We will need a single quantitative fact about primes: the density of primes near x is approximately $1/\ln x$. Formally:

Theorem 2.8 (Prime Number Theorem, Hadamard–de la Vallée-Poussin 1896)**.** $\pi(x) \sim \frac{x}{\ln x}$ *as* $x \to \infty$. *Equivalently,* $\pi(x) \ln x / x \to 1$.

We will not need the proof. For competitive programming the practical consequence is: for x around 10^9, the number of primes below x is about 5×10^7, and a randomly chosen integer near 10^9 is prime with probability $\approx 1/\ln(10^9) \approx 1/21$. This is why Pollard's rho in Chapter 9 takes about $\mathcal{O}(\sqrt{p})$ work to find a prime factor p rather than $\mathcal{O}(n)$: the smallest prime factor of a "random" large integer n is typically of size $\mathcal{O}(\log n)$.

2.6 Trial Division

The simplest way to find the prime factorisation of n is trial division: try each candidate divisor d from 2 upwards, dividing n by d as long as $d \mid n$, and continue with $d + 1$. The observation that makes this practical is that we only need to test divisors up to $\sqrt{n}$: if $n = ab$ with $a, b \geq 2$, then $\min(a, b) \leq \sqrt{n}$.

```
vector<long long> factorise(long long n) {
    vector<long long> f;
    for (long long d = 2; d * d <= n; ++d) {
        while (n % d == 0) { f.push_back(d); n /= d; }
    }
    if (n > 1) f.push_back(n);
    return f;
```

```
}
```

Listing 2.1: Trial division factorisation, $\mathcal{O}(\sqrt{n})$ time.

This runs in $\mathcal{O}(\sqrt{n})$, which is practical for n up to about 10^{14}. For n up to 10^{18} it is too slow; we will see Pollard's rho in Chapter 9 for that case.

Worked Problem (SPOJ `PRIME1` — Prime Generator). **Statement.** Given m, n with $1 \le m \le n \le 10^9$ and $n - m \le 10^5$, print every prime in $[m, n]$.

Solution sketch. Use the segmented sieve of Chapter 3. Sieve primes up to $\sqrt{10^9} < 32000$ once, then use them to sieve the small interval $[m, n]$.

2.7 Pattern Recognition

The phrases "given n, find its prime factors" or "is n prime?" obviously point to this chapter, but the same machinery applies to less direct phrasings:

Pattern Recognition. If a problem mentions constraints like "n has at most K distinct prime divisors" or "the square-free part of n," it is telling you that the prime factorisation matters. The reflex should be: factorise n (with trial division if n is small, or Pollard's rho if n is large), then work with the multiset of prime exponents. Many problems that are otherwise hard become trivial once you reduce them to operations on the prime factorisation.

Chapter 3

Sieve Methods

The sieve of Eratosthenes is the first algorithm most competitive programmers learn for enumerating primes below n. It is also the prototype for a whole family of algorithms that precompute information about every integer up to n, including the smallest prime factor, the Euler totient, and the Möbius function. This chapter covers the standard sieve, the linear sieve, the segmented sieve, and several variants that precompute multiplicative functions.

3.1 The Sieve of Eratosthenes

Algorithm 3.1 (Sieve of Eratosthenes)**.** Given $n \geq 2$, mark every number $2 \leq k \leq n$ as initially prime. For each $k = 2, 3, \ldots, \sqrt{n}$, if k is still marked prime, mark every multiple of k from k^2 to n as composite. The remaining marked numbers are primes.

```
vector<bool> sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; (long long)i * i <= n; ++i)
        if (is_prime[i])
            for (int j = i * i; j <= n; j += i)
                is_prime[j] = false;
    return is_prime;
}
```

Listing 3.1: Sieve of Eratosthenes, $\mathcal{O}(n \log \log n)$ time.

The starting point of the inner loop at i^2 rather than $2i$ is an optimisation: every composite $m \leq n$ has a smallest prime factor p, and we want to mark m exactly once, namely when $i = p$.

The smallest multiple of p that has not already been marked by a smaller prime is p^2 (since the multiples $2p, 3p, \dots, (p-1)p$ have a smaller prime factor). Starting at i^2 thus avoids redundant work.

Theorem 3.2. *The sieve of Eratosthenes runs in time $\Theta(n \log \log n)$ and uses $\Theta(n)$ memory.*

Proof. The number of times the inner statement `is_prime[j] = false` is executed is

$$\sum_{p \le n, p \text{ prime}} \left(\frac{n}{p} - p + 1 \right) = n \sum_{p \le n} \frac{1}{p} - \sum_{p \le n} p + \pi(n).$$

The harmonic series of primes $\sum_{p \le n} 1/p$ grows as $\log \log n + M$ (Mertens' theorem), and $\sum_{p \le n} p = \Theta(n^2 / \log n)$ is dominated by $n \log \log n$, so the total is $\Theta(n \log \log n)$. Memory is $\Theta(n)$ for the boolean array. □

For $n = 10^7$ this runs in well under a second; for $n = 10^8$ it is borderline, and for $n = 10^9$ the memory $\Theta(n)$ becomes prohibitive. For larger n, we use the segmented sieve of Section 3.4.

3.2 The Linear Sieve

The sieve of Eratosthenes is not "linear" in the sense that some composites are visited more than once. A different organisation of the work, due to Griedel and others, can sieve every number exactly once, giving a strict $\Theta(n)$ algorithm. The linear sieve has the additional advantage that it computes the smallest prime factor of every integer up to n in the same pass, which lets us factorise any n in $\mathcal{O}(\log n)$ time after a one-time $\Theta(n)$ sieve.

Algorithm 3.3 (Linear sieve)**.** Maintain an array `spf` (smallest prime factor) and a list `primes` of primes found so far. For $i = 2, 3, \dots, n$:

1. If $\text{spf}[i] = 0$ (i.e., i is prime), set $\text{spf}[i] = i$ and append i to `primes`.
2. For each p in `primes` such that $p \le \text{spf}[i]$ and $ip \le n$, set $\text{spf}[ip] = p$.

```
vector<int> spf;          // smallest prime factor
vector<int> primes;

void linear_sieve(int n) {
    spf.assign(n + 1, 0);
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) { spf[i] = i; primes.push_back(i); }
```

```
        for (int p : primes) {
            if (p > spf[i] || (long long)i * p > n) break;
            spf[i * p] = p;
        }
    }
}
```

Listing 3.2: Linear sieve, computing `spf` for all $i \le n$ in $\Theta(n)$.

Theorem 3.4. *The linear sieve runs in $\Theta(n)$ time and visits each composite exactly once.*

Proof. We show each composite $m \le n$ is visited exactly once. Write $m = p \cdot q$ where p is the smallest prime factor of m, so $q = m/p$ and $\mathrm{spf}[q] \ge p$ (since the smallest prime factor of q is at least p). In the outer loop, when $i = q$, we iterate p through the primes; the inner loop sets $\mathrm{spf}[q \cdot p] = p$ provided $p \le \mathrm{spf}[q]$. Hence $m = q \cdot p$ is set once when $i = q$. No other i' sets $\mathrm{spf}[m]$: any other factorisation $m = i' \cdot p'$ with $i' \ne q$ would have $p' \ne p$, but the inner loop stops at $p' > \mathrm{spf}[i']$ and the smallest prime factor of i' cannot be smaller than p (since p is the smallest prime factor of m and $p' \mid i'$ gives $p' \ge p$; equality would force $i' = q$). Hence each composite is set exactly once. □

Once spf is computed, any $n \le N$ can be factorised in $\mathcal{O}(\log n)$:

```
vector<pair<int,int>> factor(int n) {
    vector<pair<int,int>> f;
    while (n > 1) {
        int p = spf[n], c = 0;
        while (n % p == 0) { n /= p; ++c; }
        f.emplace_back(p, c);
    }
    return f;
}
```

Listing 3.3: Factorisation in $\mathcal{O}(\log n)$ after a one-time linear sieve.

This is extremely useful in problems that ask you to factor many integers up to $n \le 10^7$ in the input: a one-time $\Theta(n)$ precomputation gives $\mathcal{O}(\log n)$ per query.

3.3 Computing Multiplicative Functions with the Linear Sieve

The linear sieve can compute *any* multiplicative function (Chapter 11) on every integer up to n in $\Theta(n)$ time, provided we know how to compute the function on prime powers. The recipe is:

1. For i prime, compute $f(i)$ directly.
2. Maintain a second array $\texttt{low}[i]$, the highest power of $\text{spf}[i]$ that divides i (i.e., $\text{spf}[i]^{v_{\text{spf}[i]}(i)}$). When i is prime, $\texttt{low}[i] = i$.
3. For a composite $m = i \cdot p$ with $p = \text{spf}[m]$: if $p \mid i$ then $m = \texttt{low}[i] \cdot p \cdot (i/\texttt{low}[i])$, and $f(m) = f(\texttt{low}[i] \cdot p) \cdot f(i/\texttt{low}[i])$ since the two factors are coprime. If $p \nmid i$ then $m = i \cdot p$ with $\gcd(i, p) = 1$, so $f(m) = f(i)f(p)$.

For example, computing the Euler totient:

```
vector<int> phi;
void sieve_phi(int n) {
    spf.assign(n + 1, 0);
    phi.assign(n + 1, 0);
    phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) { spf[i] = i; primes.push_back(i); phi[i] = i - 1; }
        for (int p : primes) {
            if (p > spf[i] || (long long)i * p > n) break;
            spf[i * p] = p;
            if (p == spf[i]) phi[i * p] = phi[i] * p;         // p | i
            else             phi[i * p] = phi[i] * (p - 1);   // p !| i,
                coprime
        }
    }
}
```

Listing 3.4: Linear sieve computing φ for all $i \le n$ in $\Theta(n)$.

The rule $\varphi(p^k) = p^k - p^{k-1}$ and the multiplicativity $\varphi(ab) = \varphi(a)\varphi(b)$ when $\gcd(a, b) = 1$ (both proven in Chapter 7) translate directly into the two cases in the inner loop. The same recipe gives Möbius, divisor-count, divisor-sum, and so on; we will see concrete implementations as we encounter those functions.

3.4 The Segmented Sieve

When we need primes in a short interval $[m, n]$ with n up to (say) 10^{12} and $n - m \le 10^6$, the direct sieve is impractical (we cannot allocate an array of size 10^{12}). The fix is the segmented sieve: sieve the small interval $[\sqrt{n}]$ in $\mathcal{O}(\sqrt{n} \log\log \sqrt{n})$ time, then use those primes to sieve $[m, n]$ in $\mathcal{O}((n - m + 1) \log\log \sqrt{n})$ time.

```cpp
vector<long long> segmented(long long m, long long n) {
    long long lim = (long long)sqrt((double)n) + 1;
    // base sieve: primes up to sqrt(n)
    vector<bool> base(lim + 1, true);
    base[0] = base[1] = false;
    for (long long i = 2; i * i <= lim; ++i)
        if (base[i])
            for (long long j = i * i; j <= lim; j += i) base[j] = false;
    vector<long long> pr;
    for (long long i = 2; i <= lim; ++i) if (base[i]) pr.push_back(i);
    // segment sieve
    vector<bool> seg(n - m + 1, true);
    if (m == 1) seg[0] = false;
    for (long long p : pr) {
        long long start = max(p * p, (m + p - 1) / p * p);
        for (long long j = start; j <= n; j += p) seg[j - m] = false;
    }
    vector<long long> out;
    for (long long i = m; i <= n; ++i) if (seg[i - m]) out.push_back(i);
    return out;
}
```

Listing 3.5: Segmented sieve for primes in the interval m to n, $n - m \le 10^6$ and $n \le 10^{12}$.

The line `start` $= \max(p \cdot p, \lceil m/p \rceil \cdot p)$ chooses the first multiple of p in $[m, n]$ that is at least p^2, since smaller multiples of p have a prime factor smaller than p and were already handled. Computing $\lceil m/p \rceil \cdot p$ directly via integer arithmetic, $(m + p - 1)/p \cdot p$, is the standard idiom.

Worked Problem (SPOJ `PRIME1` — Prime Generator). **Statement.** Repeatedly query primes in $[m, n]$ with $n \le 10^9$, $n - m \le 10^5$.

Solution sketch. Segmented sieve as above. Each query is $\mathcal{O}(\sqrt{n} \log\log \sqrt{n} + (n - m) \log\log \sqrt{n}) = \mathcal{O}(\sqrt{n} + (n - m))$ in practice, dominated by 10^5 work per query.

3.5 Pattern Recognition

Pattern Recognition. The phrase "list every prime in $[m, n]$" with $n \leq 10^9$ almost always wants the segmented sieve. The phrase "precompute φ (or μ, τ, σ) for all $i \leq n$" with $n \leq 10^7$ almost always wants the linear sieve combined with the multiplicativity recipe. The phrase "for each query, factorise n" with many queries and $n \leq 10^7$ wants the linear sieve producing spf, then $\mathcal{O}(\log n)$ factorisation per query. Whenever the constraint on n is "up to 10^7 and you have many queries," the linear sieve is your friend.

Chapter 4

GCD and the Euclidean Algorithm

The Euclidean algorithm is the workhorse of elementary number theory. It computes $\gcd(a, b)$ in $\mathcal{O}(\log \min(a, b))$ time, and its extended form produces the Bézout coefficients x, y with $ax+by = \gcd(a, b)$. These coefficients are the building blocks for modular inverses, the Chinese Remainder Theorem, and the solution of linear Diophantine equations.

4.1 The Euclidean Algorithm

Lemma 4.1. *For $a, b \in \mathbb{Z}$ with $b > 0$, $\gcd(a, b) = \gcd(b, a \bmod b)$.*

Proof. Write $a = bq + r$ with $r = a \bmod b$. By Lemma 2.3, $\gcd(a, b)$ divides any linear combination $ax + by$. Since $r = a - bq = a \cdot 1 + b \cdot (-q)$, $\gcd(a, b) \mid r$, so $\gcd(a, b)$ is a common divisor of b and r, hence $\gcd(a, b) \leq \gcd(b, r)$. Conversely, $\gcd(b, r)$ divides b and r, and $a = bq + r$, so $\gcd(b, r)$ divides a, hence $\gcd(b, r) \leq \gcd(a, b)$. Combining, $\gcd(a, b) = \gcd(b, r)$. □

Iterating Lemma 4.1 gives the Euclidean algorithm:

```cpp
long long gcd(long long a, long long b) {
    while (b) { a %= b; swap(a, b); }
    return a < 0 ? -a : a;        // guard against negative input
}
```

Listing 4.1: Iterative Euclidean algorithm, $\mathcal{O}(\log \min(a, b))$ time.

The algorithm terminates because the sequence of second arguments $(b, a \bmod b, b \bmod (a \bmod b), \ldots)$ is strictly decreasing and nonnegative, so eventually reaches 0; at that point the first argument is the GCD. C++17 provides `std::gcd` in `<numeric>`, but the above is portable to any version.

4.2 Lamé's Theorem and the Complexity

How many iterations does the Euclidean algorithm take? The worst case is when the quotients q_i in the division chain are all 1, which happens exactly when a and b are consecutive Fibonacci numbers. This is the content of Lamé's theorem.

Theorem 4.2 (Lamé, 1844)**.** *If $a > b > 0$ and the Euclidean algorithm takes k divisions, then $b \geq F_{k+1}$, where F_n is the n-th Fibonacci number. In particular, $k \leq \log_\varphi b + 1 \approx 1.44 \log_2 b + 1$.*

Proof sketch. Let $r_0 = a, r_1 = b, r_2 = r_0 \bmod r_1, \ldots, r_k = \gcd(a,b), r_{k+1} = 0$. The sequence is $r_{i-1} = q_i r_i + r_{i+1}$ with $r_{i+1} < r_i$. We have $r_{k-1} \geq 2r_k$ (since $r_{k-1} = q_k r_k$ with $q_k \geq 2$, otherwise $r_{k+1} = r_{k-1} - q_k r_k \geq r_k$ would be nonzero, contradicting termination). Working backwards, $r_{k-2} = q_{k-1} r_{k-1} + r_k \geq r_{k-1} + r_k \geq F_3 r_k$ if $q_{k-1} \geq 1$. Continuing, $r_1 = b \geq F_{k+1} r_k \geq F_{k+1}$. Since $F_n \approx \varphi^n / \sqrt{5}$, we have $k \leq \log_\varphi b$. □

So the Euclidean algorithm on 64-bit integers takes at most about $1.44 \cdot 64 + 1 \approx 93$ divisions. In practice this is far less; the average number of divisions is about $\log b / \log 2 \cdot (12 \log 2 / \pi^2) \approx 0.84 \log_2 b$.

4.3 The Extended Euclidean Algorithm

The extended Euclidean algorithm computes (d, x, y) with $d = \gcd(a,b)$ and $ax + by = d$. The key observation is that if we keep track of the chain $r_0 = a = 1 \cdot a + 0 \cdot b$ and $r_1 = b = 0 \cdot a + 1 \cdot b$, then each subsequent remainder $r_{i+1} = r_{i-1} - q_i r_i$ is also a linear combination of a and b with integer coefficients, and the algorithm terminates at $r_k = d = x \cdot a + y \cdot b$.

```
long long ext_gcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1, d = ext_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return d;
}
```

Listing 4.2: Extended Euclidean algorithm. Returns $d = \gcd(a,b)$ with $ax + by = d$.

The recursion terminates at $b = 0$, where $d = a$ and $a \cdot 1 + 0 \cdot 0 = a$, so $x = 1, y = 0$. Unwinding the recursion, the new x is the old y_1, and the new y is $x_1 - q \cdot y_1$ where $q = a/b$.

Theorem 4.3 (Bézout's identity)**.** *For $a, b \in \mathbb{Z}$ not both zero, there exist $x, y \in \mathbb{Z}$ with $ax+by = \gcd(a,b)$. The extended Euclidean algorithm computes such (x, y).*

Proof. By induction on the depth of recursion. The base case $b = 0$ is verified directly. For the inductive step, suppose the recursive call ext_gcd$(b, a \bmod b)$ returns (d, x_1, y_1) with $bx_1 + (a \bmod b)y_1 = d$. Substituting $a \bmod b = a - (a/b)b$ gives $d = bx_1 + (a - (a/b)b)y_1 = ay_1 + b(x_1 - (a/b)y_1)$, so $x = y_1, y = x_1 - (a/b)y_1$ satisfies $ax + by = d$. □

4.4 Linear Diophantine Equations

Given $a, b, c \in \mathbb{Z}$, when does $ax + by = c$ have a solution in integers, and what is the general form of all solutions?

Theorem 4.4. *The equation $ax+by = c$ has an integer solution (x, y) if and only if $\gcd(a,b) \mid c$. If (x_0, y_0) is any particular solution, then the general solution is*

$$x = x_0 + \frac{b}{\gcd(a,b)} \cdot t, \qquad y = y_0 - \frac{a}{\gcd(a,b)} \cdot t, \qquad t \in \mathbb{Z}.$$

Proof. Necessity: $\gcd(a,b)$ divides $ax + by = c$. Sufficiency: write $d = \gcd(a,b)$. By Bézout, $ax' + by' = d$ for some x', y'. Multiplying by c/d gives $a(c/d \cdot x') + b(c/d \cdot y') = c$, so $x_0 = (c/d)x', y_0 = (c/d)y'$ is a solution.

For the general solution: if $ax + by = c = ax_0 + by_0$ then $a(x - x_0) = b(y_0 - y)$, i.e., $(a/d)(x-x_0) = (b/d)(y_0-y)$. Since $\gcd(a/d, b/d) = 1$, we have $b/d \mid (x-x_0)$, so $x = x_0+(b/d)t$ for some $t \in \mathbb{Z}$, and substituting gives $y = y_0 - (a/d)t$. □

```cpp
bool solve_diophantine(long long a, long long b, long long c,
                       long long &x, long long &y, long long &g) {
    g = ext_gcd(llabs(a), llabs(b), x, y);
    if (c % g != 0) return false;
    x *= c / g; y *= c / g;
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    return true;
}
// All solutions: x = x + (b/g)*t, y = y - (a/g)*t  for t in Z.
```

Listing 4.3: Solve $ax + by = c$; returns `false` if no solution.

Worked Problem (Linear Diophantine equations). **Statement.** (Codeforces 7C) Given a, b, c with $|a|, |b|, |c| \le 10^9$, find any integer solution (x, y) to $ax + by = c$, or report none.

Solution sketch. Run the extended Euclidean algorithm. If $\gcd(a, b) \nmid c$, output "No." Otherwise scale (x, y) by $c/\gcd(a, b)$ and output. Care: the result may overflow 32-bit, so use `long long`.

4.5 Pattern Recognition

Pattern Recognition. The phrase "solve $ax+by = c$ in integers" obviously points to Theorem 4.4. Less obvious: the phrase "given a, b, find the smallest positive c representable as $ax + by$" is the same problem with $c = \gcd(a, b)$ (Bézout). The phrase "how many solutions does $ax \equiv b \pmod m$ have?" (Chapter 5) is also a Diophantine problem in disguise: $ax - my = b$.

Chapter 5

Modular Arithmetic

Modular arithmetic is the algebra of congruences. Almost every competitive programming problem with the phrase "modulo $10^9 + 7$" lives in this chapter's world. This chapter develops the arithmetic of $\mathbb{Z}/n\mathbb{Z}$, fast modular exponentiation, the modular inverse, linear congruences, and Wilson's theorem.

5.1 The Ring $\mathbb{Z}/n\mathbb{Z}$

Definition 5.1. For $n \geq 2$, the set $\mathbb{Z}/n\mathbb{Z} = \{0, 1, 2, \ldots, n-1\}$ with operations $+_n$ (addition mod n) and $\cdot_n$ (multiplication mod n) is a commutative ring with unity. When n is prime it is a field, denoted $\mathbb{F}_n$.

The distinction between ring and field matters: in $\mathbb{Z}/4\mathbb{Z}$, the element 2 has no multiplicative inverse (since $2 \cdot 0 = 2 \cdot 2 = 0 \bmod 4$); in $\mathbb{F}_5 = \mathbb{Z}/5\mathbb{Z}$, every nonzero element has an inverse.

Proposition 5.2. *The element $a \in \mathbb{Z}/n\mathbb{Z}$ has a multiplicative inverse iff* $\gcd(a, n) = 1$.

Proof. If $\gcd(a, n) = 1$, by Bézout there exist x, y with $ax + ny = 1$. Modulo n this gives $ax \equiv 1 \pmod{n}$, so x is the inverse. Conversely, if $ax \equiv 1 \pmod{n}$ then $ax = 1 + kn$ for some k, so any common divisor of a and n divides 1. □

Hence in $\mathbb{Z}/n\mathbb{Z}$, the invertible elements are exactly those coprime to n, and their number is $\varphi(n)$ (Chapter 7).

5.2 Fast Modular Exponentiation

Raising a to the power b modulo m in $\mathcal{O}(\log b)$ multiplications is the single most important primitive in competitive programming number theory. The idea is to use the binary expansion of b: write $b = \sum_i b_i 2^i$, so $a^b = \prod_i a^{b_i 2^i} = \prod_{b_i=1} a^{2^i}$.

```
long long binpow(long long a, long long e, long long m) {
    a %= m; if (a < 0) a += m;
    long long r = 1 % m;
    while (e > 0) {
        if (e & 1) r = (__int128) r * a % m;
        a = (__int128) a * a % m;
        e >>= 1;
    }
    return r;
}
```

Listing 5.1: Binary exponentiation, $\mathcal{O}(\log e)$ multiplications mod m.

The use of `__int128` is critical when $m \approx 10^{18}$: the product of two residues can be up to $m^2 \approx 10^{36}$, which overflows 64-bit. For $m \le 10^9$, `long long` suffices, but `__int128` is always safe and the speed cost is negligible.

Proposition 5.3. binpow(a, e, m) *returns* $a^e \bmod m$ *in* $\mathcal{O}(\log e)$ *multiplications modulo* m.

Proof. Invariant: after iteration i (with e shifted right by i bits), $a \equiv a_0^{2^i} \pmod m$ where a_0 is the original a, and $r \equiv a_0^{e_0 \bmod 2^i} \pmod m$ where e_0 is the original e. When $e = 0$, $r \equiv a_0^{e_0} \pmod m$. The number of iterations is $\log_2 e + 1$. □

5.3 The Modular Inverse

Given a and m with $\gcd(a, m) = 1$, the modular inverse of a modulo m is the unique $a^{-1} \in \{1, \dots, m-1\}$ with $a \cdot a^{-1} \equiv 1 \pmod m$. We have three equivalent methods.

Method 1 (extended Euclidean). Run `ext_gcd(a, m, x, y)`, returning $d = 1$. Then $ax + my = 1$, so $ax \equiv 1 \pmod m$, and $x \bmod m$ is the inverse. This works for any m (not just primes).

Method 2 (Fermat). If $m = p$ is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod p$ by Fermat's little theorem (Theorem 7.5), so $a^{-1} = a^{p-2} \bmod p$, computed via `binpow`. This requires m to be prime but is otherwise the simplest method.

Method 3 (Euler). More generally, $a^{\varphi(m)} \equiv 1 \pmod{m}$ for $\gcd(a, m) = 1$ (Euler's theorem, Chapter 7), so $a^{-1} = a^{\varphi(m)-1} \bmod m$. This works for any m provided we can compute $\varphi(m)$.

```cpp
long long inv(long long a, long long m) {
    long long x, y;
    long long g = ext_gcd(((a % m) + m) % m, m, x, y);
    if (g != 1) return -1;          // inverse does not exist
    return ((x % m) + m) % m;
}
```

Listing 5.2: Modular inverse via extended Euclidean, valid for any m.

5.4 Linear Congruences

Theorem 5.4. *The congruence $ax \equiv b \pmod{m}$ has a solution iff $\gcd(a, m) \mid b$. In that case there are exactly $\gcd(a, m)$ incongruent solutions modulo m.*

Proof. $ax \equiv b \pmod{m}$ means $ax = b + my$ for some y, i.e., $ax - my = b$, a linear Diophantine equation. By Theorem 4.4, this has a solution iff $\gcd(a, m) \mid b$. If (x_0, y_0) is one solution, the general solution is $x = x_0 + (m/d)t$, $y = y_0 + (a/d)t$ where $d = \gcd(a, m)$. As t ranges over $\mathbb{Z}$, $x \bmod m$ takes exactly d distinct values (since the step m/d has order d modulo m). □

To find the smallest positive solution, solve the Diophantine equation and then take $x_0 \bmod (m/d)$.

5.5 Wilson's Theorem

Theorem 5.5 (Wilson)**.** *For a positive integer $n \geq 2$, $(n-1)! \equiv -1 \pmod{n}$ if and only if n is prime.*

Proof. ($\Rightarrow$) If n is prime, consider the elements $1, 2, \ldots, n-1$ of $\mathbb{F}_n^\times$. In $\mathbb{F}_n^\times$, the only elements that are their own inverses (i.e., $x^2 \equiv 1$) are ± 1, i.e., 1 and $n-1$. The remaining $n-3$ elements pair up with their inverses, and each pair contributes $x \cdot x^{-1} = 1$ to the product. Hence $(n-1)! = 1 \cdot 2 \cdots (n-1) \equiv 1 \cdot (n-1) \cdot 1 = n - 1 \equiv -1 \pmod{n}$.

($\Leftarrow$) If n is composite and $n > 4$, then n has a proper divisor d with $1 < d < n$. Then $d \mid (n-1)!$ (since $d \leq n-1$ appears as a factor). If $(n-1)! \equiv -1 \pmod{n}$ then $n \mid (n-1)! + 1$,

but $d \mid n$ and $d \mid (n-1)!$, so $d \mid 1$, contradiction. The case $n = 4$ is checked directly: $3! = 6 \equiv 2 \pmod 4$, not -1. $\square$

Wilson's theorem is rarely useful as a primality test (the factorial grows too fast), but it gives a slick proof that for prime p, the polynomial $x^{p-1} - 1 \equiv \prod_{a=1}^{p-1}(x - a) \pmod p$.

5.6 Pattern Recognition

Pattern Recognition. The phrase "print the answer modulo $10^9 + 7$" is the universal signal that the answer is to be computed in the ring $\mathbb{F}_{10^9+7}$. Every arithmetic operation in the computation must reduce mod p to avoid overflow. If divisions are needed, use the modular inverse via Fermat. If n up to 10^{18} appears, use `__int128` for all multiplications to avoid overflow. The phrase "compute $a^b \bmod m$" obviously wants binary exponentiation. The phrase "how many solutions does $ax \equiv b \pmod m$ have?" wants Theorem 5.4.

Chapter 6

The Chinese Remainder Theorem and Garner's Algorithm

The Chinese Remainder Theorem (CRT) is the tool that decomposes the ring $\mathbb{Z}/n\mathbb{Z}$ into a product of smaller rings whenever n is composite. Its constructive form (Garner's algorithm) is the way we reconstruct an integer from its residues modulo several pairwise coprime moduli—the standard technique behind multi-modulus NTT and large-integer arithmetic in contests.

6.1 The Classical CRT

Theorem 6.1 (Chinese Remainder Theorem)**.** *Let $m_1, m_2, \ldots, m_k$ be pairwise coprime positive integers, and let $M = m_1 m_2 \cdots m_k$. For any integers $a_1, a_2, \ldots, a_k$, the system*

$$x \equiv a_i \pmod{m_i} \qquad (i = 1, 2, \ldots, k)$$

has a unique solution modulo M.

Proof. Existence. Define $M_i = M/m_i$. Since the m_i are pairwise coprime, $\gcd(M_i, m_i) = 1$, so by Bézout there is a y_i with $M_i y_i \equiv 1 \pmod{m_i}$. Set

$$x = \sum_{i=1}^{k} a_i M_i y_i \pmod{M}.$$

Then $x \equiv a_i M_i y_i \equiv a_i \cdot 1 \equiv a_i \pmod{m_i}$ (the other terms vanish mod m_i because $m_i \mid M_j$ for $j \neq i$).

Uniqueness. If x and x' are both solutions, then $x \equiv x' \pmod{m_i}$ for all i, so $m_i \mid (x - x')$. Since the m_i are pairwise coprime, $M \mid (x - x')$, so $x \equiv x' \pmod{M}$. □

The constructive proof gives an explicit algorithm, but the formula $x = \sum a_i M_i y_i$ has a numerical pitfall: each M_i can be as large as M, so the sum is up to kM and may overflow 64-bit. The standard competitive-programming implementation processes the moduli pairwise, reducing after each step:

```
long long crt2(long long a1, long long m1, long long a2, long long m2) {
    long long p, q; long long g = ext_gcd(m1, m2, p, q);
    // m1*p + m2*q = 1
    long long M = m1 * m2;
    long long x = (a1 * (m2 % M) % M * (q % M) % M
                  + a2 * (m1 % M) % M * (p % M) % M) % M;
    return (x % M + M) % M;
}
```

Listing 6.1: Pairwise CRT: combine $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$ assuming $\gcd(m_1, m_2) = 1$.

To combine k congruences, fold over the list with this binary operation. The result is the unique solution in $[0, M)$ where $M = \prod m_i$.

6.2 Generalisation: Non-Coprime Moduli

What if the m_i are not pairwise coprime? Then the system has a solution iff a consistency condition is met.

Theorem 6.2. *The system $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$ has a solution iff $a_1 \equiv a_2 \pmod{\gcd(m_1, m_2)}$. If a solution exists, it is unique modulo $\operatorname{lcm}(m_1, m_2)$.*

Proof. The first congruence says $x = a_1 + m_1 k$ for some $k \in \mathbb{Z}$. Substituting into the second, $a_1 + m_1 k \equiv a_2 \pmod{m_2}$, i.e., $m_1 k \equiv a_2 - a_1 \pmod{m_2}$. This is a linear congruence in k, which by Theorem 5.4 has a solution iff $\gcd(m_1, m_2) \mid (a_2 - a_1)$, i.e., $a_1 \equiv a_2 \pmod{\gcd(m_1, m_2)}$. If a solution exists, the solutions for k are spaced by $m_2 / \gcd(m_1, m_2)$, so the solutions for $x = a_1 + m_1 k$ are spaced by $m_1 m_2 / \gcd(m_1, m_2) = \operatorname{lcm}(m_1, m_2)$. □

To combine k congruences in general: process them pairwise, checking the consistency condition at each step and updating the modulus to lcm of the two.

```cpp
bool crt_inconsistent(long long a1, long long m1, long long a2, long long m2,
                      long long &a, long long &m) {
    long long p, q;
    long long g = ext_gcd(m1, m2, p, q);
    if ((a2 - a1) % g != 0) return false;
    long long l = m1 / g * m2;
    a = (a1 + m1 * ((a2 - a1) / g % (m2 / g) * p % (m2 / g) + m2 / g)) % l;
    m = l;
    return true;
}
```

Listing 6.2: CRT for non-coprime moduli, returns `false` on inconsistency.

6.3 Garner's Algorithm

The formula $x = \sum a_i M_i y_i$ in the classical CRT proof computes x directly from residues. Garner's algorithm reorganises the computation as a mixed-radix representation, which is much more numerically convenient when the moduli are large (since no intermediate value exceeds M).

Suppose $m_1, m_2, \ldots, m_k$ are pairwise coprime and $M = \prod m_i$. Write

$$x = v_0 + v_1 m_1 + v_2 m_1 m_2 + \cdots + v_{k-1} m_1 m_2 \cdots m_{k-1}$$

where $0 \le v_i < m_{i+1}$. Reducing modulo m_1 gives $v_0 \equiv x \equiv a_1 \pmod{m_1}$, so $v_0 = a_1 \bmod m_1$. Then reduce $x - v_0$ modulo m_2: since m_1 is invertible mod m_2, $v_1 \equiv (x - v_0) m_1^{-1} \pmod{m_2}$, so $v_1 = (a_2 - v_0) m_1^{-1} \bmod m_2$. Continue inductively.

```cpp
long long garner(vector<long long> a, vector<long long> m) {
    int k = a.size();
    // precompute inverse of m[i] modulo m[j] for i < j
    vector<vector<long long>> inv(k, vector<long long>(k));
    for (int i = 0; i < k; ++i)
        for (int j = i + 1; j < k; ++j)
            inv[i][j] = inv_mod(m[i], m[j]);
    vector<long long> v(k);
    for (int i = 0; i < k; ++i) {
        v[i] = a[i];
```

```
        for (int j = 0; j < i; ++j) {
            v[i] = (v[i] - v[j] % m[i] + m[i]) % m[i];
            v[i] = (__int128) v[i] * inv[j][i] % m[i];
        }
    }
    long long x = 0, P = 1;
    for (int i = 0; i < k; ++i) {
        x = (x + (__int128) v[i] * P);
        P *= m[i];
    }
    return x;
}
```

Listing 6.3: Garner's algorithm: reconstruct x from (a_i, m_i) pairs.

Garner's algorithm is the standard reconstruction step in triple-modulus NTT (Chapter 15): multiply polynomials modulo three primes, then combine the three result polynomials with Garner to recover the true (un-modular) coefficients.

6.4 Applications

1. **Multi-modulus arithmetic.** To work with integers up to $\sim 10^{36}$ (product of three 10^{12}-ish primes), represent them by their residues modulo several primes. All arithmetic happens on residues; the final answer is reconstructed via Garner.

2. **NTT reconstruction.** The Number-Theoretic Transform of Chapter 15 multiplies polynomials modulo a single prime. To multiply polynomials with arbitrarily large coefficients, run NTT modulo three suitable primes, then combine the three resulting coefficient vectors via CRT.

3. **Solving linear congruences.** When the modulus is composite with known factorisation, the congruence splits via CRT into one congruence per prime-power factor, which can be easier to solve.

Worked Problem (Codeforces 498C — Multiple moduli). **Statement.** Given several congruences $x \equiv a_i \pmod{m_i}$ with various (not necessarily coprime) moduli, find x if it exists.

Solution sketch. Process pairwise with Theorem 6.2. If any pair is inconsistent, no solution exists; otherwise the unique solution modulo the LCM of all moduli is the answer.

6.5 Pattern Recognition

Pattern Recognition. Phrases like "$x \bmod m_1 = a_1, x \bmod m_2 = a_2, \ldots$" always point to CRT. The subtler signals: "restore the answer from its residues modulo $\{p_1, p_2, p_3\}$" wants Garner; "find x with $x \equiv a \pmod{m_1}$ and $x \equiv b \pmod{m_2}$ and $\gcd(m_1, m_2) \neq 1$" wants the generalised CRT; and the problem "multiply polynomials with coefficients up to 10^{18}" wants NTT under three primes plus Garner (see Chapter 15).

Chapter 7

Euler's Totient Function

Euler's totient $\varphi(n)$ counts the integers in $\{1, 2, \ldots, n\}$ that are coprime to n. It is the first multiplicative function we study, and its two most important applications in competitive programming are Euler's theorem $a^{\varphi(m)} \equiv 1 \pmod{m}$ (which generalises Fermat's little theorem to composite m) and the identity $\sum_{d|n} \varphi(d) = n$ (which lets us transform sums involving gcd).

7.1 Definition and Multiplicativity

Definition 7.1. $\varphi(n) = |\{1 \le k \le n : \gcd(k, n) = 1\}|$.

For example, $\varphi(1) = 1, \varphi(2) = 1, \varphi(3) = 2, \varphi(4) = 2, \varphi(5) = 4, \varphi(6) = 2, \varphi(7) = 6, \varphi(8) = 4$.

Theorem 7.2. *φ is multiplicative: if $\gcd(a, b) = 1$ then $\varphi(ab) = \varphi(a)\varphi(b)$.*

Proof. The integers in $\{1, 2, \ldots, ab\}$ that are coprime to ab are exactly those coprime to both a and b. Consider the map $\phi : \mathbb{Z}/ab\mathbb{Z} \to \mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ given by $x \mapsto (x \bmod a, x \bmod b)$. By the Chinese Remainder Theorem (Theorem 6.1), ϕ is a bijection. Restricting to residues coprime to ab, ϕ gives a bijection between $(\mathbb{Z}/ab\mathbb{Z})^\times$ and $(\mathbb{Z}/a\mathbb{Z})^\times \times (\mathbb{Z}/b\mathbb{Z})^\times$, which has size $\varphi(a)\varphi(b)$. Hence $\varphi(ab) = \varphi(a)\varphi(b)$. □

7.2 The Closed Form

Theorem 7.3. *If $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ is the prime factorisation of n, then*

$$\varphi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right).$$

In particular, $\varphi(p^k) = p^k - p^{k-1}$ *for prime* p *and* $k \geq 1$.

Proof. By multiplicativity (Theorem 7.2), $\varphi(n) = \prod_i \varphi(p_i^{a_i})$, so it suffices to compute $\varphi(p^k)$. The integers in $\{1, 2, \ldots, p^k\}$ that are NOT coprime to p^k are exactly the multiples of p: there are p^{k-1} of them (namely $p, 2p, \ldots, p^{k-1} \cdot p = p^k$). Hence $\varphi(p^k) = p^k - p^{k-1} = p^k(1 - 1/p)$, and the formula follows. □

```cpp
long long phi(long long n) {
    long long r = n;
    for (long long p = 2; p * p <= n; ++p) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            r -= r / p;
        }
    }
    if (n > 1) r -= r / n;        // remaining prime factor
    return r;
}
```

Listing 7.1: Compute $\varphi(n)$ from prime factorisation, $\mathcal{O}(\sqrt{n})$.

This is $\mathcal{O}(\sqrt{n})$, sufficient for n up to 10^{14}. To compute φ for many n with $n \leq 10^7$, use the linear sieve of Chapter 3.

7.3 Euler's Theorem

Theorem 7.4 (Euler)**.** *If* $\gcd(a, m) = 1$ *and* $m \geq 2$*, then* $a^{\varphi(m)} \equiv 1 \pmod{m}$.

Proof. Consider the multiplicative group $(\mathbb{Z}/m\mathbb{Z})^\times$, which has order $\varphi(m)$ by definition. Lagrange's theorem (a basic result in group theory) says that for any element g of a finite group G, $g^{|G|} = e$ (the identity). Applied to $a \in (\mathbb{Z}/m\mathbb{Z})^\times$, this gives $a^{\varphi(m)} \equiv 1 \pmod{m}$.

For a more elementary proof avoiding group theory: the elements of $(\mathbb{Z}/m\mathbb{Z})^\times$ are $r_1, r_2, \ldots, r_{\varphi(m)}$ (the residues coprime to m). Since $\gcd(a, m) = 1$, the map $x \mapsto ax$ is a permutation of $(\mathbb{Z}/m\mathbb{Z})^\times$. Hence $\prod_i (a r_i) \equiv \prod_i r_i \pmod{m}$, i.e., $a^{\varphi(m)} \prod_i r_i \equiv \prod_i r_i \pmod{m}$. Since each r_i is invertible mod m, we can cancel to get $a^{\varphi(m)} \equiv 1 \pmod{m}$. □

Corollary 7.5 (Fermat's little theorem)**.** *If* p *is prime and* $p \nmid a$*, then* $a^{p-1} \equiv 1 \pmod{p}$. *Equivalently,* $a^p \equiv a \pmod{p}$ *for all* a.

Proof. $\varphi(p) = p - 1$ for prime p, so Euler's theorem gives $a^{p-1} \equiv 1 \pmod{p}$. □

Euler's theorem immediately gives the modular inverse via $a^{-1} = a^{\varphi(m)-1} \bmod m$ (for $\gcd(a, m) = 1$). For prime $m = p$, this reduces to $a^{-1} = a^{p-2} \bmod p$ (Fermat's inverse, Chapter 5).

7.4 Summation Identity

Theorem 7.6. $\sum_{d|n} \varphi(d) = n$.

Proof. For each $k \in \{1, 2, \ldots, n\}$, let $d = \gcd(k, n)$. Then $d \mid n$, and k/d is coprime to n/d, i.e., $\gcd(k/d, n/d) = 1$. Hence $k/d \in (\mathbb{Z}/(n/d)\mathbb{Z})^\times$. The number of k that map to a given d is $\varphi(n/d)$, and summing over all $d \mid n$ gives $n = \sum_{d|n} \varphi(n/d) = \sum_{d|n} \varphi(d)$ (replacing d by n/d in the last step). □

This identity is the basis for many Möbius-inversion-style arguments in Chapter 10. For example, the count of pairs (a, b) with $1 \le a, b \le n$ and $\gcd(a, b) = g$ is $2\sum_d \varphi(d) - 1$ for $g = 1$, which is a direct application of Theorem 7.6.

7.5 Reducing Exponents Modulo φ

Theorem 7.7. *If a and m are arbitrary positive integers (not necessarily coprime) and $k \ge \log_2 m$, then $a^k \equiv a^{k \bmod \varphi(m) + \varphi(m)} \pmod{m}$.*

Proof sketch. The idea is that for any prime power $p^e \mid m$, if $p \mid a$ then a^k for $k \ge e$ is divisible by p^e, and $a^{k+\varphi(m)}$ is also divisible by p^e (since $\varphi(m)$ is divisible by $p^{e-1}(p-1)$, which is at least e); if $p \nmid a$ then $a^{\varphi(m)} \equiv 1 \pmod{p^e}$ by Euler's theorem. Combining across all $p^e \mid m$ via CRT gives the result. □

This is the standard "tower of powers" reduction: a problem like "compute $a^{b^{c^d}} \bmod m$" is solved by recursively reducing the exponent modulo $\varphi(m)$, then $\varphi(\varphi(m))$, and so on. After $\mathcal{O}(\log m)$ steps the totient reaches 1 and the tower collapses.

```
long long tower(long long a, long long b, long long m);
long long phi(long long m);  // from above

// Compute a^b mod m, using Euler's theorem with safe reduction.
long long pow_tower(long long a, long long b, long long m) {
```

```
    if (m == 1) return 0;
    if (b == 0) return 1 % m;
    long long pm = phi(m);
    long long e = pow_tower(a, b / pm, pm) + pm;   // safe exponent
    return binpow(a, e, m);
}
```

Listing 7.2: Compute $a^b \bmod m$ with tower reduction: b may itself be a tower.

Worked Problem (Codeforces 906D — Power Tower). **Statement.** Given w and a list $a_1, a_2, \ldots, a_k$, compute $a_1^{a_2^{a_3^{\cdots}}} \bmod w$. Constraints: $w \le 10^9$, $k \le 10^5$.

Solution sketch. Apply Theorem 7.7: recursively compute the exponent modulo $\varphi(\varphi(\cdots\varphi(w)))$. After $\mathcal{O}(\log w)$ steps, the totient is 1 and the tower collapses. Care: the exponents are large, so always reduce them with the safe reduction $\varphi(m)+\varphi(m) = \varphi(m)+0\cdot m$ in the formula, exactly as in Theorem 7.7.

7.6 Pattern Recognition

Pattern Recognition. The phrases "compute $a^{b^{c^{\cdots}}} \bmod m$" or "tower of powers modulo m" point directly at Theorem 7.7. The phrase "count pairs with gcd $= 1$ in $[1, n]^2$" (or any variant involving gcd) often wants Theorem 7.6 or the Möbius formula in Chapter 10. The phrase "φ for every i up to n" with $n \le 10^7$ wants the linear sieve; with $n \le 10^{14}$ wants the $\mathcal{O}(\sqrt{n})$ per-query version (or a sieve up to $\sqrt{n}$ plus a Möbius-style decomposition, for many queries).

Chapter 8

Primality Testing

When n is given and we must decide whether n is prime, trial division up to $\sqrt{n}$ is $\mathcal{O}(\sqrt{n})$, which works up to about $n = 10^{14}$ but is too slow for n up to 10^{18}. The Miller–Rabin primality test runs in $\mathcal{O}(k \log^3 n)$ for k rounds, and a deterministic variant with carefully chosen bases runs in $\mathcal{O}(\log^3 n)$, easily handling n up to 10^{18} in well under a millisecond.

8.1 The Fermat Test and Carmichael Numbers

The simplest primality test is the Fermat test: pick a random base a, and check whether $a^{n-1} \equiv 1 \pmod{n}$. If n is prime, Fermat's little theorem guarantees this for every a coprime to n. The problem is the converse: there are composite n for which $a^{n-1} \equiv 1 \pmod{n}$ for every a coprime to n.

Definition 8.1 (Carmichael number)**.** A composite $n \geq 2$ is a *Carmichael number* if $a^{n-1} \equiv 1 \pmod{n}$ for every a coprime to n. The smallest Carmichael numbers are $561 = 3 \cdot 11 \cdot 17$, $1105 = 5 \cdot 13 \cdot 17$, and $1729 = 7 \cdot 13 \cdot 19$.

The Fermat test cannot distinguish Carmichael numbers from primes, so it is not a useful primality test on its own. The Miller–Rabin test below fixes this by adding an extra check based on the structure of the powers of 2 dividing $n - 1$.

8.2 Miller–Rabin: Setup

Write $n - 1 = 2^s \cdot d$ with d odd. (For $n > 2$ odd, $s \geq 1$.) The Miller–Rabin test on base a checks the following chain:

1. Compute $x_0 = a^d \bmod n$.
2. Square x_0 up to s times: $x_{i+1} = x_i^2 \bmod n$ for $i = 0, 1, \ldots, s-1$.
3. The test passes if either $x_0 \equiv 1 \pmod{n}$, or $x_i \equiv -1 \pmod{n}$ for some $0 \le i < s$.

Equivalently, the sequence $(a^d, a^{2d}, a^{4d}, \ldots, a^{2^s d} = a^{n-1})$ modulo n must contain either 1 at the start, or -1 somewhere followed by 1's.

Theorem 8.2. *If n is an odd prime, the Miller–Rabin test passes for every $a \in \{2, 3, \ldots, n-1\}$.*

Proof. Suppose $a \not\equiv 0 \pmod{n}$. By Fermat's little theorem, $a^{n-1} \equiv 1 \pmod{n}$. Write $n - 1 = 2^s d$ with d odd. Then $a^{n-1} = (a^d)^{2^s} \equiv 1 \pmod{n}$, so $x_s \equiv 1 \pmod{n}$. Consider the sequence $(x_0, x_1, \ldots, x_s)$. Each $x_{i+1} = x_i^2$, and $x_s = 1$. Working backwards, $x_{s-1}^2 \equiv 1 \pmod{n}$, so $x_{s-1} \equiv \pm 1 \pmod{n}$ (since $\mathbb{Z}/n\mathbb{Z}$ is a field when n is prime, the equation $x^2 = 1$ has only solutions $x = \pm 1$). If $x_{s-1} \equiv 1$, repeat: $x_{s-2} \equiv \pm 1$. Continuing, either every $x_i \equiv 1$ (in which case $x_0 \equiv 1$ and we are done), or $x_i \equiv -1$ for some i (also done). □

If n is composite and the test fails on base a, we say a is a *witness* to the compositeness of n. The key fact is that for composite n, at least $3/4$ of bases $a \in \{2, \ldots, n-1\}$ are witnesses.

Theorem 8.3 (Monier–Rabin bound)**.** *If $n \ge 2$ is composite, the number of bases $a \in \{2, \ldots, n-1\}$ for which n passes the Miller–Rabin test is at most $(n-1)/4$.*

The proof is nontrivial (using the structure of the group $(\mathbb{Z}/n\mathbb{Z})^\times$) and we omit it. The consequence is that k independent random rounds give error probability at most 4^{-k}: 40 rounds give error $\le 4^{-40} \approx 10^{-24}$, which is far below the probability of a hardware error.

8.3 Deterministic Miller–Rabin

For 32-bit and 64-bit integers, a finite set of bases suffices to deterministically decide primality. The following sets of bases have been verified by exhaustive computation:

Theorem 8.4 (Deterministic MR bases)**.** *For all $n < 2^{32}$, the bases $\{2, 7, 61\}$ suffice. For all $n < 2^{64}$, the bases $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ suffice.*

In other words, running Miller–Rabin with these specific bases is a fully deterministic primality test for 64-bit integers—no randomisation needed.

```cpp
bool miller_rabin(long long n) {
    if (n < 2) return false;
    for (long long p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
        if (n % p == 0) return n == p;
    long long d = n - 1; int s = 0;
    while ((d & 1) == 0) { d >>= 1; ++s; }
    auto check = [&](long long a) {
        long long x = binpow(a, d, n);
        if (x == 1 || x == n - 1) return true;
        for (int i = 0; i < s - 1; ++i) {
            x = (__int128) x * x % n;
            if (x == n - 1) return true;
        }
        return false;
    };
    for (long long a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
        if (a % n == 0) continue;
        if (!check(a)) return false;
    }
    return true;
}
```

Listing 8.1: Deterministic Miller–Rabin for $n < 2^{64}$.

The initial small-prime trial division handles tiny inputs quickly; the loop over seven bases gives full 64-bit determinism. The use of `__int128` for the squaring step is essential, because the square of a 64-bit value can overflow.

8.4 Performance

The asymptotic cost is $\mathcal{O}(k \log^3 n)$ bit operations for k bases, dominated by $k \cdot s \le k \log_2 n$ modular exponentiations. In practice, for n up to 10^{18}, this is a few microseconds. For comparison, trial division up to $\sqrt{10^{18}} = 10^9$ would take roughly $10^9 / \log 10^9 \approx 5 \times 10^7$ trial divisions—roughly 10^9 times slower.

8.5 Worked Problem: SPOJ PON — Prime or Not

Worked Problem (SPOJ PON — Prime or Not). **Statement.** For each of T queries, decide if n is prime. $T \le 10^5$, $n \le 10^{18}$.

Solution sketch. Use the deterministic Miller–Rabin above. Total cost $\mathcal{O}(T \log^3 N) \approx 10^5 \cdot 60^3 \approx 2 \times 10^{10}$ bit operations, which sounds large but in C++ with 64-bit integers runs in well under a second because each bit operation is a single machine instruction. In practice the dominant cost is the 7 base checks per query.

8.6 Pattern Recognition

Pattern Recognition. The phrase "is n prime?" with n up to 10^{18} always wants deterministic Miller–Rabin. With n up to 10^{14} trial division also works. The phrase "find the next prime after n" wants Miller–Rabin plus iteration. The phrase "count primes below n" with n up to 10^{11} wants the Meissel–Mertens or Lucy_Hedgehog sieve (not covered here, but built on the same recursive principle as the segmented sieve).

Chapter 9

Integer Factorisation

When n is given up to 10^{18} and the problem asks for its prime factorisation, trial division is too slow ($\mathcal{O}(\sqrt{n})$) and we need Pollard's rho. The combination of Miller–Rabin (Chapter 8) for primality testing and Pollard's rho for factorisation gives a $\widetilde{\mathcal{O}}(n^{1/4})$ factorisation algorithm that runs in well under a second for $n \le 10^{18}$.

9.1 Pollard's Rho: The Idea

Pollard's rho is a probabilistic factorisation algorithm based on the birthday paradox. The setup: pick a random function $f : \mathbb{Z}/n\mathbb{Z} \to \mathbb{Z}/n\mathbb{Z}$ (typically $f(x) = x^2 + c \bmod n$ for some constant $c \neq 0, -2$), and a starting value x_0. Generate the sequence $x_{i+1} = f(x_i)$. After roughly $\mathcal{O}(\sqrt{p})$ steps, two values x_i, x_j will collide modulo p (where p is some prime factor of n), even though they do not collide modulo n. Then $\gcd(|x_i - x_j|, n)$ is a nontrivial divisor of n.

Theorem 9.1 (Birthday paradox, refined)**.** *Suppose $p \mid n$. If we draw k independent uniform samples from $\mathbb{Z}/p\mathbb{Z}$, the probability that two of them are equal is at least $1 - e^{-k(k-1)/(2p)}$. In particular, $k \ge 2\sqrt{p}$ suffices with constant probability.*

Proof. The probability that k samples are all distinct is $\prod_{i=0}^{k-1}(1 - i/p) \le \prod_{i=0}^{k-1} e^{-i/p} = e^{-k(k-1)/(2p)}$ using $1 - x \le e^{-x}$. Setting $k = 2\sqrt{p}$ gives probability at most $e^{-2} \approx 0.14$ of all distinct, i.e., at least 86% chance of a collision. □

The sequence $x_{i+1} = f(x_i)$ is not independent (it is deterministic), but in practice Pollard's rho behaves as if it were. The reason is the heuristic that f acts as a "random" function on $\mathbb{Z}/p\mathbb{Z}$, so after $\mathcal{O}(\sqrt{p})$ steps the values start to collide modulo p. Floyd's cycle-finding algorithm

finds the collision without storing all previous values: maintain a slow pointer x advancing one step at a time and a fast pointer y advancing two steps at a time, and stop when $x = y$ modulo some divisor of n.

9.2 Floyd's Cycle Finding and the GCD Step

Algorithm 9.2 (Pollard's rho)**.** Given composite n (with a known nontrivial divisor we want to extract):

1. If n is a perfect power, factor it as such.
2. Pick a random $c \in \{1, 2, \ldots, n-1\}$ and random $x_0 \in \{0, 1, \ldots, n-1\}$.
3. Initialize $x = y = x_0$.
4. Repeat:
   (a) $x \leftarrow f(x) = (x^2 + c) \bmod n$ (one step).
   (b) $y \leftarrow f(f(y)) = ((y^2 + c)^2 + c) \bmod n$ (two steps).
   (c) $g \leftarrow \gcd(|x - y|, n)$.
   (d) If $g = n$: failure, restart with new c.
   (e) If $1 < g < n$: success, return g.

```
long long pollard_rho(long long n) {
    if (n % 2 == 0) return 2;
    long long c = 1, x = 2, y = 2, g;
    auto f = [&](long long v) { return ((__int128)v * v + c) % n; };
    do {
        x = f(x);
        y = f(f(y));
        g = __gcd(llabs(x - y), n);
    } while (g == 1);
    return g == n ? -1 : g;      // -1: failure, try another c
}
```

Listing 9.1: Pollard's rho: extract a nontrivial factor of composite n.

9.3 Brent's Improvement

Floyd's algorithm makes two f-evaluations per step (one for x, two for y = two more). Brent's improvement uses only one f-evaluation per step by maintaining a single "turtle" that catches up at exponentially spaced intervals. This is asymptotically the same but about 25% faster in practice.

```
long long pollard_brent(long long n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (true) {
        long long c = rng() % (n - 1) + 1;
        long long y = rng() % n, q = 1, g = 1;
        long long x = y, ys = y;
        for (long long r = 1; g == 1; r *= 2) {
            x = y;
            for (long long i = 0; i < r; ++i) y = ((__int128)y * y + c) % n;
            for (long long k = 0; k < r && g == 1; k += 128) {
                ys = y;
                long long lim = min(r - k, 128LL);
                for (long long i = 0; i < lim; ++i) {
                    y = ((__int128)y * y + c) % n;
                    q = (__int128) q * llabs(x - y) % n;
                }
                g = __gcd(q, n);
            }
        }
        if (g == n) {
            do {
                ys = ((__int128) ys * ys + c) % n;
                g = __gcd(llabs(x - ys), n);
            } while (g == 1);
        }
        if (g != n) return g;
    }
}
```

Listing 9.2: Brent's improved Pollard's rho.

The batched GCD (the inner loop computing $q = \prod |x - y| \bmod n$, then $\gcd(q, n)$ once per 128 iterations) is a critical speedup: each GCD call is $\mathcal{O}(\log n)$ and we save a factor of ~ 128 on the GCD overhead.

9.4 Combining Miller–Rabin with Pollard's Rho

The full factorisation algorithm is recursive: test n for primality (Miller–Rabin), and if it is composite, extract a factor with Pollard's rho, then recurse on both halves.

```
void factor(long long n, vector<long long> &out) {
    if (n == 1) return;
    if (miller_rabin(n)) { out.push_back(n); return; }
    long long d = pollard_brent(n);
    while (d == -1 || d == n) d = pollard_brent(n);
    factor(d, out);
    factor(n / d, out);
}

vector<long long> factorise(long long n) {
    vector<long long> out;
    factor(n, out);
    sort(out.begin(), out.end());
    return out;
}
```

Listing 9.3: Full prime factorisation in $\widetilde{\mathcal{O}}(n^{1/4})$.

Theorem 9.3. *The expected running time of Pollard's rho on composite n is $\mathcal{O}(n^{1/4} \log n)$ per prime factor extracted. The total factorisation time is $\widetilde{\mathcal{O}}(n^{1/4})$.*

Heuristic. By the birthday bound, the rho sequence finds a collision modulo the smallest prime factor p in $\mathcal{O}(\sqrt{p})$ steps. In the worst case $p \approx \sqrt{n}$ (when n is a product of two primes of similar size), this is $\mathcal{O}(n^{1/4})$. Each step is $\mathcal{O}(\log n)$ bit operations for the function evaluation. Summing across all prime factors (which is at most $\log_2 n$), the total is $\widetilde{\mathcal{O}}(n^{1/4})$.

The bound is heuristic because the birthday-paradox analysis assumes the sequence $x_0, x_1, \ldots$ behaves like uniform samples; the actual behaviour of $f(x) = x^2 + c$ is close but not identical. For practical 64-bit n the bound is empirically accurate to within a constant factor. □

9.5 Pollard's $p-1$ Algorithm

A complementary algorithm, Pollard's $p-1$, works when one of the prime factors p of n has the property that $p-1$ is smooth (i.e., has only small prime factors). It computes $a = 2^{B!} \bmod n$ for some bound B, and if $p-1$ has all prime factors $\le B$, then $p \mid a-1$ and $\gcd(a-1, n)$ reveals p. The algorithm is fast when it works but useless when n has no smooth-order factor (which is the typical case for randomly chosen RSA-like n).

```cpp
long long pollard_pm1(long long n, long long B = 100000) {
    long long a = 2;
    for (long long j = 2; j <= B; ++j) {
        a = binpow(a, j, n);
        if (j % 1000 == 0) {
            long long g = __gcd(a - 1, n);
            if (1 < g && g < n) return g;
        }
    }
    long long g = __gcd(a - 1, n);
    return (1 < g && g < n) ? g : -1;
}
```

Listing 9.4: Pollard's $p-1$.

We will not dwell on this; the take-home is that if you suspect $p-1$ is smooth, Pollard $p-1$ can be much faster than Pollard rho.

Worked Problem (Codeforces 1032D — Barcelon Distance, fast factorisation). **Statement.** Given many queries, each asking for the prime factorisation of $n \le 10^{18}$.

Solution sketch. Use the combined Miller–Rabin + Pollard's rho code above. Each query runs in $\widetilde{\mathcal{O}}(n^{1/4}) \approx 10^{4.5} \approx 30000$ operations; 1000 queries is about 3×10^7 operations, well within time.

9.6 Pattern Recognition

Pattern Recognition. "Factorise n" with $n \le 10^{14}$ is fine with trial division. With $n \le 10^{18}$ the combination of Miller–Rabin and Pollard's rho is the universal answer. The presence of phrases like "$n = p \cdot q$ with p, q both prime and $p-1$ smooth" suggests Pollard's $p-1$. With $n \le 10^{30}$, none of the above is sufficient and you need the Quadratic Sieve or similar (outside the scope of

competitive programming). The reflex: $n \leq 10^{18}$ and the word “factor” or “prime factorisation” means pull out the Miller–Rabin + Pollard’s rho template from your library.

Chapter 10

The Möbius Function and Möbius Inversion

The Möbius function μ is the multiplicative-function analogue of "+1 and −1 alternating by parity of prime factors." Its central identity $\sum_{d\mid n}\mu(d) = [n = 1]$ gives rise to Möbius inversion, the technique that lets us convert sums over gcd into sums over divisibility—the single most important trick in intermediate competitive programming number theory.

10.1 Definition and Properties

Definition 10.1 (Möbius function)**.** For $n \in \mathbb{N}$ with $n = p_1^{a_1}\cdots p_k^{a_k}$,

$$\mu(n) = \begin{cases} 1 & n = 1, \\ (-1)^k & a_1 = a_2 = \cdots = a_k = 1 \text{ (n square-free)}, \\ 0 & \text{some } a_i \geq 2. \end{cases}$$

For example, $\mu(1) = 1, \mu(2) = -1, \mu(3) = -1, \mu(4) = 0, \mu(5) = -1, \mu(6) = 1, \mu(7) = -1, \mu(8) = 0, \mu(9) = 0, \mu(10) = 1$.

Theorem 10.2. *μ is multiplicative: if $\gcd(a, b) = 1$ then $\mu(ab) = \mu(a)\mu(b)$.*

Proof. If either a or b is divisible by a square, so is ab, and both sides are 0. Otherwise a and b are square-free and coprime, so ab is square-free and $\omega(ab) = \omega(a) + \omega(b)$ (where ω is the number of distinct prime factors). Hence $\mu(ab) = (-1)^{\omega(a)+\omega(b)} = \mu(a)\mu(b)$. □

10.2 The Fundamental Identity

Theorem 10.3. $\sum_{d|n} \mu(d) = [n = 1] = \begin{cases} 1 & n = 1, \\ 0 & n \geq 2. \end{cases}$

Proof. The case $n = 1$ is trivial. For $n \geq 2$, write $n = p_1^{a_1} \cdots p_k^{a_k}$ and let $S = \{p_1, \ldots, p_k\}$ be the set of prime divisors. The square-free divisors d of n are exactly the products of subsets of S. Hence

$$\sum_{d|n} \mu(d) = \sum_{T \subseteq S} (-1)^{|T|} = (1-1)^{|S|} = 0$$

since $|S| = k \geq 1$. (Any $d \mid n$ with $\mu(d) \neq 0$ must be square-free, hence a product of distinct primes from S.) □

Theorem 10.3 is the entire reason μ is useful. It is an arithmetic version of the inclusion-exclusion principle: summing $\mu(d)$ over divisors of n "cancels out" everything except $n = 1$.

10.3 Möbius Inversion

Theorem 10.4 (Möbius inversion formula)**.** *Let $F, G : \mathbb{N} \to \mathbb{R}$ be functions. Then*

$$F(n) = \sum_{d|n} G(d) \qquad \text{for all } n \in \mathbb{N}$$

if and only if

$$G(n) = \sum_{d|n} \mu(d) F(n/d) \qquad \text{for all } n \in \mathbb{N}.$$

Proof. ($\Rightarrow$) Substitute $F(n/d) = \sum_{e|(n/d)} G(e)$ into the right side:

$$\sum_{d|n} \mu(d) F(n/d) = \sum_{d|n} \mu(d) \sum_{e|(n/d)} G(e) = \sum_{e|n} G(e) \sum_{d|(n/e)} \mu(d) = \sum_{e|n} G(e)[n/e = 1] = G(n)$$

where the inner sum is 0 unless $n/e = 1$, i.e., $e = n$.

($\Leftarrow$) Symmetric by reversing the roles and using $\sum_{d|n} \mu(d) = [n = 1]$. □

The inversion formula converts "F is the sum of G over divisors" to "G is the sum of $\mu \cdot F$ over divisors." This is the central tool for transforming sums involving gcd.

10.4 Application: Counting Coprime Pairs

Worked Problem. Count the number of pairs (a, b) with $1 \le a, b \le n$ and $\gcd(a, b) = 1$.

Solution. Let $f(k) = |\{(a, b) : 1 \le a, b \le n, \gcd(a, b) = k\}|$. We want $f(1)$. The trick: count pairs with $k \mid \gcd(a, b)$. These are pairs where both a and b are multiples of k, i.e., $(a, b) = (kx, ky)$ with $1 \le x, y \le \lfloor n/k \rfloor$. Hence the count is $\lfloor n/k \rfloor^2$. Summing over all k:

$$\sum_{k=1}^{n} f(k) \cdot [\gcd(a, b) = k] = \sum_{k \mid \gcd(a,b)} 1 \text{ (number of pairs with } k \mid \gcd).$$

Equivalently, $\lfloor n/k \rfloor^2 = \sum_{k \mid g} f(g)$ summed over $g \mid \gcd(a, b)$. By Möbius inversion:

$$f(k) = \sum_{d=1}^{\lfloor n/k \rfloor} \mu(d) \lfloor n/(kd) \rfloor^2.$$

Setting $k = 1$:

$$f(1) = \sum_{d=1}^{n} \mu(d) \lfloor n/d \rfloor^2.$$

This is computable in $\mathcal{O}(\sqrt{n})$ using the "integer division has $\mathcal{O}(\sqrt{n})$ distinct values" trick: $\lfloor n/d \rfloor$ is constant over long intervals of d, so we group terms. Precompute prefix sums of μ in $\mathcal{O}(n)$ with the linear sieve (Chapter 3), then for each interval $[l, r]$ of constant $\lfloor n/d \rfloor$, add $(M[r] - M[l-1]) \cdot \lfloor n/d \rfloor^2$.

```
long long coprime_pairs(long long n) {
    // precompute mu[] and prefix sums M[i] = mu[1] + ... + mu[i]
    // via linear sieve; not shown here, see Chapter 3.
    long long ans = 0;
    for (long long l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        long long q = n / l;
        ans += (M[r] - M[l - 1]) * q * q;
    }
    return ans;
}
```

Listing 10.1: Count coprime pairs in $\mathcal{O}(\sqrt{n})$.

The sum telescopes to $\mathcal{O}(\sqrt{n})$ terms, each an $O(1)$ lookup, so the total is $\mathcal{O}(\sqrt{n})$ after the sieve. □

This template—precompute μ, prefix-sum it, then group by the value of $\lfloor n/d \rfloor$—is universal for problems of the form "count pairs with gcd $= 1$" or "count triples with pairwise gcd 1" or "sum of $\gcd(a, b)$ over $1 \le a, b \le n$."

10.5 Application: Sum of GCDs

Worked Problem. Compute $\sum_{a=1}^{n} \sum_{b=1}^{n} \gcd(a, b)$.

Solution. Using $\sum_{d|n} \varphi(d) = n$ (Theorem 7.6), we have $\gcd(a, b) = \sum_{d|\gcd(a,b)} \varphi(d)$. Summing over all pairs:

$$\sum_{a,b} \gcd(a, b) = \sum_{a,b} \sum_{d|\gcd(a,b)} \varphi(d) = \sum_{d=1}^{n} \varphi(d) \cdot \lfloor n/d \rfloor^2.$$

This is again $\mathcal{O}(\sqrt{n})$ with a precomputed prefix sum of φ. □

10.6 Computing μ via the Linear Sieve

```
vector<int> mu;
void sieve_mu(int n) {
    spf.assign(n + 1, 0);
    mu.assign(n + 1, 0);
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) { spf[i] = i; primes.push_back(i); mu[i] = -1; }
        for (int p : primes) {
            if (p > spf[i] || (long long)i * p > n) break;
            spf[i * p] = p;
            if (p == spf[i]) mu[i * p] = 0;              // p^2 | i*p
            else             mu[i * p] = -mu[i];         // new prime factor
        }
    }
}
```

Listing 10.2: Linear sieve computing μ in $\Theta(n)$.

The two cases: if $p \mid i$ then $p^2 \mid ip$ so $\mu(ip) = 0$; if $p \nmid i$ then $\mu(ip) = -\mu(i)$ (multiplicativity, new prime flips sign). Prefix sums $M[i] = \sum_{j=1}^{i} \mu(j)$ are then precomputed in $\mathcal{O}(n)$.

Worked Problem (Codeforces 1559E — Numbers on a Blackboard, Möbius inversion). **Statement.** A typical Möbius inversion problem: count the number of pairs (a, b) with $L \le a, b \le R$

and $\gcd(a, b) = g$.

Solution sketch. Apply the same technique as the coprime-pairs problem but with the bounds shifted: $f(1) = \sum_d \mu(d)\lfloor R/d \rfloor(\lfloor R/d \rfloor - \lfloor (L-1)/d \rfloor)$ after a change of variables.

10.7 Pattern Recognition

Pattern Recognition. The phrases "count pairs with gcd = 1", "sum $\gcd(a, b)$ over $a, b \le n$", "count pairs with gcd = k", and "number of pairs (a, b) with $k \mid \gcd(a, b)$" all want Möbius inversion or the closely related φ-sum identity. The reflex: write the desired count as a sum over d, use $[\gcd = 1] = \sum_{d|\gcd} \mu(d)$ or $\gcd = \sum_{d|\gcd} \varphi(d)$, swap the order of summation, and read off a $\mathcal{O}(\sqrt{n})$ formula with prefix sums of μ or φ.

Chapter 11

Dirichlet Convolution

Dirichlet convolution is the algebra of arithmetic functions under the operation $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$. It turns Möbius inversion from a magic formula into a trivial fact about inverses in a ring, and it unifies the formulas for φ, τ (number of divisors), σ (sum of divisors), and μ into a single framework.

11.1 Definition and Basic Properties

Definition 11.1 (Dirichlet convolution)**.** For arithmetic functions $f, g : \mathbb{N} \to \mathbb{C}$ (or to any commutative ring), the Dirichlet convolution $f * g$ is

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

Theorem 11.2. *Dirichlet convolution is commutative, associative, and bilinear (distributes over pointwise addition). The identity element is ε where $\varepsilon(1) = 1$ and $\varepsilon(n) = 0$ for $n \geq 2$.*

Proof. Commutativity: substitute $d \to n/d$ in the sum. *Associativity*: both sides of $(f * g) * h = f * (g * h)$ equal $\sum_{d_1 d_2 d_3 = n} f(d_1)g(d_2)h(d_3)$. *Identity*: $(f * \varepsilon)(n) = \sum_{d|n} f(d)\varepsilon(n/d) = f(n)\varepsilon(1) = f(n)$ since ε is nonzero only at $n/d = 1$, i.e., $d = n$. □

11.2 The Standard Functions

Definition 11.3.

- $1(n) = 1$ for all n (the constant-1 function).
- $\varepsilon(n) = [n = 1]$ (the identity for Dirichlet convolution).

- $\mathrm{id}(n) = n$.
- $\mathrm{id}_k(n) = n^k$.
- μ is the Möbius function.
- φ is Euler's totient.
- $\tau(n) = \sum_{d|n} 1$ (number of divisors, also written $d(n)$).
- $\sigma(n) = \sum_{d|n} d$ (sum of divisors).
- $\sigma_k(n) = \sum_{d|n} d^k$.

11.3 Möbius Inversion as $\mu * 1 = \varepsilon$

Theorem 11.4. *$\mu * 1 = \varepsilon$. Equivalently, μ is the Dirichlet inverse of 1.*

Proof. $(\mu * 1)(n) = \sum_{d|n} \mu(d) \cdot 1(n/d) = \sum_{d|n} \mu(d) = [n = 1] = \varepsilon(n)$ by Theorem 10.3. □

This single identity *is* the Möbius inversion formula. The inversion statement "$F = G * 1 \iff G = F * \mu$" is just multiplying both sides by the inverse μ (which exists because $\mu * 1 = \varepsilon$). All the inversion-style arguments of Chapter 10 are special cases of this.

11.4 Identities via Convolution

A multiplicative function f with $f(1) = 1$ has a unique Dirichlet inverse, and many useful identities are simply stating the inverse of one function in terms of another. The most useful identities:

Theorem 11.5 (Standard convolution identities)**.**

$$\mu * 1 = \varepsilon, \tag{11.1}$$

$$\varphi * 1 = \mathrm{id}, \qquad \textit{i.e.,}\ \sum_{d|n} \varphi(d) = n, \tag{11.2}$$

$$\varphi * \mu = \mathrm{id} \cdot \mu = ? \quad \textit{actually:}\ \varphi = \mu * \mathrm{id},\ \textit{i.e.,}\ \varphi(n) = \sum_{d|n} \mu(d) \cdot (n/d), \tag{11.3}$$

$$\tau = 1 * 1,\ \textit{i.e.,}\ \tau(n) = \sum_{d|n} 1, \tag{11.4}$$

$$\sigma = 1 * \mathrm{id},\ \textit{i.e.,}\ \sigma(n) = \sum_{d|n} d, \tag{11.5}$$

$$\sigma_k = 1 * \mathrm{id}_k\,. \tag{11.6}$$

Proof. We prove $\varphi = \mu * \mathrm{id}$, which gives $\varphi(n) = \sum_{d|n} \mu(d)(n/d)$. From Theorem 7.6, $\varphi * 1 = \mathrm{id}$. Convolving both sides by μ: $\varphi = \varphi * \varepsilon = \varphi * (1 * \mu) = (\varphi * 1) * \mu = \mathrm{id} * \mu$, which gives the identity.

Equivalently, the formula $\varphi(n) = n \prod_{p|n}(1 - 1/p)$ follows: expand the product, collect the μ-weighted terms. For example, $\varphi(60) = 60 \cdot (1-1/2)(1-1/3)(1-1/5) = 60 \cdot (3/4) \cdot (2/3) \cdot (4/5) = 16$, and also $16 = \sum_{d|60} \mu(d) \cdot 60/d = 60 - 30 - 20 - 12 + 10 + 6 + 4 - 2 = 16$. □

11.5 Multiplicativity is Preserved

Theorem 11.6. *The Dirichlet convolution of two multiplicative functions is multiplicative.*

Proof. Let f, g be multiplicative and $\gcd(a, b) = 1$. The divisors of ab are exactly $\{d_1 d_2 : d_1 \mid a, d_2 \mid b\}$ (using coprimality, every divisor of ab factors uniquely as a divisor of a times a divisor of b). Hence

$$\begin{aligned}(f * g)(ab) &= \sum_{d|ab} f(d) g(ab/d) = \sum_{d_1|a} \sum_{d_2|b} f(d_1 d_2) g((ab)/(d_1 d_2)) \\ &= \sum_{d_1|a} \sum_{d_2|b} f(d_1) f(d_2) g(a/d_1) g(b/d_2) \\ &= (f * g)(a) \cdot (f * g)(b).\end{aligned}$$

□

This means: if you can express a multiplicative function F as a Dirichlet convolution of two multiplicative functions $f * g$, you can compute F pointwise on prime powers and use the linear

sieve (Chapter 3) for F on all integers up to n in $\Theta(n)$.

11.6 Application: Sum of GCDs via Dirichlet

In Chapter 10 we computed $\sum_{a,b\le n}\gcd(a,b)$ using φ. The same computation can be written cleanly with convolutions:

$$\begin{aligned}\sum_{a,b=1}^{n}\gcd(a,b) &= \sum_{a,b}\sum_{d\mid\gcd(a,b)}\varphi(d) \qquad (\text{using } \varphi*1=\mathrm{id} \text{ so } k=\sum_{d\mid k}\varphi(d))\\ &= \sum_{d=1}^{n}\varphi(d)\cdot(\#\text{ of }(a,b)\text{ with } d\mid\gcd(a,b))\\ &= \sum_{d=1}^{n}\varphi(d)\cdot\lfloor n/d\rfloor^2.\end{aligned}$$

The convolution framework made the substitution $\gcd(a,b)=\sum_{d\mid\gcd(a,b)}\varphi(d)$ mechanical.

11.7 Application: Counting $\sum_{\gcd=k}$ via Dirichlet

For a more sophisticated example, the count of triples $(a_1,a_2,a_3)\in[1,n]^3$ with $\gcd(a_1,a_2,a_3)=1$ is

$$\sum_{d=1}^{n}\mu(d)\lfloor n/d\rfloor^3,$$

again $\mathcal{O}(\sqrt{n})$ with prefix sums. The generalisation to k-tuples is $\sum_d\mu(d)\lfloor n/d\rfloor^k$.

11.8 Pattern Recognition

Pattern Recognition. Whenever a problem asks you to "sum $f(\gcd(a,b))$" or "count pairs with $\gcd=k$" or "invert a sum over divisors," think in terms of Dirichlet convolution. The reflex: identify the relevant identity ($\mu*1=\varepsilon$, $\varphi*1=\mathrm{id}$, $\tau=1*1$, $\sigma=1*\mathrm{id}$), rewrite the sum as a convolution, swap the order of summation, and read off a $\mathcal{O}(\sqrt{n})$ formula. The convolution framework is the algebraic form of inclusion-exclusion over divisors.

Chapter 12

Quadratic Residues and Tonelli–Shanks

Given a prime p and an integer a, the equation $x^2 \equiv a \pmod{p}$ may or may not have a solution. When it does, a is called a *quadratic residue* mod p. This chapter develops the theory of quadratic residues (Legendre symbol, Euler's criterion, quadratic reciprocity) and the two standard algorithms for actually solving $x^2 \equiv a \pmod{p}$: Tonelli–Shanks (general, p odd) and Cipolla (often faster in practice).

12.1 Quadratic Residues

Definition 12.1. Let p be an odd prime and $a \in \mathbb{Z}$ with $p \nmid a$. We say a is a *quadratic residue* (QR) mod p if $x^2 \equiv a \pmod{p}$ has a solution. Otherwise a is a *quadratic non-residue* (QNR).

Theorem 12.2. *For odd prime p, exactly $(p-1)/2$ of the elements $1, 2, \ldots, p-1$ are quadratic residues, and $(p-1)/2$ are non-residues.*

Proof. Consider the map $\phi : \mathbb{F}_p^\times \to \mathbb{F}_p^\times$ given by $\phi(x) = x^2$. We claim $\ker \phi = \{1, -1\}$. If $x^2 \equiv 1 \pmod{p}$ then $(x-1)(x+1) \equiv 0 \pmod{p}$, so $x \equiv \pm 1 \pmod{p}$ (since $\mathbb{F}_p$ is a field). Conversely $1^2 = (-1)^2 = 1$. Hence $|\ker \phi| = 2$, and by the first isomorphism theorem $|\mathrm{im}(\phi)| = |\mathbb{F}_p^\times| / |\ker \phi| = (p-1)/2$. The image of ϕ is exactly the set of QRs. $\square$

12.2 The Legendre Symbol and Euler's Criterion

Definition 12.3 (Legendre symbol)**.** For odd prime p and $a \in \mathbb{Z}$ with $p \nmid a$, the Legendre symbol is

$$\left(\frac{a}{p}\right) = \begin{cases} +1 & a \text{ is a QR mod } p, \\ -1 & a \text{ is a QNR mod } p. \end{cases}$$

By convention, $\left(\frac{a}{p}\right) = 0$ if $p \mid a$.

Theorem 12.4 (Euler's criterion)**.** *For odd prime p and $p \nmid a$, $\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$.*

Proof. Let g be a primitive root mod p (existence proved in Chapter 13). Then $a = g^k$ for some k. If k is even, say $k = 2m$, then $a = (g^m)^2$ is a QR. If k is odd, a is a QNR (since any $x = g^j$ has $x^2 = g^{2j}$, and the QRs are exactly the even-indexed powers of g). Now $a^{(p-1)/2} = g^{k(p-1)/2}$. If k is even, $k(p-1)/2$ is a multiple of $p-1$, so $a^{(p-1)/2} \equiv 1$. If k is odd, $k(p-1)/2 = (p-1)/2 + (k-1)(p-1)/2$, and $a^{(p-1)/2} \equiv g^{(p-1)/2} \equiv -1 \pmod{p}$ (since g is a primitive root, $g^{(p-1)/2}$ has order 2, hence is -1). Comparing with the QR/QNR definition, $\left(\frac{a}{p}\right) \equiv a^{(p-1)/2}$. □

Euler's criterion gives a one-line test for quadratic residuosity: compute $a^{(p-1)/2} \bmod p$ via `binpow`, and check whether the result is 1 or -1. This is the basis of the Tonelli–Shanks algorithm below.

Theorem 12.5 (Quadratic reciprocity)**.** *For distinct odd primes p, q,*

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2}\cdot\frac{q-1}{2}}.$$

That is, $\left(\frac{p}{q}\right) = \left(\frac{q}{p}\right)$ unless both $p \equiv q \equiv 3 \pmod{4}$, in which case $\left(\frac{p}{q}\right) = -\left(\frac{q}{p}\right)$.

The proof of quadratic reciprocity is nontrivial (we refer to any standard algebra textbook); the theorem itself is rarely used in competitive programming, but it explains the regularity of QRs modulo different primes.

12.3 Tonelli–Shanks Algorithm

The Tonelli–Shanks algorithm solves $x^2 \equiv n \pmod{p}$ for odd prime p and QR n. The key idea: write $p - 1 = 2^s \cdot q$ with q odd. Find a quadratic non-residue z; then z^q is a "non-square 2^s-th root of unity" that we use as a "correction factor" to iteratively reduce the order of n.

Algorithm 12.6 (Tonelli–Shanks)**.** Given odd prime p and QR n (i.e., $n^{(p-1)/2} \equiv 1 \pmod p$):

1. Write $p - 1 = 2^s \cdot q$ with q odd.
2. Find a quadratic non-residue z by random search: pick z uniform in $[2, p-1]$, compute $z^{(p-1)/2}$, and accept if it is -1.
3. Set $M = s$, $c = z^q$, $t = n^q$, $R = n^{(q+1)/2}$.
4. Loop while $t \not\equiv 1 \pmod p$:
   (a) Find the smallest i with $0 < i < M$ such that $t^{2^i} \equiv 1 \pmod p$.
   (b) Set $b = c^{2^{M-i-1}} \bmod p$.
   (c) Update $M = i$, $c = b^2 \bmod p$, $t = t \cdot b^2 \bmod p$, $R = R \cdot b \bmod p$.
5. Return R.

The invariant is $R^2 \equiv n \cdot t \pmod p$, and t has order 2^M (initially $M = s$). Each iteration reduces M, and when $t \equiv 1$ the invariant gives $R^2 \equiv n$.

```
long long tonelli_shanks(long long n, long long p) {
    if (p == 2) return n & 1;
    n %= p; if (n < 0) n += p;
    if (n == 0) return 0;
    // Euler's criterion: QR check
    if (binpow(n, (p - 1) / 2, p) != 1) return -1;      // not a QR
    long long q = p - 1, s = 0;
    while ((q & 1) == 0) { q >>= 1; ++s; }
    // find non-residue z
    long long z = 2;
    while (binpow(z, (p - 1) / 2, p) != p - 1) ++z;
    long long M = s, c = binpow(z, q, p), t = binpow(n, q, p), R = binpow(n,
        (q + 1) / 2, p);
    while (t != 1) {
        // find smallest i with t^(2^i) = 1
        long long i = 0, tt = t;
        while (tt != 1) { tt = (__int128) tt * tt % p; ++i; }
        long long b = c;
        for (long long j = 0; j < M - i - 1; ++j) b = (__int128) b * b % p;
        M = i; c = (__int128) b * b % p;
        t = (__int128) t * c % p;
        R = (__int128) R * b % p;
```

```
    }
    return R;
}
```

Listing 12.1: Tonelli–Shanks: solve $x^2 \equiv n \pmod{p}$ for QR n.

The two solutions of $x^2 \equiv n$ are R and $p-R$. The algorithm runs in $\mathcal{O}(\log^4 p)$ bit operations in expectation: $\mathcal{O}(\log p)$ to find z (random search), $\mathcal{O}(\log^2 p)$ per iteration, at most $\mathcal{O}(\log p)$ iterations.

12.4 Cipolla's Algorithm

Cipolla's algorithm is an alternative that is often faster in practice. The idea: find a with $a^2 - n$ being a QNR mod p. Then work in the "extension field" $\mathbb{F}_p[\sqrt{a^2-n}]$, where $(a + \sqrt{a^2-n})^{(p+1)/2} = x + y\sqrt{a^2-n}$ with $y = 0$, and $x^2 = n$.

```
long long cipolla(long long n, long long p) {
    n %= p; if (n < 0) n += p;
    if (n == 0) return 0;
    if (binpow(n, (p - 1) / 2, p) != 1) return -1;
    if (p == 2) return n;
    // find a with a^2 - n a QNR
    long long a = 0, w;
    do {
        ++a;
        w = ((a * a - n) % p + p) % p;
    } while (binpow(w, (p - 1) / 2, p) != p - 1);
    // multiply in F_p[sqrt(w)]
    auto mul = [&](pair<long long,long long> x, pair<long long,long long> y)
        {
        return make_pair(
            ((__int128) x.first * y.first + (__int128) x.second * y.second %
                p * w) % p,
            ((__int128) x.first * y.second + (__int128) x.second * y.first) %
                p);
    };
    pair<long long,long long> r = {a, 1}, b = {1, 0};
    long long e = (p + 1) / 2;
    while (e) {
```

```
        if (e & 1) b = mul(b, r);
        r = mul(r, r);
        e >>= 1;
    }
    return (b.first % p + p) % p;
}
```

Listing 12.2: Cipolla: solve $x^2 \equiv n \pmod{p}$ for QR n.

Cipolla's algorithm is conceptually simpler than Tonelli–Shanks (no case analysis on the order of t) and is the standard choice in many contest libraries.

Worked Problem (Project Euler 277 — Modified Collatz). **Statement.** A problem requiring solving $x^2 \equiv n \pmod{p}$ for various (n, p) pairs.

Solution sketch. Use Tonelli–Shanks or Cipolla. Each call is $\mathcal{O}(\log^4 p)$. For p up to 10^{18} and many queries, this is dominated by the `__int128` multiplications.

12.5 Pattern Recognition

Pattern Recognition. The phrases "solve $x^2 \equiv a \pmod{p}$", "find a square root mod p", "is a a QR mod p?" all want Euler's criterion (for the QR test) and Tonelli–Shanks or Cipolla (for the actual square root). If the problem mentions a composite modulus, factorise it first and solve $x^2 \equiv a \pmod{p_i^{e_i}}$ per prime power (via Hensel lifting, Chapter 19), then combine via CRT. The combination "find x with $x^2 \equiv a$" and "a is a QR" is a strong signal to use the algorithms in this chapter.

Chapter 13

Primitive Roots

A primitive root modulo n is an integer g whose powers generate the entire multiplicative group $(\mathbb{Z}/n\mathbb{Z})^\times$. Primitive roots exist only for $n \in \{2, 4, p^k, 2p^k\}$ (where p is an odd prime), and they are the basis for discrete-log and NTT algorithms.

13.1 Orders of Elements

Definition 13.1 (Order)**.** For $a \in \mathbb{Z}$ with $\gcd(a, n) = 1$, the *order* of a modulo n is the smallest positive k with $a^k \equiv 1 \pmod{n}$. We write $\operatorname{ord}_n(a) = k$.

Theorem 13.2. $\operatorname{ord}_n(a) \mid \varphi(n)$.

Proof. By Euler's theorem, $a^{\varphi(n)} \equiv 1 \pmod{n}$. Write $\varphi(n) = q \operatorname{ord}_n(a) + r$ with $0 \le r < \operatorname{ord}_n(a)$. Then $1 \equiv a^{\varphi(n)} = (a^{\operatorname{ord}_n(a)})^q \cdot a^r \equiv a^r \pmod{n}$. Since $\operatorname{ord}_n(a)$ is the smallest positive power giving 1, $r = 0$. □

This theorem is the key to fast computation of $\operatorname{ord}_n(a)$: factor $\varphi(n) = p_1^{e_1} \cdots p_k^{e_k}$, then for each prime p_i divide the candidate order $\varphi(n)$ by p_i as long as $a^{\text{candidate}/p_i} \equiv 1 \pmod{n}$ still holds. This gives $\operatorname{ord}_n(a)$ in $\mathcal{O}(\log^3 n)$ bit operations.

13.2 Existence of Primitive Roots

Theorem 13.3. *Primitive roots modulo n exist iff $n \in \{1, 2, 4, p^k, 2p^k\}$ where p is an odd prime and $k \ge 1$.*

We give the proof only for $n = p$ an odd prime; the prime-power case requires an additional argument and we omit it.

Proof for $n = p$ prime. The multiplicative group $\mathbb{F}_p^\times$ has order $p-1$. We claim there is an element of order d for every $d \mid p-1$. By Lagrange's theorem (group theory), the order of any element divides $p-1$, so it suffices to show existence.

For each $d \mid p-1$, the equation $x^d \equiv 1 \pmod p$ has at most d solutions in $\mathbb{F}_p$ (a polynomial of degree d over a field has at most d roots). Let $\psi(d)$ be the number of elements of $\mathbb{F}_p^\times$ of order exactly d. We have $\sum_{d \mid p-1} \psi(d) = p-1$ (every element has some order dividing $p-1$). We claim $\psi(d) \le \varphi(d)$ for each $d \mid p-1$: if $\psi(d) > 0$, let a be an element of order d; then $1, a, a^2, \ldots, a^{d-1}$ are d distinct solutions of $x^d \equiv 1 \pmod p$, so they are ALL solutions, and the elements of order d are the a^k with $\gcd(k,d) = 1$, of which there are $\varphi(d)$. So $\psi(d) \in \{0, \varphi(d)\}$.

Now $\sum_{d \mid p-1} \psi(d) \le \sum_{d \mid p-1} \varphi(d) = p-1$ (the last equality is Theorem 7.6). Since equality holds, we must have $\psi(d) = \varphi(d)$ for all $d \mid p-1$. In particular $\psi(p-1) = \varphi(p-1) \ge 1$, so a primitive root exists. □

13.3 Finding a Primitive Root

The standard competitive-programming approach is: factor $p-1 = p_1^{e_1} \cdots p_k^{e_k}$, then for each candidate $g \in \{2, 3, \ldots\}$, check whether $g^{(p-1)/p_i} \not\equiv 1 \pmod p$ for all i. If so, g is a primitive root.

```
long long primitive_root(long long p) {
    if (p == 2) return 1;
    long long phi = p - 1;  // for prime p
    vector<long long> factors;
    long long n = phi;
    for (long long d = 2; d * d <= n; ++d) {
        if (n % d == 0) {
            factors.push_back(d);
            while (n % d == 0) n /= d;
        }
    }
    if (n > 1) factors.push_back(n);
    for (long long g = 2; g < p; ++g) {
        bool ok = true;
        for (long long f : factors) {
            if (binpow(g, phi / f, p) == 1) { ok = false; break; }
        }
        if (ok) return g;
```

```
    }
    return -1;
}
```

Listing 13.1: Find a primitive root modulo prime p.

The smallest primitive root modulo p is on average $\mathcal{O}(\log^6 p)$ (a result of Vinogradov), so the search terminates quickly.

13.4 Applications

1. **Discrete logarithm.** With g a primitive root mod p, the map $k \mapsto g^k \bmod p$ is a bijection $\mathbb{Z}/(p-1) \to \mathbb{F}_p^\times$. The discrete log is the inverse map; see Chapter 14.
2. **Number-Theoretic Transform.** For a prime p of the form $p = k2^m + 1$, the element $g^k \bmod p$ (where g is a primitive root) is a 2^m-th root of unity. This is the basis of the NTT (Chapter 15).
3. **Discrete roots.** Solving $x^k \equiv a \pmod p$ for general a reduces to finding a primitive root and solving a linear equation in the exponent via discrete log.

13.5 Pattern Recognition

Pattern Recognition. The phrase "primitive root modulo p" always wants the algorithm above. The phrase "k-th root of a mod p" (i.e., solve $x^k \equiv a$) wants primitive roots + discrete log + CRT in $\mathbb{Z}/(p-1)\mathbb{Z}$. The phrase "find ω such that $\omega^n \equiv 1$ and $\omega^{n/2} \not\equiv 1$" (NTT setup) wants $g^{(p-1)/n} \bmod p$ where g is a primitive root. Recognising primitive roots as the structural tool is essential for the next two chapters.

Chapter 14

Discrete Logarithm and BSGS

The discrete logarithm problem (DLP) asks: given g, h, p (with p prime), find x with $g^x \equiv h \pmod{p}$. The Baby-Step Giant-Step (BSGS) algorithm solves this in $\mathcal{O}(\sqrt{p})$ time and space. Pohlig–Hellman reduces the problem to the prime factors of $p-1$, making it efficient when $p-1$ is smooth.

14.1 The Discrete Logarithm Problem

Definition 14.1 (Discrete logarithm)**.** Let g be a primitive root modulo prime p. For any $h \in \mathbb{F}_p^\times$, the *discrete logarithm* $\log_g h$ is the unique $x \in \{0, 1, \ldots, p-2\}$ with $g^x \equiv h \pmod{p}$.

The discrete logarithm shares many properties with the ordinary logarithm, but is much harder to compute: the best known classical algorithms for the DLP in $\mathbb{F}_p^\times$ are $\mathcal{O}(\sqrt{p})$, while ordinary logarithms are computable in polynomial time. This hardness is the basis of several cryptosystems (Diffie–Hellman, ElGamal).

14.2 Baby-Step Giant-Step

The BSGS algorithm is a time–space tradeoff. The key idea: write $x = im + j$ where $m = \lceil \sqrt{p-1} \rceil$ and $0 \le j < m$. Then

$$g^{im+j} = h \iff g^{im} = hg^{-j} \iff (g^m)^i = h \cdot g^{-j}.$$

Precompute the "baby steps" $\{hg^{-j} \bmod p : 0 \le j < m\}$ in a hash table, then iterate over "giant steps" $i = 0, 1, \ldots, m$ and look up $(g^m)^i \bmod p$ in the table. A match gives $x = im + j$.

Theorem 14.2. *BSGS solves the DLP in $\mathcal{O}(\sqrt{p})$ time and $\mathcal{O}(\sqrt{p})$ space.*

Proof. Every $x \in [0, p-2]$ can be written as $x = im + j$ with $m = \lceil \sqrt{p-1} \rceil$, $0 \le i \le m$, and $0 \le j < m$ (take $i = \lfloor x/m \rfloor$, $j = x \bmod m$). So the search covers all candidates. Precomputation: m exponentiations to fill the table. Main loop: $m+1$ lookups. Total $\mathcal{O}(m)$ work and space. □

```cpp
long long bsgs(long long g, long long h, long long p) {
    // returns smallest x >= 0 with g^x = h (mod p), or -1 if none.
    g %= p; h %= p;
    if (h == 1) return 0;
    long long m = (long long)ceil(sqrt((double)p)) + 1;
    // baby steps: store h * g^{-j} -> j
    unordered_map<long long, long long> baby;
    long long cur = h;
    long long ginv = binpow(g, p - 2, p);
    for (long long j = 0; j < m; ++j) {
        if (baby.find(cur) == baby.end()) baby[cur] = j;
        cur = (__int128) cur * ginv % p;
    }
    // giant steps: look up (g^m)^i
    long long gm = binpow(g, m, p);
    cur = 1;
    for (long long i = 1; i <= m; ++i) {
        cur = (__int128) cur * gm % p;
        auto it = baby.find(cur);
        if (it != baby.end()) {
            long long x = i * m - it->second;
            if (x >= 0) return x;
        }
    }
    return -1;
}
```

Listing 14.1: Baby-Step Giant-Step: solve $g^x \equiv h \pmod{p}$.

A subtlety: BSGS finds the smallest x only if we are careful about which entry to keep in the hash table (the standard practice is to keep the entry with the smallest j, hence the `if (baby.find(cur) == baby.end())` check).

14.3 Pohlig–Hellman

When $p-1$ is smooth (has small prime factors), Pohlig–Hellman reduces the DLP modulo p to DLPs modulo each prime-power factor of $p-1$, then combines via CRT.

Theorem 14.3. *Let g be a primitive root mod p and $h \in \mathbb{F}_p^\times$. Write $p-1 = \prod_i q_i^{e_i}$. For each i, compute $x_i = \log_g h \bmod q_i^{e_i}$ using BSGS in $\mathcal{O}(e_i\sqrt{q_i})$. Combine the x_i via CRT to get $x = \log_g h \bmod p-1$.*

The reduction: set $g_i = g^{(p-1)/q_i^{e_i}}$ and $h_i = h^{(p-1)/q_i^{e_i}}$. Then g_i has order $q_i^{e_i}$, and $x_i = \log_{g_i} h_i$ is the discrete log modulo $q_i^{e_i}$. Each x_i is found by Pohlig–Hellman's recursion on the digits of x_i in base q_i.

For the prime-power DLP modulo q^e, the algorithm finds $x_i = a_0 + a_1 q + a_2 q^2 + \cdots + a_{e-1} q^{e-1}$ one digit at a time. At each step, raise g_i to the appropriate power, compare to h_i, and solve a small DLP modulo q (using BSGS in $\mathcal{O}(\sqrt{q})$).

```
long long dlog_prime_power(long long g, long long h, long long p, long long q
    , long long e) {
    long long pe = 1; for (long long i = 0; i < e; ++i) pe *= q;
    long long x = 0;
    long long gamma = binpow(g, pe / q, p);    // generator of order-q
        subgroup
    for (long long k = 0; k < e; ++k) {
        long long hk = binpow((__int128) h * binpow(g, p - 1 - x, p) % p, pe
            / q / pow(q, k), p);
        long long dk = bsgs(gamma, hk, p);     // small DLP mod q, O(sqrt(q))
        x += dk * (long long) pow(q, k);
    }
    return x;
}
```

Listing 14.2: Pohlig–Hellman for prime-power DLP (sketch).

This is the conceptual core; production code adds careful handling of edge cases ($q=2$, $g=0$, etc.). The total time is $\mathcal{O}(\sum_i e_i\sqrt{q_i})$, which is $\mathcal{O}(\sqrt{p_{\max}}\log p)$ where $p_{\max}$ is the largest prime factor of $p-1$. If $p-1$ is B-smooth, this is $\mathcal{O}(B\log p)$, far better than $\mathcal{O}(\sqrt{p})$.

Worked Problem (SPOJ `DISCRETLOG` — Discrete logarithm). **Statement.** Given g, h, p with p prime and g a primitive root, find x with $g^x \equiv h \pmod p$, or report no solution.

Solution sketch. BSGS, $\mathcal{O}(\sqrt{p})$ time and space. For $p \leq 10^9$ this is about 3×10^4 operations per query; very fast.

14.4 Discrete Roots

A generalisation: solve $x^k \equiv a \pmod{p}$ for given k, a, p. This reduces to a discrete log. With g a primitive root, write $a = g^y$ and $x = g^z$. Then $g^{kz} = g^y$, so $kz \equiv y \pmod{p-1}$. This is a linear congruence in z, solvable iff $\gcd(k, p-1) \mid y$. There are $\gcd(k, p-1)$ solutions if a solution exists.

Worked Problem (Codeforces 101755D — Discrete root). **Statement.** Find all x with $x^k \equiv a \pmod{p}$ for prime p.

Solution sketch. Find primitive root g, compute $y = \log_g a$ via BSGS, solve $kz \equiv y \pmod{p-1}$ via linear congruence (Theorem 5.4), then $x = g^z$ for each solution.

14.5 Pattern Recognition

Pattern Recognition. The phrase "find x with $g^x \equiv h \pmod{p}$" wants BSGS (with $\mathcal{O}(\sqrt{p})$). The phrase "$x^k \equiv a \pmod{p}$, find all x" wants primitive roots + BSGS + linear congruence. The phrase "$g^x \equiv h \pmod{p}$ with $p-1$ smooth" wants Pohlig–Hellman. The combinations are important: any problem with a cyclic-group exponent equation modulo prime p involves primitive roots and discrete log at some level. Recognising this structure is the key to applying the algorithms in this and the previous chapter.

Chapter 15

Number Theoretic Transform (NTT)

The Number Theoretic Transform (NTT) is the modular analogue of the Fast Fourier Transform (FFT). It computes the convolution of two integer sequences modulo a prime p in $\mathcal{O}(n \log n)$ time, using roots of unity in $\mathbb{F}_p$ instead of complex roots of unity. NTT is the standard competitive-programming tool for fast polynomial multiplication modulo a prime, with applications to polynomial inversion, division, and exponentiation.

15.1 Roots of Unity in Finite Fields

Definition 15.1 (Primitive n-th root of unity)**.** A primitive n-th root of unity in $\mathbb{F}_p$ is an element $\omega \in \mathbb{F}_p^\times$ with $\omega^n = 1$ and $\omega^{n/2} \neq 1$ (equivalently, ω has order exactly n).

For $n = 2^k$, such an ω exists in $\mathbb{F}_p$ iff $n \mid p - 1$, i.e., $p \equiv 1 \pmod{n}$. The standard NTT-friendly primes in competitive programming all have the form $p = c \cdot 2^m + 1$ for small c and large m, allowing large powers of two as n:

Prime	Form	Max $n = 2^k$
998244353	$119 \cdot 2^{23} + 1$	2^{23}
167772161	$5 \cdot 2^{25} + 1$	2^{25}
469762049	$7 \cdot 2^{26} + 1$	2^{26}

If g is a primitive root mod p (Chapter 13), then $\omega = g^{(p-1)/n} \bmod p$ is a primitive n-th root of unity in $\mathbb{F}_p$. For $p = 998244353$, the primitive root is $g = 3$, and $\omega = 3^{(p-1)/n} \bmod p$ is the standard choice for any $n \mid p - 1$.

15.2 The Transform

Let n be a power of 2 and ω be a primitive n-th root of unity in $\mathbb{F}_p$. The NTT of a sequence $a_0, a_1, \ldots, a_{n-1}$ is

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \pmod{p}, \qquad k = 0, 1, \ldots, n-1.$$

The inverse NTT is the same transform but with ω^{-1} in place of ω, then dividing by n:

$$a_j = n^{-1} \sum_{k=0}^{n-1} A_k \omega^{-jk} \pmod{p}.$$

Theorem 15.2 (Convolution theorem)**.** *The NTT of the convolution $(a * b)_k = \sum_{j=0}^{k} a_j b_{k-j}$ is the pointwise product $A_k B_k$ of the NTTs of a and b. Hence the convolution can be computed as $\mathrm{NTT}^{-1}(\mathrm{NTT}(a) \cdot \mathrm{NTT}(b))$ in $\mathcal{O}(n \log n)$.*

Proof. Substitute the definitions:

$$(\mathrm{NTT}(a * b))_k = \sum_j (a * b)_j \omega^{jk} = \sum_j \sum_{i=0}^{j} a_i b_{j-i} \omega^{jk} = \sum_i a_i \omega^{ik} \sum_l b_l \omega^{lk} = A_k B_k.$$

The swap of summation order is valid because we are finite sums. □

15.3 Iterative Cooley–Tukey

The Cooley–Tukey algorithm factors the NTT into smaller NTTs. Given an NTT of length $n = 2m$, split the input into even and odd indices:

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} = \sum_{j=0}^{m-1} a_{2j} \omega^{2jk} + \omega^k \sum_{j=0}^{m-1} a_{2j+1} \omega^{2jk}.$$

The two sums are NTTs of length m of the even and odd subsequences (using ω^2 as the root, which is a primitive m-th root of unity). Recursively this gives an $\mathcal{O}(n \log n)$ algorithm. The iterative version uses bit-reversal permutation to do the work in place.

```
const long long NTT_MOD = 998244353;
const long long NTT_G   = 3;

void ntt(vector<long long> &a, bool invert) {
    int n = a.size();
    // bit-reversal permutation
```

```
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long w = binpow(NTT_G, (NTT_MOD - 1) / len, NTT_MOD);
        if (invert) w = binpow(w, NTT_MOD - 2, NTT_MOD);
        for (int i = 0; i < n; i += len) {
            long long cw = 1;
            for (int j = 0; j < len / 2; ++j) {
                long long u = a[i + j];
                long long v = (__int128) a[i + j + len / 2] * cw % NTT_MOD;
                a[i + j] = (u + v) % NTT_MOD;
                a[i + j + len / 2] = (u - v + NTT_MOD) % NTT_MOD;
                cw = (__int128) cw * w % NTT_MOD;
            }
        }
    }
    if (invert) {
        long long n_inv = binpow(n, NTT_MOD - 2, NTT_MOD);
        for (long long &x : a) x = (__int128) x * n_inv % NTT_MOD;
    }
}

vector<long long> multiply(vector<long long> a, vector<long long> b) {
    int n = 1;
    while (n < (int)(a.size() + b.size() - 1)) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (__int128) a[i] * b[i] % NTT_MOD;
    ntt(a, true);
    a.resize(a.size() + b.size() - 1);  // trim
    return a;
}
```

Listing 15.1: Iterative NTT, in place, $\mathcal{O}(n \log n)$.

15.4 Arbitrary Modulus via Triple-Modulus NTT

The above NTT only works modulo an NTT-friendly prime. To multiply polynomials modulo an arbitrary modulus M (say 10^9+7), use the *triple-modulus NTT* (MTT) technique: choose three NTT-friendly primes p_1, p_2, p_3 whose product $p_1p_2p_3$ exceeds the maximum possible coefficient (which is $\sum |a_i b_j| \le n \cdot M^2$), run NTT modulo each prime, then reconstruct each coefficient via CRT (Theorem 6.1) and reduce modulo M.

The standard choice is $p_1 = 998244353$, $p_2 = 167772161$, $p_3 = 469762049$. Their product is about 7.9×10^{25}, supporting coefficients up to this size, which is enough for almost all competitive-programming polynomial-multiplication problems.

```
vector<long long> multiply_mod(vector<long long> a, vector<long long> b, long
     long M) {
    const long long P1 = 998244353, P2 = 167772161, P3 = 469762049;
    auto r1 = multiply_under(a, b, P1);  // NTT mod P1
    auto r2 = multiply_under(a, b, P2);
    auto r3 = multiply_under(a, b, P3);
    int n = r1.size();
    vector<long long> out(n);
    for (int i = 0; i < n; ++i) {
        long long v = garner({r1[i], r2[i], r3[i]}, {P1, P2, P3});
        out[i] = v % M;
    }
    return out;
}
```

Listing 15.2: Triple-modulus NTT (MTT) for arbitrary modulus M.

15.5 Polynomial Operations via NTT

The bare NTT gives convolution. From there, the entire apparatus of polynomial algebra follows:

1. **Polynomial inverse.** Given $f(x)$ of degree n, find $g(x)$ with $f(x)g(x) \equiv 1 \pmod{x^{n+1}}$. Newton iteration: $g_{k+1} = g_k(2 - fg_k)$, doubling the degree each step. Total $\mathcal{O}(n \log n)$.

2. **Polynomial division.** Compute $f(x) = q(x)g(x) + r(x)$ with $\deg r < \deg g$. Reverse f and g, divide (using inverse), reverse back.

3. **Polynomial logarithm.** $\log f(x) = \int f'(x)/f(x)\,dx$ for f with $f(0) = 1$. NTT for the derivative and division.

4. **Polynomial exponentiation.** $\exp f(x)$ via Newton iteration: $g_{k+1} = g_k(1 - \log g_k + f)$. Total $\mathcal{O}(n \log n)$.

Each of these is a competitive-programming problem in its own right, and each is built on the NTT primitive above.

Worked Problem (Codeforces 438E — Little Child and Polynomial). **Statement.** Count the number of ways to construct a polynomial of given degree with values from a multiset, modulo a prime.

Solution sketch. Use polynomial exponentiation via NTT and Newton iteration.

15.6 Pattern Recognition

Pattern Recognition. The phrase "multiply two polynomials modulo p" with $p = 998244353$ always wants NTT directly. With $p = 10^9+7$ or another non-NTT-friendly prime, it wants MTT (triple-modulus NTT plus Garner). The phrases "polynomial inverse / division / square root / logarithm / exponential modulo x^n" all want NTT plus Newton iteration. Recognising that a problem involves polynomials (sequences with "product" meaning convolution, or generating functions) is the first step; identifying which polynomial operation is needed is the second.

Chapter 16

Linear Recurrences and Berlekamp–Massey

Many competitive programming problems involve a sequence defined by a linear recurrence (Fibonacci, Lucas, tribonacci, or problem-specific recurrences). The bottleneck is always the same: the recurrence is short but the index asked for is up to 10^{18}, so direct iteration is impossible. The standard tools are matrix exponentiation ($\mathcal{O}(k^3 \log n)$ for a k-term recurrence), Berlekamp–Massey (find the recurrence from a prefix of the sequence), and Kitamasa ($\mathcal{O}(k^2 \log n)$ or $\mathcal{O}(k \log k \log n)$ for the n-th term).

16.1 Matrix Exponentiation

A linear recurrence $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$ is equivalent to the matrix–vector recurrence

$$\begin{pmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_{n-k+1} \end{pmatrix} = \begin{pmatrix} c_1 & c_2 & \cdots & c_k \\ 1 & 0 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & \cdots & 1 & 0 \end{pmatrix} \begin{pmatrix} a_{n-1} \\ a_{n-2} \\ \vdots \\ a_{n-k} \end{pmatrix}.$$

The companion matrix C is $k \times k$, and $a_n = (C^n \mathbf{v})_1$ for an appropriate initial vector $\mathbf{v}$. Computing C^n takes $\mathcal{O}(k^3 \log n)$ via binary exponentiation of matrices.

```
typedef vector<vector<long long>> Mat;

Mat matmul(const Mat &A, const Mat &B, long long mod) {
    int n = A.size(), m = B[0].size(), k = B.size();
```

```
    Mat C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            for (int l = 0; l < k; ++l)
                C[i][j] = (C[i][j] + (__int128) A[i][l] * B[l][j]) % mod;
    return C;
}

Mat matpow(Mat A, long long e, long long mod) {
    int n = A.size();
    Mat R(n, vector<long long>(n, 0));
    for (int i = 0; i < n; ++i) R[i][i] = 1;   // identity
    while (e) {
        if (e & 1) R = matmul(R, A, mod);
        A = matmul(A, A, mod);
        e >>= 1;
    }
    return R;
}
```

Listing 16.1: Matrix exponentiation for the n-th term of a linear recurrence, $\mathcal{O}(k^3 \log n)$.

For Fibonacci ($k = 2$), this is $\mathcal{O}(8 \log n) = \mathcal{O}(\log n)$, computing $F_{10^{18}} \bmod m$ in microseconds. For larger k the matrix exponentiation becomes $\mathcal{O}(k^3 \log n)$, which is practical up to $k \approx 200$ with $n \le 10^{18}$.

16.2 Berlekamp–Massey

Given a sequence $a_0, a_1, \ldots, a_{2k-1}$ (the first $2k$ terms of a k-th order linear recurrence), Berlekamp–Massey recovers the recurrence in $\mathcal{O}(k^2)$ time. The output is the coefficients $c_1, c_2, \ldots, c_k$ with $a_n = \sum_{i=1}^{k} c_i a_{n-i}$.

Algorithm 16.1 (Berlekamp–Massey)**.** Maintain the current recurrence $C(x) = 1 - c_1 x - \cdots - c_k x^k$, the current length L, the previous recurrence $B(x)$ at the last update, and the previous length L_b. For each new term a_n:

1. Compute the discrepancy $\delta = a_n - \sum_{i=1}^{L} c_i a_{n-i}$.

2. If $\delta = 0$, the current recurrence still works; continue.

3. Otherwise, set $T(x) = C(x)$. Adjust $C(x) \leftarrow C(x) - \delta \cdot \delta_b^{-1} \cdot x^{n-n_b} \cdot B(x)$, where $\delta_b, B(x), n_b$ are the stored values from the last update.

4. If $2L \le n$, update $L \leftarrow n + 1 - L$, $B(x) \leftarrow T(x)$, $\delta_b \leftarrow \delta$, $n_b \leftarrow n$.

```cpp
vector<long long> berlekamp_massey(vector<long long> a, long long p) {
    int n = a.size();
    vector<long long> C(n, 0), B(n, 0);
    C[0] = B[0] = 1;
    int L = 0, m = 1; long long b = 1;
    for (int i = 0; i < n; ++i) {
        long long d = a[i] % p;
        for (int j = 1; j <= L; ++j) d = (d + C[j] * a[i - j]) % p;
        if (d == 0) { ++m; continue; }
        auto T = C;
        long long coef = d * binpow(b, p - 2, p) % p;
        for (int j = m; j < n; ++j)
            C[j] = (C[j] - coef * B[j - m] % p + p) % p;
        if (2 * L <= i) {
            L = i + 1 - L; B = T; b = d; m = 1;
        } else ++m;
    }
    C.resize(L + 1);
    return C;  // C[0]=1, a_n = sum_{i=1}^L C[i]*a_{n-i}
}
```

Listing 16.2: Berlekamp–Massey: find the minimal recurrence of a sequence mod p.

A practical use: in some competitive-programming problems the problem setter defines a sequence via $a_n = f(a_{n-1}, \dots, a_{n-k})$ for some complicated f, and the participant is expected to compute $a_{10^{18}}$ mod p. The trick is to compute the first $2k$ terms of the sequence by brute force (assuming k is small), run Berlekamp–Massey to find the recurrence, and then use Kitamasa or matrix exponentiation to get $a_{10^{18}}$.

16.3 Kitamasa's Algorithm

Kitamasa computes a_n given a k-th order linear recurrence and initial values, in $\mathcal{O}(k^2 \log n)$ or $\mathcal{O}(k \log k \log n)$ (the latter via NTT). The key idea: write $a_n = \sum_{i=0}^{k-1} t_i a_i$ for some coefficients t_i that depend only on n (not the initial values). The polynomial x^n mod $R(x)$, where $R(x) =$

$x^k - c_1 x^{k-1} - \cdots - c_k$ is the recurrence characteristic polynomial, gives the t_i directly: $x^n \equiv \sum t_i x^i \pmod{R(x)}$.

```cpp
vector<long long> kitamasa(vector<long long> c, vector<long long> a, long
    long n, long long p) {
    int k = c.size();
    // R(x) = x^k - c_1 x^{k-1} - ... - c_k. Store as c[1..k].
    // Multiply polynomials mod R(x) using cyclic convolution.
    auto poly_mul = [&](vector<long long> &x, vector<long long> &y) {
        vector<long long> r(2 * k - 1, 0);
        for (int i = 0; i < k; ++i) if (x[i])
            for (int j = 0; j < k; ++j) if (y[j])
                r[i + j] = (r[i + j] + (__int128) x[i] * y[j]) % p;
        for (int i = 2 * k - 2; i >= k; --i) {
            for (int j = 1; j <= k; ++j)
                r[i - j] = (r[i - j] + r[i] * c[j - 1]) % p;
            r[i] = 0;
        }
        r.resize(k);
        return r;
    };
    vector<long long> result(k, 0), base(k, 0);
    result[0] = 1; if (k > 1) base[1] = 1; else base[0] = c[0];
    while (n) {
        if (n & 1) result = poly_mul(result, base);
        base = poly_mul(base, base);
        n >>= 1;
    }
    long long ans = 0;
    for (int i = 0; i < k; ++i) ans = (ans + result[i] * a[i]) % p;
    return ans;
}
```

Listing 16.3: Kitamasa: n-th term of a k-th order recurrence, $\mathcal{O}(k^2 \log n)$.

For $k \le 1000$ and $n \le 10^{18}$, this is fast enough; for k up to 10^5, the polynomial multiplications can be done via NTT, giving $\mathcal{O}(k \log k \log n)$.

Worked Problem (Codeforces 506E — Mr. Kitayuta, Berlekamp–Massey problem). **Statement.** Given the first few terms of a sequence defined by a complicated transition (e.g., a graph or

DP), compute the 10^{18}-th term modulo p.

Solution sketch. Compute the first $2k$ terms by direct DP (assuming k small), run Berlekamp–Massey to find the recurrence, then run Kitamasa for $a_{10^{18}}$. The trick is recognising that the underlying sequence obeys a linear recurrence, even when the problem setter does not state it.

16.4 Pattern Recognition

Pattern Recognition. The phrase "compute a_n mod p where a_n is defined by a linear recurrence" with $n \leq 10^{18}$ wants matrix exponentiation (small k) or Kitamasa (larger k). The phrase "the sequence is defined by [some complicated DP], compute the 10^{18}-th term" wants Berlekamp–Massey: compute the first $2k$ terms by simulation, then BM recovers the recurrence. The combination of n up to 10^{18} and a recurrence structure is the universal signal.

Chapter 17

Fibonacci Numbers and Pisano Period

Fibonacci numbers are the canonical linear recurrence: $F_0 = 0, F_1 = 1, F_{n+1} = F_n + F_{n-1}$. They appear in competitive programming because of their rich identities, their periodic behaviour modulo any m (the Pisano period), and the existence of a $\mathcal{O}(\log n)$ algorithm (fast doubling) to compute individual F_n.

17.1 Identities

Theorem 17.1 (Standard Fibonacci identities)**.** *For all $n \geq 0$:*

1. $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, \ldots$
2. ***Cassini:*** $F_{n+1}F_{n-1} - F_n^2 = (-1)^n$.
3. ***Sum:*** $F_0 + F_1 + \cdots + F_n = F_{n+2} - 1$.
4. ***Sum of squares:*** $F_0^2 + F_1^2 + \cdots + F_n^2 = F_n F_{n+1}$.
5. ***Sum of odd indices:*** $F_1 + F_3 + \cdots + F_{2n-1} = F_{2n}$.
6. ***Sum of even indices:*** $F_0 + F_2 + \cdots + F_{2n} = F_{2n+1} - 1$.
7. ***Zeckendorf:*** *every $n \in \mathbb{N}$ has a unique representation as a sum of non-consecutive Fibonacci numbers.*
8. ***GCD:*** $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.
9. ***Binet:*** $F_n = (\varphi^n - \psi^n)/\sqrt{5}$ *where* $\varphi = (1 + \sqrt{5})/2$ *and* $\psi = (1 - \sqrt{5})/2$.

Proof of (h). We prove $\gcd(F_m, F_n) = F_{\gcd(m,n)}$. First, the Euclidean reduction: $F_{m+n} = F_m F_{n+1} + F_{m-1} F_n$ (a small identity verifiable by induction). Hence $\gcd(F_m, F_n)$ divides $\gcd(F_m, F_{m+n})$, and iterating $\gcd(F_m, F_n) = \gcd(F_m, F_{n \bmod m})$ since $F_{n \bmod m} = F_{n-km}$ for appropriate k and Fibonacci numbers satisfy $F_{n-km} \equiv F_n \cdot (\text{something}) \pmod{F_m}$. This reduces $\gcd(F_m, F_n)$ to $\gcd(F_{\gcd(m,n)}, F_0) = F_{\gcd(m,n)}$. □

17.2 Fast Doubling

The naive recursion $F_{n+1} = F_n + F_{n-1}$ is $\mathcal{O}(\log n)$ with memoisation, but with constant factors from the recursion. The *fast doubling* formula computes F_n in $\mathcal{O}(\log n)$ iteratively:

Theorem 17.2 (Fast doubling)**.**

$$F_{2n} = F_n(2F_{n+1} - F_n),$$
$$F_{2n+1} = F_n^2 + F_{n+1}^2.$$

Proof. From the matrix representation: $\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$. Squaring gives the doubling formulas directly. □

```
pair<long long, long long> fib(long long n, long long p) {
    if (n == 0) return {0, 1};
    auto [a, b] = fib(n >> 1, p);
    long long c = (__int128) a * (2 * b % p - a + p) % p;
    long long d = (__int128) a * a % p + (__int128) b * b % p;
    d %= p;
    if (n & 1) return {d, (c + d) % p};
    else       return {c, d};
}
long long fib_n(long long n, long long p) { return fib(n, p).first; }
```

Listing 17.1: Fast doubling: $F_n \bmod p$ in $\mathcal{O}(\log n)$.

17.3 Pisano Period

Definition 17.3 (Pisano period)**.** The *Pisano period* $\pi(m)$ is the period of the Fibonacci sequence modulo m: the smallest positive k with $F_k \equiv 0 \pmod{m}$ and $F_{k+1} \equiv 1 \pmod{m}$.

Theorem 17.4. *For every $m \geq 2$, the Pisano period $\pi(m)$ exists and $\pi(m) \leq 6m$. Specifically:*

- *If p is prime and $p \equiv \pm 1 \pmod 5$, then $\pi(p) \mid p - 1$.*
- *If p is prime and $p \equiv \pm 2 \pmod 5$, then $\pi(p) \mid 2(p + 1)$.*
- $\pi(2) = 3, \pi(5) = 20$.
- *For $m = p_1^{e_1} \cdots p_k^{e_k}$, $\pi(m) = \operatorname{lcm}(\pi(p_1^{e_1}), \ldots, \pi(p_k^{e_k}))$.*
- $\pi(p^k) = p^{k-1}\pi(p)$ *for $p \neq 2, 5$.*

The Pisano period gives a slick way to compute $F_n \bmod m$: reduce n modulo $\pi(m)$ first. For m with small $\pi(m)$, this can be faster than fast doubling when many queries are involved. The Pisano period is also used in problems that ask "find the smallest k with $F_k \equiv 0 \pmod m$," which is the *entry point* of the Fibonacci sequence modulo m.

17.4 Applications

1. **Modular Fibonacci.** "Compute $F_n \bmod m$ for $n \leq 10^{18}$" always wants fast doubling.
2. **Fibonacci representations.** "Represent n as a sum of non-consecutive Fibonacci numbers" wants Zeckendorf's theorem.
3. **Period finding.** "Find the period of $F_n \bmod m$" wants Pisano period (factorise m, compute π per prime power, take LCM).
4. **GCD with Fibonacci.** "Compute $\gcd(F_m, F_n)$" wants Theorem 17.1(h): $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.

Worked Problem (Project Euler 2 — Even Fibonacci Numbers). **Statement.** Sum of even Fibonacci numbers up to 4×10^6.

Solution sketch. Note F_n is even iff $n \equiv 0 \pmod 3$. So the even Fibonacci numbers are $F_3, F_6, F_9, \ldots$. Sum them by identity (c) (sum of F's up to F_n is $F_{n+2} - 1$) restricted to multiples of 3, or just iterate.

17.5 Pattern Recognition

Pattern Recognition. The phrase "n-th Fibonacci number mod m" with $n \leq 10^{18}$ always wants fast doubling. The phrase "period of Fibonacci mod m" wants Pisano period via Theorem 17.4.

The phrase "find k with $F_k \equiv 0 \pmod{m}$" (entry point) wants the smallest k with $m \mid F_k$, which is a divisor of $\pi(m)$. The combination "Fibonacci" and "modulo" is a strong signal for fast doubling or Pisano period.

Chapter 18

Combinatorial Number Theory

Combinatorial number theory deals with binomial coefficients, Stirling numbers, Catalan numbers, and related combinatorial quantities computed modulo primes (or prime powers). The central tools are Lucas's theorem (binomials modulo a prime), Kummer's theorem (the p-adic valuation of a binomial), and precomputation of factorials for $\mathcal{O}(1)$ binomial queries.

18.1 Binomial Coefficients Modulo a Prime

Theorem 18.1 (Lucas's theorem)**.** *Let p be prime, and write n, k in base p:*

$$n = n_0 + n_1 p + \cdots + n_d p^d, \qquad k = k_0 + k_1 p + \cdots + k_d p^d.$$

Then

$$\binom{n}{k} \equiv \prod_{i=0}^{d} \binom{n_i}{k_i} \pmod{p}.$$

By convention, $\binom{n_i}{k_i} = 0$ if $k_i > n_i$, so $\binom{n}{k} \equiv 0 \pmod{p}$ if any digit of k exceeds the corresponding digit of n.

Proof. The polynomial $(1+x)^p \equiv 1 + x^p \pmod{p}$ (all intermediate binomial coefficients $\binom{p}{i}$ for $0 < i < p$ are divisible by p). Hence

$$(1+x)^n = (1+x)^{n_0}(1+x^p)^{n_1} \cdots (1+x^{p^d})^{n_d} \equiv \prod_i (1+x^{p^i})^{n_i} \pmod{p}.$$

The coefficient of $x^k = x^{k_0 + k_1 p + \cdots + k_d p^d}$ on the right side is $\prod_i \binom{n_i}{k_i}$. □

```
long long C_mod(long long n, long long k, long long p,
```

```
                vector<long long> &fact, vector<long long> &ifact) {
    if (k < 0 || k > n) return 0;
    long long r = 1;
    while (n > 0 || k > 0) {
        long long ni = n % p, ki = k % p;
        if (ki > ni) return 0;
        r = (__int128) r * fact[ni] % p * ifact[ki] % p * ifact[ni - ki] % p;
        n /= p; k /= p;
    }
    return r;
}
```

Listing 18.1: Binomial $\binom{n}{k} \bmod p$ via Lucas's theorem.

For $n, k \le 10^{18}$ and $p \le 10^6$, Lucas's theorem reduces the problem to $\mathcal{O}(\log_p n)$ lookups in a precomputed factorial table.

18.2 Kummer's Theorem: p-adic Valuation of Binomials

Theorem 18.2 (Kummer). *The p-adic valuation $v_p\left(\binom{n}{k}\right)$ (the largest e with $p^e \mid \binom{n}{k}$) equals the number of carries when adding k and $n-k$ in base p.*

Equivalently, $v_p(\binom{n}{k})$ is the number of borrows when subtracting k from n in base p. Kummer's theorem is the basis for the formula $\binom{n}{k} \bmod p^e$ (generalised Lucas, see Andrew Granville's survey).

18.3 Precomputation for Fast Binomials

For $n, k \le 10^6$ and prime modulus p, precompute $f[i] = i! \bmod p$ and $f^{-1}[i] = (i!)^{-1} \bmod p$ in $\mathcal{O}(n)$. Then $\binom{n}{k} \equiv f[n] \cdot f^{-1}[k] \cdot f^{-1}[n-k] \pmod p$ in $\mathcal{O}(1)$ per query.

```
vector<long long> fact, ifact;

void build_factorials(int n, long long p) {
    fact.resize(n + 1);
    ifact.resize(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; ++i) fact[i] = (__int128) fact[i - 1] * i % p;
    ifact[n] = binpow(fact[n], p - 2, p);
```

```
    for (int i = n - 1; i >= 0; --i) ifact[i] = (__int128) ifact[i + 1] * (i
        + 1) % p;
}

long long C(int n, int k, long long p) {
    if (k < 0 || k > n) return 0;
    return (__int128) fact[n] * ifact[k] % p * ifact[n - k] % p;
}
```

Listing 18.2: Precompute factorials and inverse factorials in $\mathcal{O}(n)$.

The trick of computing $ifact[n] = (fact[n])^{-1}$ by Fermat and then $ifact[i-1] = ifact[i] \cdot i$ in reverse order is $\mathcal{O}(n + \log p)$, much faster than computing each inverse independently.

18.4 Stirling Numbers

Definition 18.3 (Stirling numbers)**.**

- **First kind (unsigned):** $\left[{n \atop k}\right]$ is the number of permutations of n elements with exactly k cycles. Recurrence: $\left[{n \atop k}\right] = \left[{n-1 \atop k-1}\right] + (n-1)\left[{n-1 \atop k}\right]$.
- **Second kind:** $\left\{{n \atop k}\right\}$ is the number of partitions of an n-element set into k non-empty subsets. Recurrence: $\left\{{n \atop k}\right\} = \left\{{n-1 \atop k-1}\right\} + k\left\{{n-1 \atop k}\right\}$. Closed form: $\left\{{n \atop k}\right\} = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$.

```
vector<vector<long long>> stirling2(int n, int k, long long p) {
    vector<vector<long long>> S(n + 1, vector<long long>(k + 1, 0));
    S[0][0] = 1;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= k; ++j)
            S[i][j] = (S[i-1][j-1] + (__int128) j * S[i-1][j]) % p;
    return S;
}
```

Listing 18.3: Stirling numbers of the second kind modulo p, $\mathcal{O}(nk)$.

The closed form of $\left\{{n \atop k}\right\}$ as a sum of k-fold binomials modulo p is computable in $\mathcal{O}(k \log n)$ using fast exponentiation, useful when k is small and n is large (e.g., $n \le 10^{18}$, $k \le 10^5$).

18.5 Catalan Numbers

The n-th Catalan number is $C_n = \frac{1}{n+1}\binom{2n}{n}$, the number of balanced parentheses sequences, binary trees, Dyck paths, and many other combinatorial objects. Catalan numbers appear in

competitive programming problems involving balanced structures.

```
long long catalan(long long n, long long p, vector<long long> &fact, vector<
    long long> &ifact) {
    if (n == 0) return 1;
    // C_n = binom(2n, n) / (n+1). If p > n+1, modular inverse of n+1 exists.
    long long c = (__int128) fact[2 * n] * ifact[n] % p * ifact[n] % p;
    long long inv = binpow(n + 1, p - 2, p);
    return (__int128) c * inv % p;
}
```

Listing 18.4: Catalan number modulo p.

18.6 Applications

1. **Large binomials modulo prime.** $\binom{n}{k} \bmod p$ with $n \le 10^{18}$ and $p \le 10^6$ wants Lucas's theorem. With $n, k \le 10^6$ and p any prime, the precomputed factorial approach is faster.
2. **Binomials modulo prime power.** $\binom{n}{k} \bmod p^e$ wants the Granville–Andrew generalisation of Lucas, using Kummer's theorem.
3. **Stirling numbers modulo prime.** For small k and large n, use the closed form (sum of binomials times j^n).
4. **Catalan numbers.** Whenever a problem involves "balanced" structures, reach for Catalan.

Worked Problem (Codeforces 300C — Beautiful Numbers). **Statement.** Count the number of n-digit positive integers whose digits are all from $\{a, b\}$ and whose digit sum is "beautiful" (i.e., the sum itself has only digits a, b).

Solution sketch. Iterate over the number of a's, compute the binomial $\binom{n}{k}$, and check the beauty of $ka + (n-k)b$. Use the precomputed factorial table.

18.7 Pattern Recognition

Pattern Recognition. The phrase "$\binom{n}{k} \bmod p$" with p prime and small ($\le 10^6$) wants Lucas or precomputed factorials. With $n \le 10^{18}$ and p small specifically, Lucas. With $n \le 10^6$, precomputed factorials are simpler. The phrase "count of partitions" or "number of ways to

partition" often wants Stirling numbers of the second kind. The phrase "balanced parentheses" or "number of binary trees" wants Catalan numbers. The phrase "$v_p(\binom{n}{k})$" wants Kummer's theorem.

Chapter 19

Number Theory Tricks

This chapter collects miscellaneous tricks that are useful in competitive programming but do not fit cleanly into any other chapter. Each section is self-contained.

19.1 Josephus Problem

The Josephus problem: n people stand in a circle, and we eliminate every k-th person; what is the survivor's index? For $k = 2$ there is a beautiful closed-form:

Theorem 19.1. *For $k = 2$, the survivor's index (0-indexed) is $J(n) = 2(n - 2^{\lfloor \log_2 n \rfloor}) + 1$ (1-indexed), or equivalently $J(n) = n - 2^{\lfloor \log_2 n \rfloor} + (n - 2^{\lfloor \log_2 n \rfloor}) + 1$, which simplifies to a bit-shift trick: write n in binary, move the leading 1 to the end.*

Proof. The recurrence is $J(1) = 0$ (0-indexed), $J(2n) = 2J(n) - 1$, $J(2n + 1) = 2J(n) + 1$. By induction, $J(n) = 2(n - 2^k) + 1$ where $2^k \le n < 2^{k+1}$. Equivalently, $J(n)$ is the left-cyclic shift of the binary representation of n. □

```cpp
long long josephus2(long long n) {
    long long h = 1LL << (63 - __builtin_clzll(n));  // highest power of 2 <=
        n
    return (n - h) * 2 + 1;                          // 1-indexed
}
```

Listing 19.1: Josephus survivor for $k = 2$ in $\mathcal{O}(\log n)$.

For general k, the recurrence is $J(1) = 0$ (0-indexed), $J(n) = (J(n - 1) + k) \bmod n$, computable in $\mathcal{O}(n)$ iteratively. For very large n with small k, there are $\mathcal{O}(k \log n)$ and $\mathcal{O}(\sqrt{n})$ algorithms.

19.2 Gray Code

A Gray code is a sequence $g_0, g_1, \ldots, g_{2^n-1}$ of all n-bit binary numbers such that consecutive values differ in exactly one bit.

Theorem 19.2. *The standard binary-reflected Gray code is* $g(n) = n \oplus (n \gg 1)$.

```
long long to_gray(long long n)  { return n ^ (n >> 1); }
long long from_gray(long long g) {
    long long n = g;
    while (g >>= 1) n ^= g;
    return n;
}
```

Listing 19.2: Gray code encoding and decoding.

19.3 Fast Subset Enumeration (SOS DP)

Given a function $f : 2^{[n]} \to \mathbb{R}$, the sum-over-subsets (SOS) DP computes $g(S) = \sum_{T \subseteq S} f(T)$ in $\mathcal{O}(n2^n)$ time, much faster than the naive $\mathcal{O}(3^n)$.

```
void sos_dp(vector<long long> &f, int n) {
    // f has size 2^n; output is the same array, with f[S] = sum_{T subset S}
        f_0(T).
    for (int i = 0; i < n; ++i)
        for (int mask = 0; mask < (1 << n); ++mask)
            if (mask & (1 << i))
                f[mask] += f[mask ^ (1 << i)];
}
```

Listing 19.3: SOS DP: $g(S) = \sum_{T \subseteq S} f(T)$ in $\mathcal{O}(n2^n)$.

19.4 Popcount and Bit Manipulation

```
int popcount(unsigned long long x)  { return __builtin_popcountll(x); }
int clz(unsigned long long x)      { return __builtin_clzll(x); }
int ctz(unsigned long long x)       { return __builtin_ctzll(x); }
int lg2(unsigned long long x)       { return 63 - __builtin_clzll(x); }
bool is_pow2(unsigned long long x)  { return x && !(x & (x - 1)); }
```

```
long long next_pow2(unsigned long long x) { return x == 1 ? 1 : 1ULL << (64 -
    __builtin_clzll(x - 1)); }
long long rev_bits(unsigned long long x, int n) {
    // reverse n lowest bits of x
    long long r = 0;
    for (int i = 0; i < n; ++i) r |= ((x >> i) & 1) << (n - 1 - i);
    return r;
}
```

Listing 19.4: Bit manipulation tricks.

The built-in GCC functions `__builtin_popcount`, `__builtin_clzll`, `__builtin_ctzll` compile to single machine instructions on x86 with the POPCNT, LZCNT, and TZCNT extensions respectively.

19.5 Hensel Lifting (Newton's Method Modulo p^k)

Given a solution x_0 to $f(x) \equiv 0 \pmod{p}$, Hensel lifting constructs a solution modulo p^k for any k. The formula is the modular analogue of Newton's iteration:

$$x_{i+1} \equiv x_i - f(x_i) \cdot f'(x_i)^{-1} \pmod{p^{2^{i+1}}}$$

provided $f'(x_i) \not\equiv 0 \pmod{p}$ ("non-singular" case). Each step doubles the exponent, giving $\mathcal{O}(\log k)$ iterations to reach p^k.

Theorem 19.3 (Hensel's lemma)**.** *If $f(x_0) \equiv 0 \pmod{p}$ and $f'(x_0) \not\equiv 0 \pmod{p}$, then there is a unique $x \in \mathbb{Z}/p^k\mathbb{Z}$ with $f(x) \equiv 0 \pmod{p^k}$ and $x \equiv x_0 \pmod{p}$, obtained by Newton iteration.*

Hensel lifting is used to solve $x^2 \equiv a \pmod{p^k}$ (square roots modulo prime powers, generalising Tonelli–Shanks), $x^k \equiv a \pmod{p^k}$, and similar congruences modulo prime powers.

```
long long hensel_sqrt(long long a, long long p, int e) {
    // x^2 = a mod p^e, given x0 = sqrt(a) mod p (Tonelli-Shanks)
    long long x0 = tonelli_shanks(a, p);
    if (x0 == -1) return -1;
    long long pe = 1; for (int i = 0; i < e; ++i) pe *= p;
    long long x = x0 % p;
    long long cur_p = p;
    while (cur_p < pe) {
```

```
        // Newton: x = x - (x^2 - a) * (2x)^{-1} mod (p*cur_p)
        long long inv = binpow(2 * x % cur_p, cur_p / p * (p - 2) - 1, cur_p
            * p);
        x = (x - (x * x - a) / cur_p * inv % (cur_p * p) + cur_p * p) % (
            cur_p * p);
        cur_p *= p;
    }
    return x;
}
```

Listing 19.5: Hensel lift $x^2 \equiv a \pmod{p^e}$ from a solution mod p.

(The implementation above is schematic; production code carefully handles the modulus at each step.)

19.6 Pigeonhole in Modular Arithmetic

If $a_1, a_2, \ldots, a_{m+1}$ are integers, then by pigeonhole two of them are congruent modulo m. The classic application: among $n+1$ integers in $\{0, 1, \ldots, 2n\}$, two are equal. A modular version: among n integers, there is a nonempty subset with sum divisible by n (consider the prefix sums mod n; if any is 0 we are done, otherwise two prefix sums are equal by pigeonhole and their difference is a subset sum divisible by n).

19.7 Other Useful Identities

- **Hockey stick:** $\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$.
- **Vandermonde:** $\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$.
- **Modular square root by Euler's criterion** $a^{(p+1)/4} \bmod p$ when $p \equiv 3 \pmod 4$ (special case of Tonelli–Shanks).
- **Lifting the Exponent Lemma (LTE):** for prime p and $p \nmid a$, $p \nmid b$ with $p \mid a - b$, $v_p(a^n - b^n) = v_p(a-b) + v_p(n)$ for p odd (with extensions for $p = 2$).

19.8 Pattern Recognition

Pattern Recognition. The phrase "every k-th person" wants Josephus (the closed form for $k = 2$, the iterative formula for general k). The phrase "Gray code" or "consecutive values differ by one bit" wants the Gray code formula. The phrase "count subsets whose sum has property P" with $n \leq 20$ wants SOS DP or meet-in-the-middle. The phrase "$\sqrt{a} \bmod p^e$" wants Hensel lifting. The phrase "at least two of the integers" or "subset with sum divisible by n" wants pigeonhole.

Chapter 20

Generating Functions and Formal Power Series

Generating functions translate combinatorial recurrences into algebraic identities on formal power series. The single trick "encode the sequence as a power series, manipulate algebraically, extract coefficients" turns hard combinatorial problems into routine algebra. This chapter covers ordinary and exponential generating functions, the rules for translating combinatorial constructions, and several worked competitive-programming examples that combine generating functions with NTT-based polynomial multiplication.

20.1 Ordinary Generating Functions

Definition 20.1 (OGF)**.** The *ordinary generating function* (OGF) of a sequence $a_0, a_1, a_2, \ldots$ is the formal power series

$$A(x) = \sum_{n \geq 0} a_n x^n.$$

We say $a_n = [x^n]A(x)$ is the coefficient of x^n in A.

The correspondence between sequences and power series is one-to-one: the sequence a_n uniquely determines $A(x)$, and vice versa. The power of generating functions comes from the dictionary between sequence operations and series operations:

Theorem 20.2 (OGF dictionary)**.** *Let $A(x), B(x)$ be the OGFs of $(a_n), (b_n)$ respectively.*

1. ***Shift right by** k: the OGF of $(0, \ldots, 0, a_0, a_1, \ldots)$ (with k zeros) is $x^k A(x)$.*
2. ***Shift left by** k: the OGF of $(a_k, a_{k+1}, \ldots)$ is $(A(x) - a_0 - a_1 x - \cdots - a_{k-1}x^{k-1})/x^k$.*

3. ***Partial sums:*** *the OGF of* $(a_0, a_0 + a_1, a_0 + a_1 + a_2, \ldots)$ *is* $A(x)/(1 - x)$.

4. ***Convolution:*** *the OGF of* $c_n = \sum_{k=0}^{n} a_k b_{n-k}$ *is* $A(x)B(x)$.

5. ***Hadamard (pointwise) product:*** $\sum a_n b_n x^n$ *= the coefficient of* x^n *in* $A \otimes B$*, computed via the identity* $\sum a_n b_n x^n$ *= the diagonal of the bivariate series* $A(xt) \cdot B(t)/t$.

6. ***Recurrence****: if* $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$ *for* $n \geq k$*, then* $A(x) = P(x)/Q(x)$ *where* $Q(x) = 1 - c_1 x - \cdots - c_k x^k$ *and* $P(x)$ *encodes the initial conditions.*

Sketch of (vi). Multiply the recurrence by x^n and sum from $n = k$ to ∞. The left side is $A(x) -$ (polynomial of degree $< k$). The right side is $c_1 x(A -$ (lower part)$) + \cdots + c_k x^k A(x)$. Solving for $A(x)$ gives a rational function with denominator $1 - c_1 x - \cdots - c_k x^k$. □

The recurrence rule (vi) is the algebraic form of the Berlekamp–Massey + Kitamasa approach of Chapter 16: a linear recurrence corresponds exactly to a rational generating function $A(x) = P(x)/Q(x)$, and computing a_n for large n reduces to extracting the n-th coefficient of P/Q, which is Kitamasa's algorithm.

20.2 Exponential Generating Functions

Definition 20.3 (EGF)**.** The *exponential generating function* (EGF) of $a_0, a_1, \ldots$ is $\hat{A}(x) = \sum_{n \geq 0} a_n x^n / n!$.

EGFs are the right tool when the sequence counts labelled objects (where the order of elements matters). The convolution rule is different:

Theorem 20.4 (EGF convolution)**.** *If* $\hat{A}, \hat{B}$ *are the EGFs of* $(a_n), (b_n)$*, then the EGF of* $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ *is* $\hat{A}(x)\hat{B}(x)$*. The binomial coefficient appears because of the labelling: we choose which* k *of the* n *labels go to the first structure.*

EGFs are particularly suited to combinatorial constructions that involve "sets" or "sequences" of objects:

- **Sequence of objects:** if the OGF of one object is $A(x)$, the OGF of a sequence of zero or more such objects is $1/(1 - A(x))$.
- **Set of objects:** if the EGF of one object is $\hat{A}(x)$, the EGF of a set of zero or more such objects is $\exp(\hat{A}(x))$ (since a set is an unordered collection; the exp accounts for the partition of labels).

- **Cycle of objects:** EGF of cycles is $-\log(1 - \hat{A}(x))$.

These rules are the basis of the *symbolic method* in combinatorics, a powerful framework for translating structural descriptions into generating functions.

20.3 Closed Forms and the Rational Ansatz

Many combinatorial sequences have rational OGFs of the form $A(x) = P(x)/Q(x)$ with $\deg P < \deg Q$. The denominator $Q(x) = \prod(1 - \alpha_i x)^{e_i}$ factors into linear terms over $\mathbb{C}$, and the partial-fraction decomposition gives

$$a_n = \sum_i \mathrm{poly}_i(n)\alpha_i^n,$$

where poly_i is a polynomial of degree $e_i - 1$. This is the closed form of any sequence with a rational OGF.

Example 20.5 (Fibonacci)**.** F_n has OGF $F(x) = x/(1 - x - x^2)$. The roots of $1 - x - x^2 = 0$ are $x = 1/\varphi$ and $x = -1/\psi$ where $\varphi = (1 + \sqrt{5})/2$, $\psi = (1 - \sqrt{5})/2$. The partial fractions give $F_n = (\varphi^n - \psi^n)/\sqrt{5}$ (Binet's formula, Theorem 17.1(i)).

20.4 Newton Iteration for Polynomial Operations

The NTT (Chapter 15) gives us $\mathcal{O}(n \log n)$ polynomial multiplication. From this single primitive, we can solve a wide range of polynomial operations in $\mathcal{O}(n \log n)$ via Newton iteration:

Theorem 20.6 (Newton iteration for power series)**.** *Suppose we want to find $g(x) \in \mathbb{F}_p[[x]]$ with $F(g) = 0$ modulo x^n, given an initial approximation g_0 with $F(g_0) \equiv 0 \pmod{x}$. The Newton update*

$$g_{k+1} = g_k - F(g_k)/F'(g_k)$$

doubles the precision: if $F(g_k) \equiv 0 \pmod{x^{2^k}}$, then $F(g_{k+1}) \equiv 0 \pmod{x^{2^{k+1}}}$.

The proof is the formal power series analogue of Newton's method in numerical analysis; the doubling of precision follows from Taylor expansion $F(g_k + h) = F(g_k) + F'(g_k)h + O(h^2)$.

Corollary 20.7. *The following operations on a power series f of degree n are computable in $\mathcal{O}(n \log n)$ via Newton iteration:*

- ***Inverse:*** *g with $fg \equiv 1 \pmod{x^{n+1}}$. Update: $g_{k+1} = g_k(2 - fg_k)$.*
- ***Square root:*** *g with $g^2 \equiv f \pmod{x^{n+1}}$. Update: $g_{k+1} = (g_k + f/g_k)/2$.*
- ***Logarithm:*** $\log f = \int f'/f \, dx$ *(requires $f(0) = 1$).*
- ***Exponential:*** $\exp f$ *via $g_{k+1} = g_k(1 - \log g_k + f)$.*

20.5 Worked Example: Counting Labeled Trees (Cayley)

Worked Problem. Count the number of labeled rooted trees on n vertices.

Solution. By the symbolic method: a rooted tree is a vertex with a set of rooted trees attached. The EGF satisfies $T(x) = x \exp(T(x))$ (the outer x for the root vertex, exp for the set of subtrees). Lagrange inversion gives $T_n = n^{n-1}$ (Cayley's formula). For a competitive programming problem asking $T_n \bmod p$ for large n, we use the polynomial exponential via Newton iteration (NTT-based) on $T(x) = x \exp(T(x))$. □

20.6 Worked Example: Subset Sum Generating Function

Worked Problem. Given a multiset S of integers, count the number of subsets with sum k for each k.

Solution. The OGF for the count is $\prod_{s \in S}(1 + x^s)$. Computing this product modulo x^{k+1} via divide-and-conquer NTT takes $\mathcal{O}(k \log^2 k \log |S|)$. For $|S| \le 10^5$ and $k \le 10^5$, this fits within time limits. □

20.7 Pattern Recognition

Pattern Recognition. The phrases "count subsets with property P", "number of trees on n vertices", "number of sequences with sum k" often want generating functions. The phrases "$[x^n]F(x) \bmod p$" or "compute the n-th coefficient of a power series" want NTT plus Newton iteration. The phrase "the sequence satisfies a linear recurrence" wants the rational OGF P/Q and Kitamasa. Generating functions are the bridge between combinatorics and algebra, and NTT is the engine that makes generating-function manipulation practical in contests.

Chapter 21

Continued Fractions and Rational Approximation

Continued fractions provide the best rational approximations to real numbers and the solution to Pell's equation. They appear in competitive programming whenever a problem requires finding a rational number close to an irrational one, or solving $x^2 - Dy^2 = 1$ for nonsquare D.

21.1 Definitions

Definition 21.1 (Continued fraction)**.** A (simple) continued fraction is an expression

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \cdots}}}$$

with $a_0 \in \mathbb{Z}$ and $a_i \in \mathbb{N}$ for $i \geq 1$. We write $[a_0; a_1, a_2, \ldots]$. If the sequence terminates, it represents a rational number; if it is infinite, it represents an irrational number uniquely.

The Euclidean algorithm naturally produces a continued fraction expansion: the partial quotients $a_0, a_1, \ldots$ are the quotients in the Euclidean chain.

```
vector<long long> cf(long long p, long long q) {
    vector<long long> a;
    while (q) {
        a.push_back(p / q);
```

```
        p %= q; swap(p, q);
    }
    return a;
}
```

Listing 21.1: Continued fraction expansion of a rational number p/q.

21.2 Convergents

Definition 21.2 (Convergent)**.** The n-th convergent of $[a_0; a_1, a_2, \ldots]$ is $p_n/q_n = [a_0; a_1, \ldots, a_n]$.

Theorem 21.3 (Convergent recurrence)**.** *The convergents satisfy the recurrence*

$$p_n = a_n p_{n-1} + p_{n-2}, \qquad q_n = a_n q_{n-1} + q_{n-2},$$

with $p_{-1} = 1, p_{-2} = 0, q_{-1} = 0, q_{-2} = 1$.

Proof. By induction on n. For $n = 0$: $p_0/q_0 = a_0/1$, and the recurrence gives $p_0 = a_0 p_{-1} + p_{-2} = a_0 \cdot 1 + 0 = a_0$, $q_0 = a_0 q_{-1} + q_{-2} = 0 + 1 = 1$. For the inductive step: $[a_0; \ldots, a_{n+1}] = [a_0; \ldots, a_n + 1/a_{n+1}]$, so the convergent is a linear function of a_{n+1} and the recurrence extends. □

Theorem 21.4 (Best rational approximation)**.** *If p_n/q_n is a convergent of an irrational α and p/q is any other rational with $0 < q \le q_n$, then $|q\alpha - p| > |q_n\alpha - p_n|$.*

In other words, the convergents are the best rational approximations in the sense of minimizing $|q\alpha - p|$ for bounded q. The competitive-programming use: given a target real α and a bound Q on the denominator, find the best p/q with $q \le Q$ approximating α.

```
pair<long long,long long> best_rational(double alpha, long long Q) {
    // Returns p/q with |q*alpha - p| minimal, 1 <= q <= Q.
    long long p0 = 0, q0 = 1, p1 = 1, q1 = 0;
    double a = alpha;
    while (true) {
        long long ai = (long long) floor(a);
        long long p2 = ai * p1 + p0;
        long long q2 = ai * q1 + q0;
        if (q2 > Q) {
            // Adjust ai so q2 fits; the last good one is (p1*floor((Q-q0)/q1
                ) + p0)/(q1*floor((Q-q0)/q1) + q0)
```

```
            long long k = (Q - q0) / q1;
            return {k * p1 + p0, k * q1 + q0};
        }
        p0 = p1; q0 = q1; p1 = p2; q1 = q2;
        if (a == (double) ai) break;
        a = 1.0 / (a - ai);
    }
    return {p1, q1};
}
```

Listing 21.2: Best rational approximation to α with denominator $\le Q$.

21.3 Pell's Equation

Definition 21.5 (Pell's equation)**.** For nonsquare $D \in \mathbb{N}$, Pell's equation is $x^2 - Dy^2 = 1$ in integers x, y.

Theorem 21.6. *Pell's equation $x^2 - Dy^2 = 1$ has infinitely many solutions for nonsquare D. The minimal solution (x_1, y_1) is given by (p_n, q_n) where n is the period of the continued fraction of $\sqrt{D}$. All solutions are (x_k, y_k) with $x_k + y_k\sqrt{D} = (x_1 + y_1\sqrt{D})^k$.*

Sketch. $\sqrt{D}$ has a periodic continued fraction $[a_0; \overline{a_1, a_2, \ldots, a_n, 2a_0}]$. The convergent p_n/q_n at the end of the first period gives the minimal solution to Pell. The general solutions come from the multiplication rule $(x_1 + y_1\sqrt{D})(x_2 + y_2\sqrt{D}) = (x_1x_2 + Dy_1y_2) + (x_1y_2 + x_2y_1)\sqrt{D}$. □

```
pair<long long, long long> pell(long long D) {
    long long a0 = (long long) sqrt((double) D);
    // Check D not a perfect square (else no nontrivial solution)
    if (a0 * a0 == D) return {0, 0};  // D square
    long long m = 0, d = 1, a = a0;
    long long p0 = 1, p1 = a0, q0 = 0, q1 = 1;
    while (p1 * p1 - D * q1 * q1 != 1) {
        m = d * a - m;
        d = (D - m * m) / d;
        a = (a0 + m) / d;
        long long p2 = a * p1 + p0;
        long long q2 = a * q1 + q0;
        p0 = p1; p1 = p2; q0 = q1; q1 = q2;
```

```
    }
    return {p1, q1};
}
```

Listing 21.3: Solve Pell's equation $x^2 - Dy^2 = 1$ for nonsquare D.

The minimal solution can be exponentially large in $\sqrt{D}$, so for $D \approx 10^{18}$ this becomes infeasible with `long long`. With `__int128` or BigInt, D up to 10^9 is practical.

Worked Problem (SPOJ `PELL` — Pell Equation). **Statement.** For each query D, find the minimal positive solution to $x^2 - Dy^2 = 1$.

Solution sketch. Continued fraction of $\sqrt{D}$. The minimal solution (x_1, y_1) is found in $\mathcal{O}(\sqrt{D})$ iterations of the convergent recurrence; for typical $D \leq 10^9$ this runs in milliseconds.

21.4 Applications

1. **Rational reconstruction.** Given $a \bmod m$ for coprime a, m, recover a if we know a is small. Continued fraction of m/a gives convergents p_n/q_n; the smallest p_n with $ap_n \equiv q_n \pmod{m}$ gives the answer.
2. **Floating-point to fraction.** "Given α as a float, find the smallest-denominator rational in $[\alpha - \epsilon, \alpha + \epsilon]$." This is the convergent problem above.
3. **Pell equations.** "Find all solutions to $x^2 - Dy^2 = 1$" or "find the smallest solution."
4. **Solving $ax + by = c$ with smallest $|x| + |y|$.** Continued fractions give the best rational approximations.

21.5 Pattern Recognition

Pattern Recognition. The phrase "Pell's equation $x^2 - Dy^2 = 1$" wants continued fractions of $\sqrt{D}$. The phrase "best rational approximation to α with denominator $\leq Q$" wants convergents. The phrase "rational reconstruction" (recover an integer from its modular residue) wants the continued fraction of m/a. The phrase "find x, y with $|ax - by| = 1$ and x, y small" wants the convergent that gives the Bézout coefficients with smallest magnitudes. Continued fractions are the universal tool for rational approximation problems.

Chapter 22

Advanced Topics: Lucas Sequences, Beatty Sequences, and Prime Counting

This chapter collects several advanced topics that appear less often but are useful in harder contests: Lucas sequences (generalising Fibonacci), Beatty sequences (related to irrational rotation), and fast prime-counting algorithms.

22.1 Lucas Sequences

Definition 22.1 (Lucas sequences)**.** For integers P, Q with $D = P^2 - 4Q \neq 0$, the Lucas sequences are

$$U_n(P,Q) = \frac{\alpha^n - \beta^n}{\alpha - \beta}, \qquad V_n(P,Q) = \alpha^n + \beta^n,$$

where α, β are the roots of $x^2 - Px + Q = 0$.

The Fibonacci numbers are $U_n(1,-1)$ and the Lucas numbers are $V_n(1,-1)$. The Lucas sequences satisfy analogues of the Fibonacci identities:

Theorem 22.2 (Lucas identities)**.** *For all $n, m \geq 0$:*

- $U_0 = 0, U_1 = 1, U_{n+1} = PU_n - QU_{n-1}$.
- $V_0 = 2, V_1 = P, V_{n+1} = PV_n - QV_{n-1}$.
- $U_{2n} = U_n V_n$ *and* $V_{2n} = V_n^2 - 2Q^n$.

- $U_{m+n} = U_m U_{n+1} - Q U_{m-1} U_n$.
- $\gcd(U_m, U_n) = U_{\gcd(m,n)}$ *when* $\gcd(P, Q) = 1$.

The fast doubling formulas (Theorem 17.2 for Fibonacci) generalise to Lucas sequences, giving $\mathcal{O}(\log n)$ computation of $U_n \bmod p$ and $V_n \bmod p$.

```
pair<long long, long long> lucas_doubling(long long n, long long P, long long
     Q, long long p) {
    // Returns (U_n, V_n) mod p
    if (n == 0) return {0, 2 % p};
    auto [u, v] = lucas_doubling(n >> 1, P, Q, p);
    long long Un = (__int128) u * v % p;
    long long Vn = ((__int128) v * v % p - 2 * binpow(Q, n >> 1, p) % p + 2 *
         p) % p;
    if (n & 1) {
        long long Un1 = ((__int128) P * Un % p - (__int128) Q * u % p + p) %
            p;
        long long Vn1 = ((__int128) P * Vn % p - (__int128) Q * v % p + p) %
            p;
        return {Un1, Vn1};
    }
    return {Un, Vn};
}
```

Listing 22.1: Compute $U_n(P, Q) \bmod p$ via fast doubling in $\mathcal{O}(\log n)$.

The Lucas sequences are central in the Lucas–Lehmer primality test for Mersenne numbers $M_p = 2^p - 1$, which checks $V_{p-1}(M_p, 1) \equiv 0 \pmod{M_p}$. They also appear in Pollard's $p-1$ (when $p-1$ is smooth, the order of a in $(\mathbb{Z}/p\mathbb{Z})^\times$ divides $p-1$, which is a Lucas-type argument).

22.2 Beatty Sequences

Definition 22.3 (Beatty sequence)**.** For irrational $\alpha > 1$, the Beatty sequence B_α is $\{\lfloor n\alpha \rfloor : n \in \mathbb{N}\}$.

Theorem 22.4 (Beatty's theorem, Rayleigh–Beatty)**.** *If $\alpha, \beta > 1$ are irrationals with $1/\alpha + 1/\beta = 1$, then B_α and B_β partition $\mathbb{N}$.*

Proof. $|B_\alpha \cap [1, N]| = \lfloor N/\alpha \rfloor$ and similarly for B_β. Summing, $\lfloor N/\alpha \rfloor + \lfloor N/\beta \rfloor = N$ because

$1/\alpha + 1/\beta = 1$ implies $\lfloor N/\alpha \rfloor + \lfloor N/\beta \rfloor \in \{N-1, N\}$, and irrationally of α, β rules out $N-1$. So the two sequences cover $\{1, \ldots, N\}$ exactly, for every N. □

Beatty's theorem gives a slick solution to problems like "partition $\{1, 2, 3, \ldots\}$ into two sequences with densities $1/\alpha$ and $1/\beta$." The classic example: $\alpha = \varphi$ (golden ratio), $\beta = \varphi^2$, giving the Wythoff sequences which underlie Wythoff's game.

22.3 Counting Primes: Lucy Hedgehog's Algorithm

The naive sieve of Eratosthenes (Chapter 3) takes $\Theta(n)$ memory, which is impractical for n up to 10^{11}. The Lucy–Hedgehog algorithm computes $\pi(n)$ in $\mathcal{O}(n^{3/4})$ time and $\mathcal{O}(\sqrt{n})$ memory, making it the standard approach for n up to about 10^{11}.

Algorithm 22.5 (Lucy–Hedgehog)**.** The key idea: maintain a function $S(v, p)$ = "the count of integers in $\{2, \ldots, v\}$ that are not divisible by any prime $\leq p$." Initially $S(v, 1) = v - 1$. When we process prime p, we update

$$S(v, p) = S(v, p_{\text{prev}}) - \left(S(v/p, p_{\text{prev}}) - S(p-1, p_{\text{prev}})\right)$$

for each v in our set of values. The values v we care about are $\{n/k : k = 1, \ldots, \sqrt{n}\} \cup \{1, 2, \ldots, \sqrt{n}\}$, of which there are $\mathcal{O}(\sqrt{n})$ distinct ones. The algorithm terminates when $p > \sqrt{n}$, at which point $S(n, \sqrt{n}) = \pi(n)$.

```
long long count_primes(long long n) {
    long long r = (long long) sqrt((double) n);
    while (r * r > n) --r;
    while ((r + 1) * (r + 1) <= n) ++r;
    vector<long long> V;
    for (long long i = 1; i <= r; ++i) V.push_back(n / i);
    for (long long i = V.back() - 1; i >= 1; --i) V.push_back(i);
    // S[i] = number of integers in [2, V[i]] not sieved by primes <= 1
    vector<long long> S(V.size());
    for (size_t i = 0; i < V.size(); ++i) S[i] = V[i] - 1;
    for (long long p = 2; p <= r; ++p) {
        if (S[V.size() - p] == S[V.size() - (p - 1)]) continue;  // p not
            prime
        long long sp = S[V.size() - (p - 1)];  // S(p-1, p-1) = pi(p-1)
```

```
        for (size_t i = 0; i < V.size(); ++i) {
            if (V[i] < p * p) break;
            S[i] -= S[V.size() - V[i] / p] - sp;
        }
    }
    return S[0];
}
```

Listing 22.2: Lucy–Hedgehog prime counting in $\mathcal{O}(n^{3/4}/\log n)$ time and $\mathcal{O}(\sqrt{n})$ space.

The Lucy–Hedgehog algorithm generalises to summing multiplicative functions on primes: to compute $\sum_{p \le n, p \text{ prime}} f(p)$, replace the count $S(v, p)$ with the partial sum and update accordingly. This is the Min_25 sieve, used to compute $\sum_{i=1}^{n} \varphi(i)$, $\sum_{i=1}^{n} \mu(i)$, and similar sums for n up to 10^{10} or 10^{11}.

22.4 Bibliography Note

The serious competitive programmer who wants to go deeper into number theory for contests should consult:

- *CP-Algorithms* (cp-algorithms.com): the canonical online reference for competitive-programming algorithms, including a detailed treatment of each topic in this book.
- *Project Euler* (projecteuler.net): over 800 problems, many requiring nontrivial number theory.
- *Introduction to Algorithms*, Cormen et al.: number-theory chapter.
- *The Art of Computer Programming, Vol. 2*, Knuth: a deep treatment of random number generation, factorisation, and arithmetic.
- *Concrete Mathematics*, Graham, Knuth, Patashnik: combinatorics and recurrence relations.
- *An Introduction to the Theory of Numbers*, Niven, Zuckerman, Montgomery: the standard elementary number theory reference.
- *A Classical Introduction to Modern Number Theory*, Ireland and Rosen: a graduate-level reference covering quadratic reciprocity, primes in arithmetic progressions, and Diophantine equations.

22.5 Pattern Recognition

Pattern Recognition. The phrase "Lucas sequence" or "U_n, V_n" wants the fast doubling formulas for Lucas sequences. The phrase "partition $\mathbb{N}$ into two sequences" or "$\lfloor n\alpha \rfloor$ and $\lfloor n\beta \rfloor$ partition $\mathbb{N}$" wants Beatty's theorem with $1/\alpha + 1/\beta = 1$. The phrase "$\pi(n)$ for $n \leq 10^{11}$" wants the Lucy–Hedgehog sieve. The phrase "$\sum_{p \leq n} f(p)$ for a multiplicative function f" wants the Min_25 sieve (advanced Lucy–Hedgehog). The phrases "Mersenne prime" and "Lucas–Lehmer test" want Lucas sequence evaluation. Each of these is a small but identifiable niche that appears in harder contests.

Chapter 23

Pattern Recognition Guide

This chapter consolidates the pattern-recognition notes scattered throughout the previous twenty-two chapters into a single, organised reference. Given a competitive programming problem statement, the goal is to identify—from a few signal words and constraints—which chapter's technique is likely to apply. We give three views: (i) a list of "signal phrases" and the techniques they suggest, (ii) a constraint-driven decision table ("given the bounds on n and queries, which algorithm fits?"), and (iii) a final consolidated table of problem signatures.

23.1 Signal Phrases and Likely Techniques

The following table maps signal phrases (extracted from typical Codeforces / AtCoder / SPOJ problem statements) to the most likely chapter. The mapping is heuristic: a problem rarely uses only one technique, and the same phrase may suggest multiple chapters depending on context.

Signal phrase in problem	**Likely technique**
"Print modulo $10^9 + 7$"	Modular arithmetic (Ch. 5); φ/Fermat if division needed
"Compute $a^b \bmod m$"	Binary exponentiation (Ch. 5)
"Tower of powers $a^{b^{c^{\cdots}}} \bmod m$"	Euler totient reduction (Ch. 7, Thm 7.7)
"Count pairs with $\gcd = 1$"	Möbius inversion (Ch. 10)
"Count pairs with $\gcd = k$"	Möbius inversion with k-shift (Ch. 10)
"Sum $\gcd(a, b)$ over $a, b \le n$"	φ + summation (Ch. 7, 10)
"Sum $f(\gcd(a, b))$ for some function f"	Dirichlet convolution (Ch. 11)

Signal phrase in problem	**Likely technique**
"Solve $ax \equiv b \pmod{m}$"	Linear congruence (Ch. 5, Thm 5.4)
"Solve $ax + by = c$ in integers"	Linear Diophantine (Ch. 4, Thm 4.4)
"Combine $x \equiv a_i \pmod{m_i}$"	CRT (Ch. 6)
"$\sqrt{a} \bmod p$" or "$x^2 \equiv a \pmod{p}$"	Tonelli–Shanks / Cipolla (Ch. 12)
"$x^k \equiv a \pmod{p}$, find all x"	Discrete root: primitive roots + BSGS + linear congruence (Ch. 13, 14)
"$g^x \equiv h \pmod{p}$"	BSGS or Pohlig–Hellman (Ch. 14)
"Is n prime?" ($n \le 10^{18}$)	Miller–Rabin deterministic (Ch. 8)
"Factorise n" ($n \le 10^{18}$)	Miller–Rabin + Pollard's rho (Ch. 8, 9)
"Factorise n" ($n \le 10^7$) with many queries	Linear sieve producing spf (Ch. 3)
"List primes in $[m, n]$" ($n \le 10^{12}$)	Segmented sieve (Ch. 3)
"Precompute $\varphi/\mu/\tau/\sigma$ for $i \le n$" ($n \le 10^7$)	Linear sieve (Ch. 3)
"n-th Fibonacci number mod m" ($n \le 10^{18}$)	Fast doubling (Ch. 17)
"n-th term of a linear recurrence" ($n \le 10^{18}$, $k \le 100$)	Matrix exponentiation (Ch. 16)
"n-th term of a linear recurrence" ($k \le 1000$)	Kitamasa (Ch. 16)
"Sequence defined by complicated DP, find 10^{18}-th term"	Berlekamp–Massey + Kitamasa (Ch. 16)
"Multiply polynomials modulo 998244353"	NTT (Ch. 15)
"Multiply polynomials modulo $10^9 + 7$"	MTT: triple-modulus NTT + Garner (Ch. 6, 15)
"Polynomial inverse / division / log / exp"	NTT + Newton iteration (Ch. 15)
"$\binom{n}{k} \bmod p$ with $n \le 10^{18}$, $p \le 10^6$"	Lucas's theorem (Ch. 18)
"$\binom{n}{k} \bmod p$ with $n, k \le 10^6$"	Precomputed factorials (Ch. 18)
"Number of partitions into k non-empty sets"	Stirling numbers of the 2nd kind (Ch. 18)
"Number of balanced parenthesizations"	Catalan numbers (Ch. 18)
"Survivor in circle, eliminate every 2nd"	Josephus closed form (Ch. 19)
"Subset with property P in $[n]$, $n \le 20$"	SOS DP or meet-in-the-middle (Ch. 19)
"Square root modulo p^k"	Hensel lifting (Ch. 19)

Signal phrase in problem	**Likely technique**
"Compute $[x^n]F(x) \bmod p$" or "n-th coefficient of power series"	Generating functions + NTT + Newton (Ch. 20)
"Count labeled trees / set partitions"	EGF and exp (Ch. 20)
"Pell equation $x^2 - Dy^2 = 1$"	Continued fraction of $\sqrt{D}$ (Ch. 21)
"Best rational approximation to α with denominator $\le Q$"	Convergents (Ch. 21)
"Rational reconstruction" (recover small integer from residue)	Continued fraction of m/a (Ch. 21)
"Count primes below n" ($n \le 10^{11}$)	Lucy–Hedgehog sieve (Ch. 22)
"Mersenne prime" or "Lucas–Lehmer test"	Lucas sequences (Ch. 22)
"$\lfloor n\alpha \rfloor$ and $\lfloor n\beta \rfloor$ partition $\mathbb{N}$"	Beatty's theorem (Ch. 22)

23.2 Constraint-Driven Decision

The constraints on n (the main input size) and the number of queries are usually the strongest signals. The following table gives the algorithm of choice for typical (n, q) pairs in "factor / prime / counting" problems.

Problem type	Constraint on n	Best algorithm
Primality testing	$n \le 10^7$	Trial division u
Primality testing	$n \le 10^{14}$	Trial division u
Primality testing	$n \le 10^{18}$	Deterministic M
Factorisation	$n \le 10^{14}$	Trial division (
Factorisation	$n \le 10^{18}$	MR + Pollard'
Factorisation per query, many queries, $n \le 10^7$	Linear sieve + $\mathcal{O}(\log n)$ per query (Ch. 3)	
Prime counting	$n \le 10^7$	Sieve of Eratos
Prime counting	$n \le 10^{11}$	Lucy Hedgehog
$\varphi(n)$ for one n	$n \le 10^{14}$	$\mathcal{O}(\sqrt{n})$ factoris
$\varphi(n)$ for all $n \le N$	$N \le 10^7$	Linear sieve (C
$\varphi(n)$ for many $n \le N$	$N \le 10^{12}$	Sieve + Möbiu

Binomial mod$p$	$n, k \le 10^6$, p prime	Precomputed f
Binomial mod$p$	$n \le 10^{18}$, $p \le 10^6$	Lucas's theore
Binomial modp^e	$n \le 10^{18}$, $p \le 10^6$, $e \ge 1$	Granville–And
Prime counting	$n \le 10^{11}$	Lucy–Hedgeho
Pell's equation $x^2 - Dy^2 = 1$	$D \le 10^9$	Continued frac
Best rational approx. to α	$Q \le 10^{18}$	Convergents of
Counting labeled trees, set partitions	$n \le 10^5$, mod prime	EGF + Newto

23.3 The Constraint Hierarchy

When facing a number-theory problem in contest, apply the following reasoning hierarchy:

1. **Identify the modulus.** If the problem says "modulo p" for prime p, you are working in $\mathbb{F}_p$. If $p = 10^9 + 7$ (or 998244353), expect Fermat modular inverses, NTT, and so on. If the modulus is composite, factorise it (you might use CRT to split the problem into smaller pieces).

2. **Read the input bounds.** The constraint on n tells you which algorithm is feasible: $\mathcal{O}(n)$ is fine up to 10^7, $\mathcal{O}(n \log n)$ up to 10^6, $\mathcal{O}(\sqrt{n})$ up to 10^{14}, $\mathcal{O}(\log n)$ for n up to 10^{18}. The reflex "n up to 10^{18}" should instantly trigger a search for $\mathcal{O}(\log n)$ or $\mathcal{O}(\sqrt{n})$ algorithms.

3. **Read the queries.** If there are q queries each on input of size n, the per-query cost is $\mathcal{O}(T/q)$ where T is the time budget. For $q = 10^5$ and $T = 10^9$ operations, each query gets $\mathcal{O}(10^4)$, which fits $\mathcal{O}(\sqrt{n})$ for n up to 10^8.

4. **Identify the algebraic structure.** Is the problem in $\mathbb{F}_p$? In $\mathbb{Z}/n\mathbb{Z}$? In $\mathbb{Z}/(p-1)\mathbb{Z}$? In the polynomial ring $\mathbb{F}_p[x]$? The structure tells you which operations are valid (e.g., inverses, square roots, logarithms).

5. **Identify the operation.** Is it a count, a sum, an extraction of an element, a query, or a transformation? Counts want combinatorial identities; sums want Dirichlet convolution; extraction wants $\mathcal{O}(\log n)$ fast exponentiation or BSGS; transformations want NTT or polynomial algebra.

6. **Combine.** The trickiest problems combine two or three of these. The classic example is "multiply polynomials with large coefficients modulo $10^9 + 7$," which combines NTT

(Ch. 15) with three-modulus arithmetic and Garner (Ch. 6).

23.4 The Consolidated Signature Table

The final table maps high-level problem signatures ("the problem has this kind of statement and these constraints") to a single chapter. This is the cheat sheet the author uses in actual contests.

Problem signature	**Chapter**
"Modulo p" + simple arithmetic	Ch. 5 (Modular arithmetic)
"Modulo m" (composite) + tower of exponents	Ch. 7 (φ reduction)
"Modulo p" + division	Ch. 5 (modular inverse)
"Modulo m_i" for several m_i	Ch. 6 (CRT)
"gcd in the answer"	Ch. 4 (Euclidean) or Ch. 10 (Möbius)
"φ, σ, τ, μ"	Ch. 7, 10, 11
"Prime factorisation" + $n \le 10^{18}$	Ch. 8, 9 (MR + Pollard)
"Mod p^e" (prime power)	Ch. 19 (Hensel)
"Quadratic equation mod p"	Ch. 12 (Tonelli–Shanks)
"Cyclic group exponent equation"	Ch. 13, 14 (primitive roots + BSGS)
"Polynomial multiplication"	Ch. 15 (NTT)
"Polynomial inverse / log / exp"	Ch. 15 (NTT + Newton)
"Linear recurrence" + $n \le 10^{18}$	Ch. 16 (matrix expo / Kitamasa)
"Sequence defined implicitly" + $n \le 10^{18}$	Ch. 16 (Berlekamp–Massey)
"Fibonacci" + "mod"	Ch. 17 (fast doubling / Pisano)
"Binomials" + large n or prime modulus	Ch. 18 (Lucas / precompute)
"Subsets" + $n \le 20$	Ch. 19 (SOS / meet-in-middle)
"Pigeonhole argument"	Ch. 19
"Generating function" or "$[x^n]F(x)$ mod p"	Ch. 20 (Generating functions)
"Pell's equation $x^2 - Dy^2 = 1$"	Ch. 21 (Continued fractions)
"Best rational approx. to α" or "rational reconstruction"	Ch. 21 (Convergents)
"$\pi(n)$ for $n \le 10^{11}$"	Ch. 22 (Lucy–Hedgehog)

"Mersenne prime / Lucas–Lehmer" or "$U_n(P,Q), V_n(P,Q)$"	Ch. 22 (Lucas sequences)
"Beatty sequence" or "partition $\mathbb{N}$ into $\lfloor n\alpha \rfloor, \lfloor n\beta \rfloor$"	Ch. 22 (Beatty)

23.5 Final Heuristics

A few heuristics that don't fit the tables but are useful in contest:

1. **When in doubt, factorise.** A surprising fraction of number-theory problems reduce to operations on the prime factorisation of n. If you are stuck, factorising n and listing the prime powers is a productive first step.

2. **Watch for $\varphi(m)$ in the exponent.** Any time the answer involves $a^k \bmod m$ with k huge, you should be reducing k modulo $\varphi(m)$. The tower-of-powers reduction (Theorem 7.7) is the universal technique.

3. **Watch for Möbius when the answer is "count with $\gcd = 1$".** Any time the problem counts pairs with coprimality, the substitution $[\gcd = 1] = \sum_{d \mid \gcd} \mu(d)$ is the universal first step.

4. **Watch for NTT when the answer is a polynomial.** Polynomials in the answer suggest NTT for fast multiplication, polynomial inverse / log / exp via Newton iteration, or generating functions.

5. **Watch for "n up to 10^{18}".** This is the universal signal that the algorithm must be $\mathcal{O}(\log n)$ or $\mathcal{O}(\sqrt{n})$, never $\mathcal{O}(n)$. Match the structure (recurrence? fast doubling. Tower? φ reduction. Power? binary exponentiation. Polynomial? NTT.) to the appropriate $\mathcal{O}(\log n)$ technique.

Chapter 24

Exercises and Practice Problems

This chapter collects over 150 practice problems spanning the topics of the previous chapters. They are organised by chapter, with the difficulty (E—Easy, M—Medium, H—Hard) noted. Hints are given at the end of each problem in italics; full solutions are deliberately omitted—solving the problems yourself is the only way to build the pattern-recognition reflexes that this book aims to teach. Most of the problems are taken from Codeforces, AtCoder, SPOJ, Project Euler, or the ICPC regionals; the reader is encouraged to look up the original problems online to verify their solutions.

24.1 Divisibility, Primes, and Sieves (Chapters 2–3)

E1 (SPOJ `PRIME1`) Given T test cases, each with $m \le n \le 10^9$ and $n - m \le 10^5$, print every prime in $[m, n]$. *Hint: segmented sieve.*

E2 (Project Euler 10) Sum of primes below 2 million. *Hint: sieve of Eratosthenes.*

E3 (Codeforces 26A) Count integers $\le n$ with exactly two distinct prime factors.*Hint: linear sieve with ω.*

E4 (Project Euler 3) Largest prime factor of 600851475143. *Hint: trial division.*

M1 (Codeforces 231B) Given n, count the number of pairs (a, b) with $1 \le a, b \le n$ and $a \cdot b$ a perfect square. *Hint: ab is a square iff a and b have the same square-free kernel.*

M2 (Codeforces 776B) Find the colour of each integer from 2 to n if a number is coloured by the number of distinct prime factors. *Hint: linear sieve with ω.*

M3 (AtCoder ABC 170F) For each k from 1 to n, count square-free integers in $[1, k]$. *Hint: sieve computing μ.*

H1 (Project Euler 50) The prime below 10^6 that can be written as the longest sum of consecutive primes. *Hint: prefix sums of primes plus two pointers.*

H2 (Codeforces 665F) Count the number of $n \le 10^{11}$ with $\omega(n) = 2$ (two distinct prime factors). *Hint: Lucy–Hedgehog-like counting.*

24.2 GCD, Euclidean, and Modular Arithmetic (Chapters 4–5)

E1 (Codeforces 7C) Solve $ax + by = c$ in integers, or report no solution. *Hint: extended Euclidean.*

E2 (AtCoder ABC 064C) Given N, count the number of integers in $[0, N]$ divisible by both G and L. *Hint:* lcm.

E3 (SPOJ `CEQU`) Solve $ax + by = c$ in integers. *Hint: extended Euclidean.*

M1 (Codeforces 1105C) Count x in $[l, r]$ with $x \equiv 0, 1, 2 \pmod 3$ separately. *Hint: floor division.*

M2 (Codeforces 1244C) Solve $p \cdot x = n \pmod{2^K}$ for x, or report no solution. *Hint: modular inverse, careful with even moduli.*

M3 (Project Euler 48) Find the last 10 digits of $\sum_{i=1}^{1000} i^i$. *Hint: binary exponentiation under 10^{10}.*

H1 (AtCoder AGC 012B) Solve a system of n linear congruences in one variable. *Hint: pairwise CRT.*

H2 (Codeforces 1152C) Given a, b, find k minimising $\gcd(a + k, b + k)$. *Hint: $\gcd(a + k, b + k) = \gcd(a + k, |b - a|)$; iterate divisors of $|b - a|$.*

24.3 Chinese Remainder Theorem (Chapter 6)

E1 (SPOJ `CRT`) Solve $x \equiv a \pmod{m_1}, x \equiv b \pmod{m_2}$ with m_1, m_2 coprime. *Hint: classical CRT.*

M1 (Codeforces 498E) Solve $x \equiv a_i \pmod{m_i}$ for several (non-coprime) moduli. *Hint: pairwise generalised CRT.*

M2 Given n residues modulo m_1, m_2, m_3 (NTT-friendly primes), reconstruct the integer using Garner's algorithm. *Hint: Garner's algorithm with precomputed inverses.*

M3 (Project Euler 134) For each pair of consecutive primes p_1, p_2 with $p_1 \le 10^6$, find the smallest n with n's last p_1 digits $= p_1$ and $n \equiv 0 \pmod{p_2}$.*Hint: $n = p_1 \cdot k$ with $p_1 k \equiv p_2 \pmod{10^{\lceil \log p_1 \rceil}}$.*

H1 Multiply two polynomials with coefficients up to 10^{18} modulo $10^9 + 7$, length up to 10^5. *Hint: MTT with triple-modulus NTT plus Garner.*

24.4 Euler's Totient (Chapter 7)

E1 (SPOJ `ETF`) Compute $\varphi(n)$ for $n \le 10^{12}$. *Hint: factorise, then formula $\varphi(n) = n \prod (1 - 1/p)$.*

M1 (Codeforces 1135D) For each query, compute the sum of φ in a range. *Hint: precompute φ via linear sieve, prefix sums.*

M2 (Project Euler 70) Find $n \le 10^7$ with $\varphi(n)$ a permutation of n's digits and $n/\varphi(n)$ minimal. *Hint: sieve φ, check permutation, minimise ratio.*

H1 (Codeforces 906D) Compute $a_1^{a_2^{a_3^{\cdots}}} \bmod m$ for $m \le 10^9$. *Hint: recursive Euler reduction with φ.*

H2 Count the number of fractions a/b with $1 \le a, b \le n$ in lowest terms (i.e., $\gcd(a, b) = 1$). *Hint: $\sum_d \varphi(d)$ over $\lfloor n/d \rfloor$.*

24.5 Primality Testing and Factorisation (Chapters 8–9)

E1 (SPOJ `PON`) Decide if $n \le 10^{18}$ is prime, for $T \le 10^5$ queries. *Hint: deterministic Miller–Rabin.*

M1 (Project Euler 3) Find the largest prime factor of 600851475143. *Hint: trial division to $\sqrt{n}$, or Pollard's rho.*

M2 (Codeforces 1032D) Factorise $n \le 10^{18}$ quickly. *Hint: Pollard's rho + Miller–Rabin.*

M3 (Project Euler 12) First triangular number with over 500 divisors. *Hint:* $\tau(n) = \prod(e_i + 1)$ *from prime factorisation.*

H1 (Project Euler 187) Count the semiprimes $\leq 10^8$. *Hint: count primes* $p \leq \sqrt{n}$ *and primes* $q \geq p$ *with* $pq \leq n$.

H2 (Codeforces 1312G) Given a list of n composite numbers, find their smallest prime factor. *Hint: Pollard's rho for each, or sieve if* $n \leq 10^7$.

24.6 Möbius Function and Dirichlet Convolution (Chapters 10–11)

E1 (Project Euler 7) 10001st prime. *Hint: sieve.*

M1 Compute $\sum_{a,b=1}^{n} \gcd(a, b)$. *Hint:* φ *via Dirichlet convolution.*

M2 (Codeforces 803F) Count pairs (a, b) with $1 \leq a, b \leq n$ and $\gcd(a, b) = 1$, modulo $10^9 + 7$. *Hint: Möbius inversion with prefix sums.*

M3 (Codeforces 1135D) Compute $\sum_{i=1}^{n} \varphi(i) \cdot \lfloor n/i \rfloor$. *Hint: identity via Dirichlet.*

H1 (Project Euler 35) Count circular primes below 10^6. *Hint: sieve, check rotations.*

H2 (Codeforces 1559E) Count triples (a, b, c) with $L \leq a, b, c \leq R$ and $\gcd(a, b, c) = 1$. *Hint: Möbius inversion with bounds.*

H3 Compute $\sum_{i=1}^{n} \sigma(i)$ for $n \leq 10^{12}$, where σ is the sum-of-divisors function. *Hint: Dirichlet convolution* $\sigma = 1 * \mathrm{id}$ *and the identity* $\sum_{d \leq n} d = \lfloor n(\lfloor n/d \rfloor + 1)/2 \rfloor$ *over the divisor ranges.*

24.7 Quadratic Residues and Primitive Roots (Chapters 12–13)

E1 (Project Euler 100) Find b with $\binom{b}{blue} = \binom{b+1}{red}/2$. *Hint: Pell equation* $x^2 - 2y^2 = -1$.

M1 Solve $x^2 \equiv a \pmod{p}$ for p prime, a QR. *Hint: Tonelli–Shanks or Cipolla.*

M2 (Codeforces 101755D) Find all x with $x^k \equiv a \pmod{p}$ for prime p. *Hint: primitive roots + BSGS + linear congruence.*

M3 Find a primitive root mod p, $p \leq 10^9$ prime. *Hint: factor* $p - 1$*, test candidates.*

H1 Solve $x^2 \equiv a \pmod{p^e}$ for p odd prime, $e \geq 1$, given a is a QR mod p. *Hint: Hensel lifting on Tonelli–Shanks output.*

H2 (Codeforces 101011F) Given p prime and a QR, count x with $x^2 \equiv a \pmod{p^k}$. *Hint: Hensel + closed form per prime power.*

24.8 Discrete Logarithm (Chapter 14)

E1 (SPOJ `DISCRETLOG`) Find x with $g^x \equiv h \pmod{p}$. *Hint: BSGS.*

M1 (Project Euler 429) Sum of unitary divisors of $n!$ modulo $10^9 + 9$. *Hint: factorisation of $n!$ via Legendre's formula.*

M2 Solve the DLP $g^x \equiv h \pmod{p}$ where $p - 1$ is smooth (all prime factors $\leq 10^3$). *Hint: Pohlig–Hellman.*

H1 Given g, h, p, find the smallest non-negative x with $g^x \equiv h \pmod{p}$, $p \leq 10^{18}$. *Hint: BSGS in $\mathcal{O}(\sqrt{p})$, careful with* log *and space.*

H2 (Project Euler 27) Quadratic primes: $n^2 + an + b$ primes for $n = 0, \ldots, N$, find best (a, b). *Hint: sieve, search, optimise.*

24.9 Number Theoretic Transform (Chapter 15)

E1 (AtCoder ABC 049D) Compute $a \cdot b$ where a, b are polynomials with coefficients modulo 998244353. *Hint: NTT.*

M1 (Codeforces 438C) Count polynomials of degree n with coefficients from a multiset, modulo a prime. *Hint: polynomial exponential via Newton + NTT.*

M2 Compute $f(x)^k \bmod g(x) \bmod p$ where $\deg f, \deg g \leq 10^5$, $k \leq 10^{18}$. *Hint: NTT-based polynomial exponentiation + reduction.*

H1 Multiply two polynomials with coefficients up to 10^{18}, modulo $10^9 + 7$. *Hint: MTT (triple-modulus NTT + Garner).*

H2 (Project Euler 235) Solve a polynomial equation numerically using NTT as an exact-multiplication engine. *Hint: NTT under* 998244353 *for the linear-algebra step.*

H3 (Codeforces 1107D) Compute the inverse of a polynomial modulo x^n, with deg $\le 10^5$. *Hint: Newton iteration with NTT.*

24.10 Linear Recurrences (Chapter 16)

E1 Compute the 10^{18}-th Fibonacci number modulo $10^9 + 7$. *Hint: matrix exponentiation or fast doubling.*

M1 (Codeforces 1182D) Given a graph and a starting position, compute the expected number of moves to reach a sink in 10^{18} moves. *Hint: linear recurrence, Berlekamp–Massey.*

M2 Given the first 100 terms of a sequence, find the 10^{18}-th term mod p. *Hint: Berlekamp–Massey + Kitamasa.*

H1 (Codeforces 506E) Compute the n-th term of a k-th order linear recurrence, $k \le 10^5$, $n \le 10^{18}$. *Hint: NTT-based Kitamasa, $\mathcal{O}(k \log k \log n)$.*

H2 (Codeforces 506D) Multiple queries on the k-th term of the same recurrence. *Hint: precompute Kitamasa polynomials once, query in $\mathcal{O}(k)$.*

24.11 Fibonacci and Pisano Period (Chapter 17)

E1 (Project Euler 2) Sum of even Fibonacci numbers up to 4×10^6. *Hint: F_{3k} is even.*

M1 Find the smallest n with $F_n \equiv 0 \pmod m$, for given m. *Hint: Pisano period divisor.*

M2 (Project Euler 25) First Fibonacci number with 1000 digits. *Hint: iterate or use $\log_{10} \varphi^n / \sqrt{5}$.*

H1 (Project Euler 137) Fibonacci golden nuggets: $\sum \lfloor F_{2n+1}/2 \rfloor$. *Hint: Zeckendorf and Fibonacci identities.*

H2 Compute $\gcd(F_{10^{18}}, F_{10^{18}+1}) \bmod p$. *Hint:* $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.

24.12 Combinatorial Number Theory (Chapter 18)

E1 Compute $\binom{n}{k} \bmod p$ for $n, k \le 10^6$. *Hint: precomputed factorials.*

M1 Compute $\binom{10^{18}}{10^{17}} \bmod 10^6 + 3$. *Hint: Lucas's theorem.*

M2 (Project Euler 15) Lattice paths in a 20×20 grid. *Hint:* $\binom{40}{20}$.

M3 Count the number of partitions of $n = 100$ into k parts. *Hint: Stirling numbers of second kind.*

H1 (Project Euler 53) How many $\binom{n}{k}$ with $1 \le n \le 100$ exceed 10^6? *Hint:* $\binom{n}{k}$ *can be computed exactly via Pascal's triangle.*

H2 Compute $\binom{n}{k} \bmod p^e$ for $n \le 10^{18}$, $p \le 10^6$, $e \le 5$. *Hint: Granville–Andrew generalisation of Lucas with Kummer.*

H3 (Project Euler 78) Smallest n with $p(n) \equiv 0 \pmod{10^6}$ where p is the partition function. *Hint: pentagonal number theorem, generating function.*

24.13 Number Theory Tricks (Chapter 19)

E1 Josephus problem with $n = 41$, $k = 2$. *Hint: closed form* $2(n - 2^{\lfloor \log_2 n \rfloor}) + 1$.

M1 (Codeforces 590C) Generate all 2^{10} Gray codes of length 10. *Hint:* $g(n) = n \oplus (n \gg 1)$.

M2 (Codeforces 1207D) Count subsets with property P, $n \le 20$. *Hint: SOS DP.*

M3 Solve $x^2 \equiv a \pmod{p^2}$ for $p \le 10^6$ prime. *Hint: Hensel lifting on Tonelli–Shanks.*

H1 (Codeforces 1342E) Given n, k, count permutations σ of $\{1, \dots, n\}$ with exactly k "splittable" inversions. *Hint: combinatorics with Hensel + Dirichlet.*

24.14 Generating Functions (Chapter 20)

E1 Compute the OGF of $(1, 1, 1, \dots)$. *Hint:* $1/(1 - x)$.

M1 Count the number of subsets of $\{1, \dots, n\}$ with sum $\le k$. *Hint:* $\prod (1 + x^i)/(1 - x)$, *NTT.*

M2 (Codeforces 1096G) Find the number of n-digit numbers whose digits are from a given multiset. *Hint: polynomial multiplication via NTT.*

M3 Compute $T_n = n^{n-1} \bmod p$ for $n \le 10^5$. *Hint: Cayley's formula, polynomial exponential via Newton.*

H1 (Codeforces 1177D) Given n, find a permutation of $1, \ldots, n$ such that a generating-function-derived condition holds. *Hint: EGF manipulation.*

H2 Compute the n-th term of a sequence defined by $a_n = \sum_{i+j=n} a_i a_j$ (Catalan-like) for $n \le 10^5$. *Hint: NTT-based polynomial square root.*

24.15 Continued Fractions (Chapter 21)

E1 Compute the continued fraction of $355/113$. *Hint: Euclidean algorithm.*

M1 Find the minimal solution to $x^2 - 61y^2 = 1$. *Hint: continued fraction of $\sqrt{61}$.*

M2 Find the best rational approximation to π with denominator ≤ 100. *Hint: convergents of π.*

H1 (Project Euler 64) Count $D \le 10^4$ with $\sqrt{D}$ having odd continued-fraction period. *Hint: continued fraction period of $\sqrt{D}$.*

H2 (Project Euler 65) Sum of digits of the 100th convergent of e. *Hint: continued fraction of $e = [2; 1, 2, 1, 1, 4, 1, 1, 6, \ldots]$.*

H3 (Project Euler 66) Find $D \le 10^3$ (nonsquare) with largest minimal x solving $x^2 - Dy^2 = 1$. *Hint: Pell, continued fractions, compare minimal solutions.*

24.16 Advanced Topics: Lucas Sequences, Beatty, and Prime Counting (Chapter 22)

E1 Verify the Lucas–Lehmer test for $M_7 = 127$. *Hint: $V_{p-1}(127, 1) \bmod 127$.*

M1 Compute $\pi(10^{10})$ exactly. *Hint: Lucy–Hedgehog sieve.*

M2 Compute $\sum_{p \le 10^{10}} p \bmod 10^9 + 7$. *Hint: Min_25 sieve.*

M3 Partition $\mathbb{N}$ into Beatty sequences with $\alpha = \sqrt{2}$. *Hint: Beatty with $\beta = 2 + \sqrt{2}$.*

H1 (Project Euler 10) Sum of primes below 2×10^6 using the Min_25 sieve. *Hint: Min_25 with $\sum p$ update.*

H2 Compute $\sum_{i=1}^{10^{10}} \varphi(i) \bmod 10^9 + 7$. *Hint: Min_25 sieve with φ via Dirichlet.*

H3 Verify Beatty's theorem for $\alpha = \varphi, \beta = \varphi^2$ by listing the first 20 terms of each sequence. *Hint:* $\varphi = (1+\sqrt{5})/2$, $1/\varphi + 1/\varphi^2 = 1$.

24.17 General Pattern-Recognition Exercises

These problems are stated without indicating which chapter applies. The exercise is to identify the technique first, then solve.

M1 Given n and queries of the form "sum φ in $[l, r]$," process q queries. *Hint: precompute φ via linear sieve.*

M2 Solve $a^x \equiv b \pmod{m}$ for $x \geq 0$, given $\gcd(a, m) = 1$. *Hint: BSGS.*

M3 Count pairs (a, b) in $[1, n]^2$ with $\gcd(a, b)$ squarefree. *Hint: Möbius inversion.*

M4 Solve $x^2 - Dy^2 = -1$ for $D = 2$. *Hint: continued fraction of $\sqrt{2}$.*

M5 Given n integers, count pairs with $\gcd = 1$. *Hint: Möbius with ω and inclusion-exclusion.*

H1 Find the smallest prime p such that $p \equiv 1 \pmod{2^k}$ for $k = 23$. *Hint: NTT-friendly prime* 998244353 *has* $k = 23$.

H2 Given n numbers $a_1, \ldots, a_n \leq 10^{12}$, count the number of pairs (i, j) with $\operatorname{lcm}(a_i, a_j) > 10^{18}$. *Hint: factorise each a_i, count pairs by* lcm *structure.*

H3 Compute the 100-th term of the sequence $a_0 = 1, a_1 = 2, a_{n+1} = 2a_n + a_{n-1}^2 \bmod p$ for $p = 998244353$. *Hint: nonlinear recurrence; try Berlekamp–Massey if it appears linear over a finite field, else direct simulation.*

H4 Given a directed graph with n nodes and edge weights, compute the sum over all paths of length k of the product of edge weights, modulo $10^9 + 7$, for $k \leq 10^{18}$. *Hint: matrix exponentiation.*

H5 For each prime $p \leq 10^5$, compute the smallest g with g a primitive root mod p. *Hint: factor $p - 1$, test $g = 2, 3, 5, \ldots$.*

24.18 Closing Note on Practice

These exercises are intentionally a mix of elementary and challenging problems. The reader who has worked through 50 of them will have developed the pattern-recognition reflexes that

this book aims to teach; 100 will give the contestant a solid foundation for ICPC regionals and Codeforces Div 1 contests. The last few hundred are designed to be at the level of ICPC World Finals and Codeforces round #700+.

The single most important practice habit: when you solve a problem, take a moment to identify *which* signal in the problem statement triggered your identification of the technique. Was it the phrase "modulo 10^9+7"? The constraint $n \leq 10^{18}$? The mention of gcd in the answer? Cataloguing these signals is how you build the pattern-recognition reflexes of Chapter 23 into something automatic.

Chapter 25

Common Pitfalls, Edge Cases, and Implementation Tips

This chapter collects the implementation mistakes that the author has personally made (and seen others make) in competitive programming contests. Each pitfall is paired with the correct idiom. The reader who internalises this chapter will save themselves many contest-debugging sessions.

25.1 Integer Overflow

Pitfall 1: Computing $a \cdot b$ where $a, b \le 10^9$ with `int` variables. The product overflows 32-bit. **Fix:** use `long long` for both operands, or cast to `long long` before multiplication: `(long long) a * b`.

Pitfall 2: Computing $a \cdot b \bmod m$ where $a, b, m \le 10^{18}$ with `long long`. The product overflows 64-bit. **Fix:** use `__int128`: `(__int128) a * b % m`. For portable code (without `__int128`), use a "mulmod" routine based on the binary expansion of b or on `long double` binary search.

Pitfall 3: Computing n^2 for n near 10^9 with `long long`. This is fine ($10^{18} < 9.2 \times 10^{18} =$ max `int64_t`), but $n = 3 \cdot 10^9$ overflows. **Fix:** always check the maximum value before deciding on the type.

Pitfall 4: `abs(LLONG_MIN)` is undefined behaviour (signed overflow). **Fix:** use `llabs` for `long long`, or write `a < 0 ? -(unsigned long long)a : (unsigned long long)a` to be safe.

25.2 Modular Arithmetic

Pitfall 5: Negative `a % m`. In C++, `(-7) % 3 == -1`, not 2. **Fix:** use `(a % m + m) % m`.

Pitfall 6: Computing `binpow(a, 0, m)` returns 0 when $m = 1$ (since $1\%1 = 0$). **Fix:** initialise `r = 1 % m` (not `r = 1`).

Pitfall 7: Computing modular inverse $a^{-1} \bmod p$ where $p \mid a$. The inverse does not exist. **Fix:** check $\gcd(a, p) = 1$ first, or handle $p \mid a$ as a special case (the inverse is conventionally taken as 0 in such problems).

Pitfall 8: Using Fermat's little theorem $a^{-1} = a^{p-2}$ for composite p. Fermat only works for primes. **Fix:** use the extended Euclidean algorithm if p may be composite.

Pitfall 9: Division in modular arithmetic: `a / b % m` is `a * inv(b, m) % m` only when $\gcd(b, m) = 1$. For composite m with $\gcd(b, m) \neq 1$, division is more subtle and may not exist or may have multiple solutions.

Pitfall 10: Negative modular results from subtraction. `(a - b) % m` can be negative if $a < b$. **Fix:** `(a - b + m) % m`.

25.3 Sieve Memory and Cache

Pitfall 11: Sieve of Eratosthenes with `vector<int>` for $n = 10^8$. Memory: 4×10^8 bytes = 400 MB, often exceeds the contest memory limit (typically 256 MB). **Fix:** use `vector<bool>` (1 bit per element, 12.5 MB for $n = 10^8$) or `vector<char>` (1 byte per element, 100 MB).

Pitfall 12: Sieve of Eratosthenes with `vector<bool>` is slower than `vector<char>` because `vector<bool>` is bit-packed and access requires bit operations. **Fix:** if speed matters and memory permits, use `vector<char>`; if memory is tight, use `vector<bool>`.

Pitfall 13: Linear sieve for $n = 10^8$ uses $\sim$ 800 MB for `spf` array. **Fix:** for $n > 10^7$ use segmented sieve or other techniques; linear sieve is memory-heavy.

Pitfall 14: Cache misses in inner loops. A naive sieve touches the array in strides of p, which exceeds the L1/L2 cache line for large p. **Fix:** segmented sieve to keep the working set in cache.

25.4 Factorisation

Pitfall 15: Trial division starting from 2 and incrementing by 1 is twice as slow as needed. **Fix:** handle $p = 2$ separately, then iterate odd primes $3, 5, 7, \ldots$ (halving the work).

Pitfall 16: Pollard's rho with $f(x) = x^2 + c \bmod n$ can cycle forever if the cycle exists modulo n but not modulo any prime factor of n. **Fix:** limit the number of iterations (e.g., 10000) and restart with a different c if no factor found.

Pitfall 17: Pollard's rho with $c = 0$ or $c = n - 2$ produces degenerate sequences. **Fix:** use $c \in \{1, 2, \ldots, n - 1\} \setminus \{n - 2\}$; avoid $c = 0$.

Pitfall 18: Miller–Rabin bases that are multiples of n are skipped, but bases ≤ 1 can cause issues. **Fix:** guard with `a % n == 0`.

Pitfall 19: Miller–Rabin on $n = 2$ or $n = 3$ needs special handling (the base list typically does not include these). **Fix:** check small primes first.

25.5 CRT and Garner

Pitfall 20: CRT with non-coprime moduli requires the generalised version. The classical CRT formula $x = \sum a_i M_i y_i$ assumes $\gcd(m_i, m_j) = 1$ and gives wrong answers otherwise. **Fix:** use the pairwise generalised CRT, checking the consistency condition $a_i \equiv a_j \pmod{\gcd(m_i, m_j)}$ at each step.

Pitfall 21: Garner's algorithm with $a_i \geq m_i$ (out of range). **Fix:** normalise each a_i to $[0, m_i)$ before running Garner.

Pitfall 22: Overflow in the final summation step of Garner's algorithm. The product $v_i \cdot (m_1 \cdots m_{i-1})$ can overflow 64-bit even when each v_i and m_i fits in 64-bit. **Fix:** take the modulus after each step, or use `__int128` for intermediate products.

25.6 NTT

Pitfall 23: NTT with n not a power of 2. The Cooley–Tukey algorithm requires n to be a power of 2. **Fix:** pad the input to the next power of 2, and trim the output.

Pitfall 24: NTT under a non-NTT-friendly prime. The prime p must admit an n-th primitive root of unity, i.e., $n \mid p - 1$. **Fix:** use 998244353 (which has $n \mid p - 1$ for all $n = 2^k$ with $k \leq 23$), or use MTT with three NTT-friendly primes plus Garner.

Pitfall 25: Forgetting the inverse-normalisation step. After the inverse NTT, the result must be divided by n (modular inverse of n mod p). **Fix:** add `for (ll& x : a) x = (lll) x * inv(n) % p;` after the inverse transform.

Pitfall 26: In-place NTT without bit-reversal permutation. The iterative Cooley–Tukey algorithm assumes the input is bit-reversed. **Fix:** always include the bit-reversal permutation step before the butterfly loop.

Pitfall 27: Polynomial operations via Newton iteration with degenerate initial conditions (e.g., $f(0) \neq 1$ for $\log f$ or $\exp f$). **Fix:** check $f(0) = 1$ for log, $f(0) = 0$ for exp, and adjust accordingly.

25.7 Berlekamp–Massey and Kitamasa

Pitfall 28: Berlekamp–Massey on a sequence with less than $2k$ terms for a k-th order recurrence. The algorithm may produce a wrong (too-short) recurrence. **Fix:** ensure the input has at least $2k$ terms; for unknown k, use $2 \cdot \max(k_{\text{guess}}) + 10$ terms as a safety margin.

Pitfall 29: Berlekamp–Massey requires the modulus p to be prime. For composite modulus, the algorithm can fail (no division by the discrepancy is possible). **Fix:** use a prime modulus, or work over $\mathbb{Q}$ (which is much harder and not generally feasible).

Pitfall 30: Kitamasa with characteristic polynomial $R(x) = x^k - c_1 x^{k-1} - \cdots - c_k$. The polynomial reduction step must reduce x^i for $i \geq k$ using $R(x)$, i.e., $x^k \equiv c_1 x^{k-1} + c_2 x^{k-2} + \cdots + c_k$. The signs in the reduction step are easy to flip. **Fix:** carefully derive the reduction formula and test on a small example (e.g., Fibonacci) before applying to large cases.

25.8 Discrete Logarithm

Pitfall 31: BSGS with g not a generator of $\mathbb{F}_p^\times$. The discrete log may not exist if h is not in the subgroup generated by g. **Fix:** check that $h^{(p-1)/d} \equiv 1 \pmod{p}$ where $d = \text{ord}_p(g)$, or just check that no solution exists if BSGS returns -1.

Pitfall 32: BSGS with p not prime. The algorithm assumes $\mathbb{F}_p^\times$ is a cyclic group; for composite p, the structure is more complex (multiple prime factors). **Fix:** factor p and solve the DLP in each $(\mathbb{Z}/p_i^{e_i}\mathbb{Z})^\times$ via CRT.

Pitfall 33: BSGS hash table collisions can cause incorrect (non-minimal) results. **Fix:** use a proper hash table (e.g., `unordered_map`) or sort-and-binary-search for deterministic results.

25.9 Recursion and Stack Overflow

Pitfall 34: Recursive extended Euclidean algorithm on very large inputs can hit the recursion limit (typically 10^4 to 10^6 levels). The Euclidean algorithm has depth $O(\log n)$, so this is rarely an issue, but Berlekamp–Massey or recursive NTT-based Kitamasa can hit limits. **Fix:** convert to iterative, or increase the stack limit at the start of `main`: `int main() { ... }` with `-Wl,-stack_size,0x10000000` linker flag, or use `ulimit -s unlimited`.

Pitfall 35: Recursive Miller–Rabin is fine (depth $O(\log n)$), but recursive Pollard's rho is not (it can loop arbitrarily). **Fix:** use the iterative form of Pollard's rho.

25.10 Floating-Point

Pitfall 36: Computing $\lfloor\sqrt{n}\rfloor$ with `(long long) sqrt((double) n)` can give wrong answers for large n due to floating-point rounding. For $n = (2^{25})^2 = 2^{50}$, `sqrt(n)` may return $2^{25} + 1$. **Fix:** compute via integer Newton's method, or correct with `while (r * r > n) -r; while ((r+1) * (r+1) <= n) ++r;`.

Pitfall 37: Continued fraction expansion with floating-point `double` loses precision for α near an integer. **Fix:** use rational arithmetic when possible, or extended precision (`long double` or arbitrary precision).

Pitfall 38: Comparing `double` values for equality. `(a == (double) ai)` in continued fractions may not work as expected due to floating-point error. **Fix:** compare with a tolerance `abs(a - ai) < 1e-9`, or use exact rational arithmetic.

25.11 Time Limits

Pitfall 39: Solution runs in $\mathcal{O}(n \log n)$ but exceeds the time limit due to large constants. **Fix:** profile; common culprits are `__int128` multiplications (10x slower than 64-bit), unnecessary recursion (function-call overhead), and cache misses (large array striding).

Pitfall 40: The $\mathcal{O}(\log n)$ algorithm for Fibonacci via fast doubling runs in microseconds, but the same algorithm with $\mathcal{O}(k^3 \log n)$ for a k-th order recurrence becomes seconds for $k = 200$. **Fix:** for $k \geq 100$, use Kitamasa ($\mathcal{O}(k^2 \log n)$) or NTT-based Kitamasa ($\mathcal{O}(k \log k \log n)$).

25.12 Common Heuristics

Heuristic 1: If the problem says "$n \leq 10^{18}$" and the answer is a single integer, the algorithm is almost certainly $\mathcal{O}(\log n)$ or $\mathcal{O}(\sqrt{n})$. Reach for fast exponentiation, fast doubling, or BSGS.

Heuristic 2: If the problem says "modulo p" for prime p and asks for a count, use generating functions or inclusion-exclusion (Möbius). The prime modulus is the giveaway that you are in $\mathbb{F}_p$ and need modular inverses.

Heuristic 3: If the problem mentions gcd in the answer, the universal first step is $[\gcd = 1] = \sum_{d|\gcd} \mu(d)$ or $\gcd = \sum_{d|\gcd} \varphi(d)$. Apply this identity and swap summation order.

Heuristic 4: If the problem asks for "the 10^{18}-th term of a sequence defined by a complicated DP," reach for Berlekamp–Massey: simulate $2k$ terms, find the recurrence, then apply Kitamasa.

Heuristic 5: If the problem involves polynomials with degree $\geq 10^5$, reach for NTT. If the modulus is not 998244353, use MTT (three NTTs under different primes, then Garner).

25.13 Final Tip: Build a Personal Template Library

The single best practice habit for competitive programmers is to maintain a personal library of pre-tested C++ templates. After each contest, copy the cleaned-up version of every algorithm you used into your library. After a year of contests, the library will contain all the templates in this book's appendix, plus many more. Then, in a new contest, the implementation work reduces to: (i) identify the technique (using the pattern-recognition framework of Chapter 23), (ii) copy the template, (iii) write the problem-specific glue code. This is how contest champions achieve 30-second implementations of complex algorithms.

Chapter 26

Selected Solutions

This chapter gives complete worked solutions to a selection of the exercises from Chapter 24, chosen to illustrate the most common patterns. The solutions are presented in the same order as the corresponding exercises.

26.1 Solution to Exercise 24.M1 (Segmented Sieve)

The exercise: "Given T test cases, each with $m \le n \le 10^9$ and $n - m \le 10^5$, print every prime in $[m, n]$."

```
vector<long long> seg_primes(long long m, long long n) {
    long long lim = (long long) sqrt((double) n) + 1;
    vector<bool> base(lim + 1, true); base[0] = base[1] = false;
    for (long long i = 2; i * i <= lim; ++i) if (base[i])
        for (long long j = i * i; j <= lim; j += i) base[j] = false;
    vector<bool> seg(n - m + 1, true);
    if (m == 1) seg[0] = false;
    for (long long p = 2; p <= lim; ++p) if (base[p]) {
        long long start = max(p * p, (m + p - 1) / p * p);
        for (long long j = start; j <= n; j += p) seg[j - m] = false;
    }
    vector<long long> ans;
    for (long long i = m; i <= n; ++i) if (seg[i - m]) ans.push_back(i);
    return ans;
}
```

The complexity is $\mathcal{O}(\sqrt{n}\log\log\sqrt{n} + (n-m)\log\log\sqrt{n})$ per query, dominated by $\mathcal{O}(\sqrt{n} + (n-m))$ in practice.

26.2 Solution to Exercise 24.M2 (Codeforces 231B)

The exercise: "Given n, count the number of pairs (a, b) with $1 \le a, b \le n$ and $a \cdot b$ a perfect square."

Solution sketch. $a \cdot b$ is a perfect square iff a and b have the same square-free kernel (the product of distinct prime factors of a). The square-free kernel of a is $a/(\text{largest square dividing } a)$. So the answer is $\sum_k (\text{count of } a \in [1, n] \text{ with kernel } k)^2$.

To compute this: for each $a \in [1, n]$, factorise it (linear sieve gives spf), remove all squared prime factors, and the remaining square-free integer is the kernel. Then count pairs with the same kernel via a hash map. The complexity is $\mathcal{O}(n \log \log n)$ (sieve) + $\mathcal{O}(n \log n)$ (factorisation) + $\mathcal{O}(n)$ (counting), giving $\mathcal{O}(n \log n)$.

26.3 Solution to Exercise 24.M3 (Codeforces 803F)

The exercise: "Count pairs (a, b) with $1 \le a, b \le n$ and $\gcd(a, b) = 1$, modulo $10^9 + 7$."

Solution. Use Möbius inversion: $f(1) = \sum_{d=1}^{n} \mu(d) \lfloor n/d \rfloor^2$. Precompute μ and its prefix sum M via the linear sieve. Then group by the value of $\lfloor n/d \rfloor$: for each interval $[l, r]$ where $\lfloor n/d \rfloor$ is constant, add $(M[r] - M[l-1]) \cdot q^2$ where $q = \lfloor n/l \rfloor$.

```
long long solve(long long n) {
    const long long P = 1e9 + 7;
    long long ans = 0;
    for (long long l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        long long q = n / l;
        ans = (ans + (M[r] - M[l - 1]) * (q % P) % P * (q % P)) % P;
    }
    return (ans % P + P) % P;
}
```

The complexity is $\mathcal{O}(\sqrt{n})$ per query after the $\mathcal{O}(n)$ sieve.

26.4 Solution to Exercise 24.H1 (Codeforces 906D)

The exercise: "Compute $a_1^{a_2^{a_3^{\cdots}}} \bmod m$ for $m \le 10^9$."

Solution. Apply Euler's theorem with the safe-reduction formula (Theorem 7.7). Define

$f(i,m) = a_i^{a_{i+1}^{\cdots^{a_{i+2}}}} \bmod m$. If a_i and m are coprime, $f(i,m) = a_i^{f(i+1,\varphi(m))} \bmod m$. Otherwise, use the safe reduction $f(i,m) = a_i^{f(i+1,\varphi(m))+\varphi(m)} \bmod m$ (provided the exponent is at least $\log_2 m$).

```
long long phi(long long m);  // from factorisation
map<long long, long long> memo;

long long solve(int i, vector<long long>& a, long long m) {
    if (m == 1) return 0;
    if (i == (int) a.size()) return 1;
    long long e = solve(i + 1, a, phi(m));
    if (gcd(a[i], m) != 1) e += phi(m);
    return binpow(a[i], e, m);
}
```

The recursion has depth $O(\log m)$ (because φ reduces m by at least a factor of 2 each step), and each level does $\mathcal{O}(\log m)$ work via `binpow`.

26.5 Solution to Exercise 24.H2 (Codeforces 506E)

The exercise: "Compute the n-th term of a k-th order linear recurrence, $k \le 10^5$, $n \le 10^{18}$."

Solution. Kitamasa's algorithm reduces the problem to $O(\log n)$ polynomial multiplications modulo the characteristic polynomial $R(x) = x^k - c_1 x^{k-1} - \cdots - c_k$. The polynomial multiplications are computed via NTT in $\mathcal{O}(k \log k)$ each. Total: $\mathcal{O}(k \log k \log n)$.

The key step is the reduction $x^i \bmod R(x)$ for $i \ge k$, which uses the identity $x^k \equiv c_1 x^{k-1} + c_2 x^{k-2} + \cdots + c_k$. Each multiplication step computes the product $P(x) \cdot Q(x)$, reduces modulo $R(x)$ (by iteratively reducing the leading term), and stores the result.

26.6 Solution to Exercise 24.H3 (Project Euler 78)

The exercise: "Smallest n with $p(n) \equiv 0 \pmod{10^6}$ where p is the partition function."

Solution sketch. Use the pentagonal number theorem: $p(n) = \sum_{k \neq 0} (-1)^{k-1} p(n - k(3k-1)/2)$ summed over $k \in \mathbb{Z} \setminus \{0\}$. Compute $p(1), p(2), \ldots$ iteratively in $\mathcal{O}(n\sqrt{n})$ time (since the number of pentagonal numbers $\le n$ is $O(\sqrt{n})$). Stop when $p(n) \equiv 0 \pmod{10^6}$. The answer for Project Euler 78 is $n = 55374$.

```
long long solve(long long mod) {
    vector<long long> p = {1};
```

```
    for (long long n = 1; ; ++n) {
        long long s = 0;
        for (long long k = 1; ; ++k) {
            long long g1 = k * (3 * k - 1) / 2;
            long long g2 = k * (3 * k + 1) / 2;
            if (g1 > n) break;
            long long sign = (k % 2 == 1) ? 1 : -1;
            s = (s + sign * p[n - g1] + mod) % mod;
            if (g2 <= n) s = (s + sign * p[n - g2] + mod) % mod;
        }
        p.push_back(s);
        if (s == 0) return n;
    }
}
```

26.7 Solution to Exercise 24.H1 (Continued Fractions: Project Euler 64)

The exercise: "Count $D \le 10^4$ (nonsquare) with $\sqrt{D}$ having odd continued-fraction period."

Solution sketch. For each nonsquare $D \le 10^4$, compute the continued fraction of $\sqrt{D}$ via the iterative algorithm $m_{i+1} = d_i a_i - m_i$, $d_{i+1} = (D - m_{i+1}^2)/d_i$, $a_{i+1} = \lfloor (a_0 + m_{i+1})/d_{i+1} \rfloor$. The period ends when $a_i = 2a_0$ and the triple (m, d, a) repeats. Count D with odd period.

```
int period(long long D) {
    long long a0 = (long long) sqrt((double) D);
    if (a0 * a0 == D) return 0;
    long long m = 0, d = 1, a = a0, period = 0;
    set<tuple<long long, long long, long long>> seen;
    while (true) {
        auto key = make_tuple(m, d, a);
        if (seen.count(key)) return period;
        seen.insert(key);
        m = d * a - m;
        d = (D - m * m) / d;
        a = (a0 + m) / d;
        ++period;
    }
}
```

The Project Euler 64 answer for $D \le 10^4$ is 1322 (number of D with odd period).

26.8 Solution to Exercise 24.H2 (Pell, Project Euler 66)

The exercise: "Find $D \le 10^3$ (nonsquare) with largest minimal x solving $x^2 - Dy^2 = 1$."

Solution sketch. For each nonsquare D, compute the continued fraction of $\sqrt{D}$; the convergent at the end of the first period gives the minimal solution (x_1, y_1). Track the maximum minimal x across all D. The Project Euler 66 answer is $D = 661$ with minimal $x = 16421658242965910275055840472270271024 95$.

26.9 Solution to Exercise 24.H1 (Lucy–Hedgehog)

The exercise: "Compute $\pi(10^{10})$ exactly."

Solution sketch. Use the Lucy–Hedgehog algorithm of Chapter 22. The algorithm maintains $S(v, p)$ = count of integers in $\{2, \dots, v\}$ not sieved by primes $\le p$, for $v \in \{n/k\} \cup \{1, \dots, \sqrt{n}\}$. After processing all primes $\le \sqrt{n}$, $S(n, \sqrt{n}) = \pi(n)$. The complexity is $\mathcal{O}(n^{3/4}/\log n)$ time and $\mathcal{O}(\sqrt{n})$ memory.

For $n = 10^{10}$, $\sqrt{n} = 10^5$, so the algorithm needs about $2 \cdot 10^5$ memory cells and runs in a few seconds. The exact answer is $\pi(10^{10}) = 455052511$.

26.10 Solution to Exercise 24.H4 (Matrix Exponentiation on Graphs)

The exercise: "Given a directed graph with n nodes and edge weights, compute the sum over all paths of length k of the product of edge weights, modulo $10^9 + 7$, for $k \le 10^{18}$."

Solution sketch. Construct the adjacency matrix A where A_{ij} = weight of edge $i \to j$ (or 0 if no such edge). Then $\sum_{\text{paths}}(\text{product of weights}) = \sum_{i,j}(A^k)_{ij}$. Compute A^k via binary exponentiation in $\mathcal{O}(n^3 \log k)$.

```
long long solve(vector<vector<long long>> A, long long k, long long p) {
    int n = A.size();
    auto R = identity(n);
    while (k) {
```

```
        if (k & 1) R = matmul(R, A, p);
        A = matmul(A, A, p);
        k >>= 1;
    }
    long long ans = 0;
    for (int i = 0; i < n; ++i) for (int j = 0; j < n; ++j) ans = (ans + R[i
        ][j]) % p;
    return ans;
}
```

26.11 Closing Note on Solutions

These ten solutions illustrate the patterns that recur across most of the exercises. The single most important lesson is the translation from "problem statement" to "algorithm identification" to "implementation": the pattern-recognition step (Chapter 23) gives you the algorithm, the appendix gives you the templates, and the solutions above show the glue code that ties them together. Practice each of the templates until they are automatic; the algorithms themselves are short, and the difficulty is always in recognising when to apply them.

Chapter 27

Historical Notes and Further Reading

Number theory is one of the oldest branches of mathematics, with a continuous tradition stretching back to the ancient Greeks. This chapter collects historical notes on the theorems and algorithms covered in this book, with references to the original sources and to modern surveys that the reader can consult for more depth.

27.1 Antiquity: Euclid, Diophantus, and the Foundations

The Euclidean algorithm (Chapter 4) appears in the *Elements* (Book VII, propositions 1–2), written around 300 BCE. Euclid's proof of the infinitude of primes (Theorem 2.7) is in Book IX, proposition 20. The *Fundamental Theorem of Arithmetic* (Theorem 2.6) is implicit in Euclid but was first stated and proved rigorously by Gauss in his *Disquisitiones Arithmeticae* (1801).

Diophantus of Alexandria (c. 250 CE) wrote the *Arithmetica*, a collection of problems asking for integer or rational solutions to polynomial equations—the original "Diophantine equations" of Chapter 4. The 1621 edition by Bachet de Méziriac inspired Fermat's work on number theory, including the famous marginal note that became Fermat's Last Theorem (proved by Andrew Wiles in 1994).

27.2 The Renaissance and Early Modern Period

Pierre de Fermat (1607–1665) made foundational contributions: Fermat's little theorem (Theorem 7.5), the method of infinite descent, and the conjecture that became Fermat's Last Theorem.

The little theorem appears in a letter to Frénicle de Bessy in 1640 and was first proved by Euler in 1736.

Leonhard Euler (1707–1783) was the most prolific mathematician of the 18th century and the founder of analytic number theory. He introduced the totient function φ (Chapter 7), proved Fermat's little theorem in its general form (Euler's theorem, Theorem 7.4), and conjectured the quadratic reciprocity law (Theorem 12.5). His *Introductio in Analysin Infinitorum* (1748) develops the theory of generating functions (Chapter 20) and contains the product formula for the Riemann zeta function $\zeta(s) = \sum n^{-s} = \prod_p (1 - p^{-s})^{-1}$ that links prime numbers to analysis.

Joseph-Louis Lagrange (1736–1813) proved Wilson's theorem (Theorem 5.5) and laid the foundations of group theory, including the theorem that bears his name (used in our proof of Euler's theorem). His *Réflexions sur la résolution algébrique des équations* (1771) influenced the development of Galois theory.

27.3 The 19th Century: Rigor and Abstraction

Carl Friedrich Gauss (1777–1855) is the central figure of 19th-century number theory. His *Disquisitiones Arithmeticae* (1801), written when he was 21, contains the first rigorous proof of the quadratic reciprocity law (Theorem 12.5), the theory of congruences (the notation $a \equiv b \pmod{n}$ is due to Gauss), the theory of binary quadratic forms, and the construction of the regular 17-gon by straightedge and compass. Gauss is also the first to conjecture the Prime Number Theorem, proved independently by Hadamard and de la Vallée-Poussin in 1896.

Adrien-Marie Legendre (1752–1833) introduced the Legendre symbol (Chapter 12) in his *Essai sur la théorie des nombres* (1798) and conjectured the prime number theorem and the law of quadratic reciprocity (which Gauss proved).

Augustin-Louis Cauchy (1789–1857) and Niels Henrik Abel (1802–1829) developed group theory further, including the notion of a normal subgroup and the theorem that bears Cauchy's name (every finite group of order divisible by p has an element of order p).

Leopold Kronecker (1823–1891) developed the theory of algebraic numbers and introduced the Kronecker symbol, a generalisation of the Legendre symbol. He is famous for the slogan "God created the integers; all else is the work of man."

27.4 Late 19th and Early 20th Century

Bernhard Riemann (1826–1866) introduced the Riemann zeta function and its analytic continuation in his 1859 paper *On the Number of Primes Less Than a Given Magnitude*, which contains the famous conjecture that bears his name. The Riemann Hypothesis is one of the seven Millennium Prize Problems and remains open as of 2025.

David Hilbert (1862–1943) unified algebraic number theory in his *Zahlbericht* (1897) and posed 23 problems at the International Congress of Mathematicians in 1900 (the Hilbert Problems) that shaped 20th-century mathematics. Several of the problems concern number theory.

Srinivasa Ramanujan (1887–1920) made extraordinary contributions to number theory, including the partition function (Chapter 18) and the Ramanujan–Petersson conjecture. His collaboration with G. H. Hardy at Cambridge produced dozens of papers and the famous Hardy–Ramanujan formula for the partition function.

Emil Artin (1898–1962) developed the theory of Artin reciprocity, a vast generalisation of quadratic reciprocity. The Artin conjecture on L-functions is one of the major open problems in analytic number theory.

27.5 The Computer Era

The development of electronic computers in the 1940s and 1950s revolutionised computational number theory. Alan Turing's work on computability and the development of the Turing machine at Bletchley Park influenced the early theory of algorithms. The first primality tests and factorisation algorithms were implemented in the 1950s.

The Miller–Rabin primality test (Chapter 8) was developed by Gary Miller in 1975 (with the deterministic analysis conditional on the Generalised Riemann Hypothesis) and by Michael Rabin in 1980 (with the unconditional randomised version). The deterministic set of bases for $n < 2^{64}$ (Theorem 8.4) was determined by Jim Sinclair in 2011 and independently by others.

The Number-Theoretic Transform (Chapter 15) was developed in the 1970s as a modular alternative to the Fast Fourier Transform (Cooley and Tukey, 1965). The iterative Cooley–Tukey algorithm with bit-reversal permutation is the standard implementation, used in virtually every NTT library.

The Pollard rho and $p-1$ algorithms (Chapter 9) were developed by John Pollard in 1974 and 1978 respectively, for factoring large integers of the form $2^p - 1$ (Mersenne numbers). The

Brent improvement (Chapter 9) is due to Richard Brent in 1980.

The Berlekamp–Massey algorithm (Chapter 16) was developed independently by Elwyn Berlekamp in 1968 (for decoding BCH codes) and by James Massey in 1969 (for shift-register synthesis). Its application to linear recurrences in competitive programming is more recent, popularised by Codeforces round #506 and similar contests.

27.6 Modern Era and Competitive Programming

The competitive programming community has codified many of the algorithms in this book into reusable templates, distributed via sites like CP-Algorithms, Codeforces blogs, and ICPC team handbooks. The standard references for competitive programming number theory are:

- Steven Halim and Felix Halim, *Competitive Programming* (4th edition, 2020). The book includes a chapter on number theory and a comprehensive appendix of templates.
- Antti Laaksonen, *Guide to Competitive Programming* (Springer, 2017). Includes mathematical topics including number theory, with worked examples.
- The CP-Algorithms website (`https://cp-algorithms.com`), originally a translation of Russian CP resources. The site covers most of the algorithms in this book with code.
- The ICPC Competitive Programming Handbook series, published by various regional ICPC organisations.

27.7 Mathematical References

For deeper mathematical study:

- G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers* (6th edition, Oxford, 2008). The classic elementary number theory text, in print since 1938.
- Ivan Niven, Herbert Zuckerman, and Hugh Montgomery, *An Introduction to the Theory of Numbers* (5th edition, Wiley, 1991). A standard undergraduate text covering all the elementary theory.
- Kenneth Ireland and Michael Rosen, *A Classical Introduction to Modern Number Theory* (2nd edition, Springer, 1990). A graduate text covering quadratic reciprocity, primes in arithmetic progressions, and Diophantine equations.

- Donald Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd edition, Addison-Wesley, 1997). The canonical reference for arithmetic algorithms including factorisation, primality testing, and modular arithmetic.
- Henri Cohen, *A Course in Computational Algebraic Number Theory* (Springer, 1993). A modern reference for computational number theory, including advanced factorisation and primality algorithms.

27.8 Online Resources

For competitive programmers, the following online resources are essential:

- `https://cp-algorithms.com` — the most comprehensive online reference for competitive programming algorithms.
- `https://projecteuler.net` — over 800 problems requiring nontrivial number theory and programming.
- `https://codeforces.com` — weekly contests with a tag system; search for problems tagged `number theory`, `math`, `matrices`, `fft`.
- `https://atcoder.jp` — Japanese contest site with strong mathematical problems, especially in the `Math` and `Convolution` tags.
- `https://spoj.com` — Sphere Online Judge, with classic problems like `PRIME1`, `PON`, `ETF`, `DISCRETLOG`, `PELL`.
- `https://oeis.org` — the Online Encyclopedia of Integer Sequences, an essential resource for identifying sequences and finding formulas.
- `https://mathoverflow.net` — for the very hard problems; research-level discussions.

27.9 Closing Note

Number theory is a discipline with a 2000-year history, and the algorithms covered in this book span that history: from Euclid's algorithm (300 BCE) to Berlekamp–Massey (1968) and the Number-Theoretic Transform (1970s). The competitive programmer who masters the techniques in this book stands at the end of a long tradition, applying tools developed by Gauss, Euler, and Ramanujan to problems that can be solved in seconds on a modern computer. The

same ingenuity that drove the early pioneers drives the modern contestant: a love of patterns, a refusal to take “impossible” for an answer, and the patient working-out of consequences from a small set of powerful ideas. May this book serve as a useful guide to that tradition.

Chapter 28

Appendix: Templates and Reference Tables

This appendix consolidates the C++ templates from across the book into a single place, ready to be pasted into a contest solution. We also collect a complexity cheat sheet, a list of NTT-friendly primes, and a list of common competitive-programming moduli.

28.1 Core Number Theory Templates

```
typedef long long ll;
typedef unsigned long long ull;
typedef __int128 lll;

ll ext_gcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, d = ext_gcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return d;
}

ll binpow(ll a, ll e, ll m) {
    a %= m; if (a < 0) a += m;
    ll r = 1 % m;
    while (e > 0) {
        if (e & 1) r = (lll) r * a % m;
        a = (lll) a * a % m;
```

```
        e >>= 1;
    }
    return r;
}

ll inv_mod(ll a, ll m) {
    ll x, y; ll g = ext_gcd(((a % m) + m) % m, m, x, y);
    if (g != 1) return -1;
    return ((x % m) + m) % m;
}
```

Listing 28.1: Core number-theory templates: extended Euclidean, modular exponentiation, modular inverse.

28.2 Primality Testing and Factorisation Templates

```
bool miller_rabin(ll n) {
    if (n < 2) return false;
    for (ll p : {2,3,5,7,11,13,17,19,23,29,31,37}) if (n % p == 0) return n
        == p;
    ll d = n - 1; int s = 0;
    while ((d & 1) == 0) { d >>= 1; ++s; }
    auto check = [&](ll a) {
        ll x = binpow(a, d, n);
        if (x == 1 || x == n - 1) return true;
        for (int i = 0; i < s - 1; ++i) { x = (lll) x * x % n; if (x == n -
            1) return true; }
        return false;
    };
    for (ll a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
        if (a % n == 0) continue;
        if (!check(a)) return false;
    }
    return true;
}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll pollard(ll n) {
    if (n % 2 == 0) return 2;
```

```
    while (true) {
        ll c = rng() % (n - 1) + 1, y = rng() % n, q = 1, g = 1, x = y, ys =
            y;
        for (ll r = 1; g == 1; r *= 2) {
            x = y;
            for (ll i = 0; i < r; ++i) y = (lll) y * y + c % n;
            for (ll k = 0; k < r && g == 1; k += 128) {
                ys = y;
                ll lim = min(r - k, 128LL);
                for (ll i = 0; i < lim; ++i) {
                    y = (lll) y * y + c % n;
                    q = (lll) q * llabs(x - y) % n;
                }
                g = __gcd(q, n);
            }
        }
        if (g == n) do {
            ys = (lll) ys * ys + c % n;
            g = __gcd(llabs(x - ys), n);
        } while (g == 1);
        if (g != n) return g;
    }
}

void factor(ll n, vector<ll> &out) {
    if (n == 1) return;
    if (miller_rabin(n)) { out.push_back(n); return; }
    ll d = pollard(n);
    factor(d, out); factor(n / d, out);
}
```

Listing 28.2: Miller–Rabin and Pollard's rho (Brent's variant) for $n \leq 2^{64}$.

28.3 NTT Template

```
const ll NTT_MOD = 998244353, NTT_G = 3;

void ntt(vector<ll> &a, bool inv) {
    int n = a.size();
```

```
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll w = binpow(NTT_G, (NTT_MOD - 1) / len, NTT_MOD);
        if (inv) w = binpow(w, NTT_MOD - 2, NTT_MOD);
        for (int i = 0; i < n; i += len) {
            ll cw = 1;
            for (int j = 0; j < len / 2; ++j) {
                ll u = a[i + j], v = (lll) a[i + j + len/2] * cw % NTT_MOD;
                a[i + j] = (u + v) % NTT_MOD;
                a[i + j + len/2] = (u - v + NTT_MOD) % NTT_MOD;
                cw = (lll) cw * w % NTT_MOD;
            }
        }
    }
    if (inv) {
        ll n_inv = binpow(n, NTT_MOD - 2, NTT_MOD);
        for (ll &x : a) x = (lll) x * n_inv % NTT_MOD;
    }
}

vector<ll> multiply(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < (int)(a.size() + b.size() - 1)) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (lll) a[i] * b[i] % NTT_MOD;
    ntt(a, true);
    return a;
}
```

Listing 28.3: Iterative NTT and polynomial multiplication under 998244353.

28.4 Berlekamp–Massey and Kitamasa Templates

```cpp
vector<ll> berlekamp_massey(vector<ll> a, ll p) {
    int n = a.size();
    vector<ll> C(n, 0), B(n, 0);
    C[0] = B[0] = 1;
    int L = 0, m = 1; ll b = 1;
    for (int i = 0; i < n; ++i) {
        ll d = a[i] % p;
        for (int j = 1; j <= L; ++j) d = (d + C[j] * a[i - j]) % p;
        if (d == 0) { ++m; continue; }
        auto T = C;
        ll coef = d * binpow(b, p - 2, p) % p;
        for (int j = m; j < n; ++j)
            C[j] = (C[j] - coef * B[j - m] % p + p) % p;
        if (2 * L <= i) { L = i + 1 - L; B = T; b = d; m = 1; }
        else ++m;
    }
    C.resize(L + 1);
    return C;
}
```

Listing 28.4: Berlekamp–Massey: find the minimal linear recurrence of a sequence mod p.

28.5 BSGS Template

```cpp
ll bsgs(ll g, ll h, ll p) {
    g %= p; h %= p;
    if (h == 1) return 0;
    ll m = (ll)ceil(sqrt((double)p)) + 1;
    unordered_map<ll, ll> baby;
    ll cur = h, ginv = binpow(g, p - 2, p);
    for (ll j = 0; j < m; ++j) {
        if (!baby.count(cur)) baby[cur] = j;
        cur = (lll) cur * ginv % p;
    }
    ll gm = binpow(g, m, p);
    cur = 1;
    for (ll i = 1; i <= m; ++i) {
        cur = (lll) cur * gm % p;
```

```
        auto it = baby.find(cur);
        if (it != baby.end()) {
            ll x = i * m - it->second;
            if (x >= 0) return x;
        }
    }
    return -1;
}
```

Listing 28.5: Baby-step giant-step: discrete logarithm mod p.

28.6 Complexity Cheat Sheet

Operation	Algorithm	Complexity
GCD (a, b)	Euclidean	$\mathcal{O}(\log \min(a, b))$
GCD with Bézout	Extended Euclidean	$\mathcal{O}(\log \min(a, b))$
Modular inverse	Fermat / Extended Euclid	$\mathcal{O}(\log p)$
$a^e \bmod m$	Binary exponentiation	$\mathcal{O}(\log e)$
Prime test, $n \le 10^{18}$	Deterministic MR	$\mathcal{O}(\log^3 n)$
Factorisation, $n \le 10^{18}$	MR + Pollard rho	$\widetilde{\mathcal{O}}(n^{1/4})$
Sieve, n primes	Eratosthenes	$\mathcal{O}(n \log \log n)$
Sieve with spf	Linear sieve	$\Theta(n)$
Segmented sieve, $[m, n]$	Sieve + segment	$\mathcal{O}(\sqrt{n} + (n-m) \log \log \sqrt{n})$
Euler $\varphi(n)$ from factors	Factorisation	$\mathcal{O}(\sqrt{n})$
Mobius $\mu(n)$ from factors	Factorisation	$\mathcal{O}(\sqrt{n})$
φ or μ for all $i \le n$	Linear sieve	$\Theta(n)$
Modular square root, $\sqrt{a} \bmod p$	Tonelli–Shanks	$\mathcal{O}(\log^4 p)$
Discrete log $g^x \equiv h \pmod p$	BSGS	$\mathcal{O}(\sqrt{p})$
Discrete log with smooth $p-1$	Pohlig–Hellman	$\mathcal{O}(\sqrt{p_{\max}} \log p)$
Primitive root mod p	Search	$\mathcal{O}(\log^6 p)$ expected
Polynomial multiplication mod p	NTT	$\mathcal{O}(n \log n)$
Polynomial multiplication mod m	MTT (3 NTT + Garner)	$\mathcal{O}(n \log n)$
Polynomial inverse	Newton iteration + NTT	$\mathcal{O}(n \log n)$

Linear recurrence a_n, k small	Matrix exponentiation	$\mathcal{O}(k^3 \log n)$
Linear recurrence a_n, k large	Kitamasa	$\mathcal{O}(k^2 \log n)$ or $\mathcal{O}(k \log k \log n)$
Find recurrence from $2k$ terms	Berlekamp–Massey	$\mathcal{O}(k^2)$
Fibonacci $F_n \bmod m$	Fast doubling	$\mathcal{O}(\log n)$
Lucas binomial $\binom{n}{k} \bmod p$	Lucas's theorem	$\mathcal{O}(\log_p n)$

28.7 NTT-friendly Primes

Prime	**Form**	**Primitive root**	**Max power of two n**
998244353	$119 \cdot 2^{23} + 1$	3	2^{23}
167772161	$5 \cdot 2^{25} + 1$	3	2^{25}
469762049	$7 \cdot 2^{26} + 1$	3	2^{26}
1004535809	$479 \cdot 2^{21} + 1$	3	2^{21}
985661441	$233 \cdot 2^{22} + 1$	3	2^{22}
754974721	$45 \cdot 2^{24} + 1$	11	2^{24}

The standard triple for triple-modulus NTT is $\{998244353, 167772161, 469762049\}$, with product $\approx 7.9 \times 10^{25}$, enough for coefficients up to $\sim 10^{25}$.

28.8 Common Competitive Programming Moduli

Modulus	**Notes**
$10^9 + 7$	The de facto default prime modulus. Fermat inverse via a^{10^9+5}.
$10^9 + 9$	A common second modulus for double-hashing.
998244353	The NTT-friendly prime. $119 \cdot 2^{23} + 1$.
$10^9 + 21$	Another popular prime, used at Codeforces.
2^{64}	Used as the modulus of `ull` multiplication; effectively a rolling hash modulus.

$2^{61}-1$	Mersenne prime, allows fast modular reduction (used in fast hashing).

28.9 Closing Note

This book has walked through every standard technique of competitive-programming number theory, from the elementary division algorithm to the Number-Theoretic Transform and Berlekamp–Massey. The reader who has implemented each template in their own library and worked through the practice problems will have at their disposal the same toolkit that has won many ICPC and Codeforces contests. The next step is practice: pick problems that explicitly mention number theory (Codeforces tags: `number theory`, `math`, `matrices`, `fft`; AtCoder: `Math`, `Convolution`; Project Euler: all problems), and apply the pattern-recognition framework of Chapter 20. After fifty or so such problems, the reflexes become automatic.

The single most important lesson of competitive number theory is this: behind every "modulo 10^9+7" problem is a piece of algebra. The more fluent you are with the algebra—rings, fields, multiplicative functions, convolutions, primitive roots, polynomial rings—the less time you spend searching for the right trick and the more time you spend implementing it. Read, implement, compete, repeat.